UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

FACULTAD DE CIENCIAS E INGENIERÍA

PROYECTO DE VINCULACIÓN

TEMA:

Pensamiento Computacional en Bachilleres graduados del cantón Milagro

TIPO DE PROYECTO:

Transferencia de Conocimiento

HORAS TOTALES:

96 horas

COMPENDIO DE LA ASIGNATURA DE PENSAMIENTO COMPUTACIONAL

DIRECTOR DEL PROYECTO:

Ing. Erick Ruano Lara, MSc. | PhD(c)

DOCENTE ASOCIADO:

Ing. Jhonny Darwin Ortiz Mata, MSc.

PERIODO:

Agosto - Diciembre 2026

Unidad 1: Competencias Digitales para el Entorno Universitario..... 4
Tema 1: Entornos Digitales..... 4
Subtema 1.1: Dominio integral de internet, redes y mensajería efectiva.....4
Subtema 1.2: Técnicas de búsqueda y validación crítica de información académica..... 10
Tema 2: Herramientas Colaborativas..... 17
Subtema 2.1: Almacenamiento y trabajo colaborativo en la nube..... 17
Subtema 2.2: Uso de plataformas y herramientas para la gestión educativa.....20
Cierre de la Unidad 1..... 23
Unidad 2: Fundamentos del Pensamiento Computacional.....26
Tema 1: Fundamentos del Pensamiento Computacional.....26
Subtema 1.1: Concepto de pensamiento computacional y su importancia..... 26
Subtema 1.2: Origen, definición y los cuatro pilares.....29
Tema 2: Enfoque del Pensamiento Computacional..... 36
Subtema 2.1: Procesamiento de la información (Entrada – Proceso – Salida)... 36
Cierre de la Unidad 2..... 40
Unidad 3: Lógica Computacional..... 43
Tema 1: Elementos del Lenguaje Algorítmico..... 43
Subtema 1.1: Tipos de datos, variables, constantes y expresiones..... 43
Subtema 1.2: Operadores aritméticos, relacionales y lógicos..... 46
Tema 2: Pseudocódigo, Flujogramas y Estructuras de Control..... 51
Subtema 2.1: Construcción de pseudocódigo y flujogramas con estructuras secuenciales..... 51
Subtema 2.2: Estructuras de control.....55
Cierre de la Unidad 3..... 64
Unidad 4: Programación Visual con Scratch..... 66
Tema 1: Introducción a Scratch.....66
Subtema 1.1: Instalación, interfaz y lógica de bloques.....66
Subtema 1.2: Secuencias, eventos y movimientos..... 74
Tema 2: Desarrollo de Soluciones Interactivas..... 81
Subtema 2.1: Variables y datos..... 81
Subtema 2.2: Creación de proyectos interactivos..... 86
Cierre de la Unidad 4..... 95

PENSAMIENTO COMPUTACIONAL

Guía Didáctica – Unidad 1

Proyecto de Vinculación – Facultad de Ciencias e Ingeniería

Universidad Estatal de Milagro (UNEMI)

Unidad 1: Competencias Digitales para el Entorno Universitario

Hoy en día, entrar a la universidad no es solo asistir a clases. También implica moverse con confianza en entornos digitales, saber dónde buscar información y comunicarse de forma efectiva usando herramientas tecnológicas. La UNESCO (2020) señala que la pandemia de COVID-19 demostró que las competencias digitales son ahora cruciales para participar plenamente en la sociedad, independientemente de la carrera que uno estudie.

No todos llegamos al mismo punto de partida. Hay estudiantes que ya han usado computadoras y plataformas digitales durante años, y hay otros que están comenzando desde cero. Eso está bien. Esta unidad está diseñada para que, sin importar desde dónde partes, termines con las herramientas necesarias para moverte con seguridad en el mundo digital universitario.

Según Area (2012), las competencias digitales no se limitan a saber hacer clic en un ícono o abrir un archivo. Implican saber buscar información, evaluarla, comunicarla y usarla de manera crítica y responsable. Eso es lo que vamos a construir en esta unidad, paso a paso.

Tema 1: Entornos Digitales

Subtema 1.1: Dominio integral de internet, redes y mensajería efectiva

Para poder estudiar, investigar, colaborar y comunicarse en la universidad, lo primero es entender el entorno digital en el que vamos a movernos. En esta sección vamos aprendiendo qué es Internet, cómo funciona, qué son las redes informáticas y cómo usar la mensajería digital de forma efectiva.

1.1.1 ¿Qué es Internet y cómo funciona?

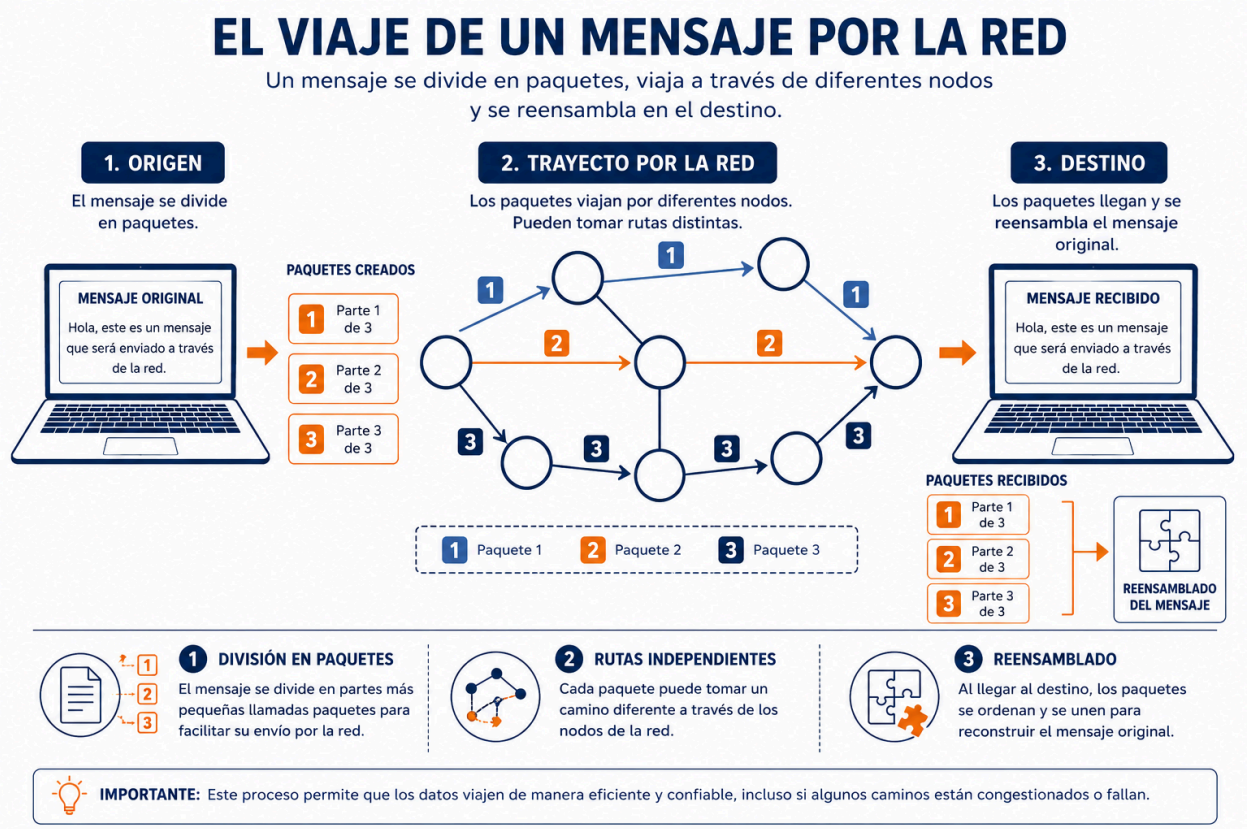
Internet es una red global de computadoras y dispositivos conectados entre sí, capaz de intercambiar información en tiempo real. Aunque la usamos todos los días para ver videos, mandar mensajes o buscar información, pocas veces nos preguntamos cómo funciona por dentro.

Pensemos en el servicio postal. Cuando enviamos una carta, no va directamente de nuestra casa a la del destinatario. Primero pasa por una oficina local, luego por un centro de distribución regional, luego por uno nacional, y eventualmente llega. El proceso toma días. Internet funciona de forma parecida, solo que en lugar de cartas, lo que viaja son datos, y en lugar de días o semanas, el viaje toma fracciones de segundo.

Cuando enviamos un mensaje o abrimos una página web, esa información no viaja entera de un punto a otro. Se divide en pequeños fragmentos llamados paquetes. Cada paquete viaja de forma independiente por la ruta más eficiente disponible en ese momento, y cuando todos llegan al destino, se vuelven a ensamblar en el orden correcto. Según Tanenbaum & Wetherall (2011), este proceso se basa en protocolos, que son como los "reglamentos de tránsito" que todos los dispositivos conectados deben respetar para entenderse entre sí. El más conocido es el TCP/IP, que define cómo se dividen, envían y reensamblan los paquetes.

Para acceder a Internet, necesitamos un proveedor de servicio de internet, conocido como ISP (del inglés Internet Service Provider). En Ecuador, ejemplos de ISP son CNT, Claro, Netlife o Movistar. Ellos son los que "llevan" la señal de internet hasta nuestros hogares o celulares, y desde ahí nosotros nos conectamos a la red global.

Término	¿Qué significa en términos simples?
Paquete de datos	Pequeño fragmento de información que viaja por la red
Protocolo (TCP/IP)	Conjunto de reglas que permite a los dispositivos comunicarse
ISP	Empresa que nos da acceso a Internet (CNT, Claro, Netlife)
Servidor	Computadora potente que almacena y entrega información a los usuarios
Ancho de banda	Capacidad de la conexión, indica qué tan rápido puede viajar la información



1.1.2 ¿Qué es una red informática?

Una red informática es un conjunto de dos o más dispositivos conectados entre sí con el objetivo de compartir información, recursos o servicios. En palabras más simples: si dos computadoras pueden intercambiar archivos entre ellas, ya forman una red.

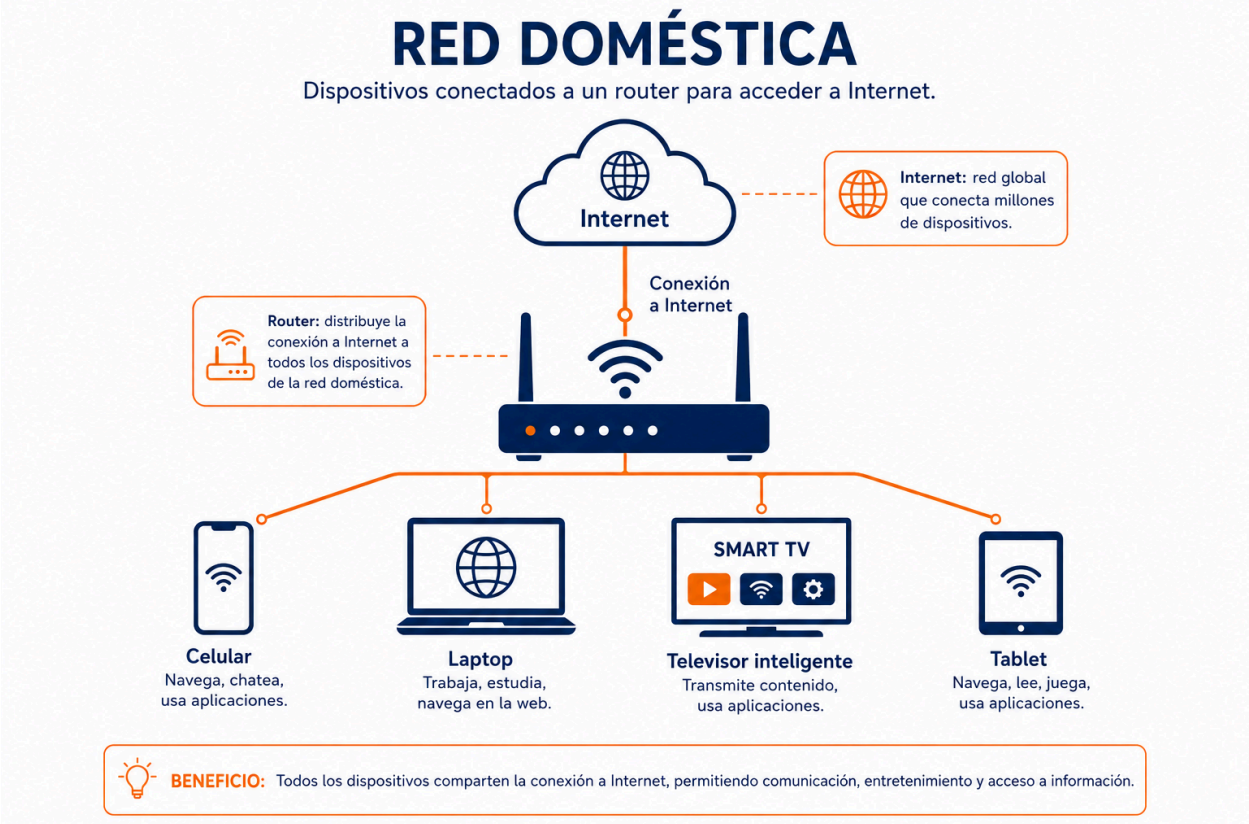
Piensa en el barrio donde vives. Hay calles que conectan las casas, y por esas calles se puede circular de un punto a otro. En una red informática, los cables o señales inalámbricas son esas "calles", y los dispositivos conectados son las "casas". Cuando alguien quiere enviar un archivo, ese archivo "camina" por esas calles hasta llegar a donde debe llegar.

La red de tu casa es un ejemplo cotidiano: el router recibe la señal del ISP y la distribuye entre tu celular, tu laptop, el televisor inteligente y cualquier otro dispositivo conectado. Todos esos aparatos forman una red local. Según Forouzan (2007), las redes se clasifican principalmente por su tamaño y alcance geográfico. No vamos a entrar en detalles técnicos profundos, pero sí conviene conocer los tres tipos más comunes.

Tipo de red	¿Qué abarca?	Ejemplo cotidiano

Red LAN (Local)	Un edificio o campus. Dispositivos cercanos entre sí.	Computadoras del laboratorio universitario
Red WLAN (Inalámbrica)	Red local pero sin cables, usando WiFi.	Red WiFi de la casa o la cafetería
Red WAN (Amplia)	Ciudades, países o el mundo entero.	Internet en su totalidad

El router que tienes en casa es el dispositivo que hace de "guardia de tránsito" de tu red local. Él recibe la señal de Internet del ISP por un lado, y por el otro la distribuye a todos los dispositivos de la casa, ya sea por cable de red o por WiFi. Sin el router, los dispositivos pueden existir, pero no pueden comunicarse entre sí ni acceder a internet.



1.1.3 Navegadores web y motores de búsqueda

Hay una confusión muy común entre estas dos cosas, y vale la pena aclararlo bien desde el principio: el navegador y el motor de búsqueda no son lo mismo.

El navegador es el programa que usamos para acceder a páginas web. Es como el vehículo con el que nos movemos por Internet. Google Chrome, Mozilla Firefox,

Microsoft Edge y Safari son navegadores. Su trabajo es recibir una dirección web (URL) y mostrar el contenido de esa página en la pantalla.

El motor de búsqueda, en cambio, es un sistema que rastrea, indexa y organiza millones de páginas web para que podamos encontrarlas. Google, Bing y DuckDuckGo son motores de búsqueda. Su trabajo es ayudarnos a encontrar qué página visitar, no llevarnos hasta allá. Eso lo hace el navegador.

Categoría	¿Qué hace?	Ejemplos
Navegador web	Muestra el contenido de las páginas web en la pantalla	Chrome, Firefox, Edge, Safari
Motor de búsqueda	Encuentra y ordena páginas web según lo que buscamos	Google, Bing, DuckDuckGo
URL	Dirección única de una página web específica	https://www.unemi.edu.ec

Cuando escribimos una dirección completa en la barra del navegador (como https://www.unemi.edu.ec), el navegador va directo a esa página. Cuando escribimos palabras sueltas (como "pensamiento computacional"), el navegador lo envía al motor de búsqueda que tengamos configurado, y ese motor nos muestra los resultados.

1.1.4 Mensajería digital y correo electrónico efectivo

En la vida universitaria nos comunicamos digitalmente todo el tiempo: WhatsApp, Telegram, Instagram. Pero la comunicación formal con docentes, coordinadores y la institución pasa principalmente por el correo electrónico institucional, y saber usarlo correctamente es una competencia que se espera desde el primer semestre.

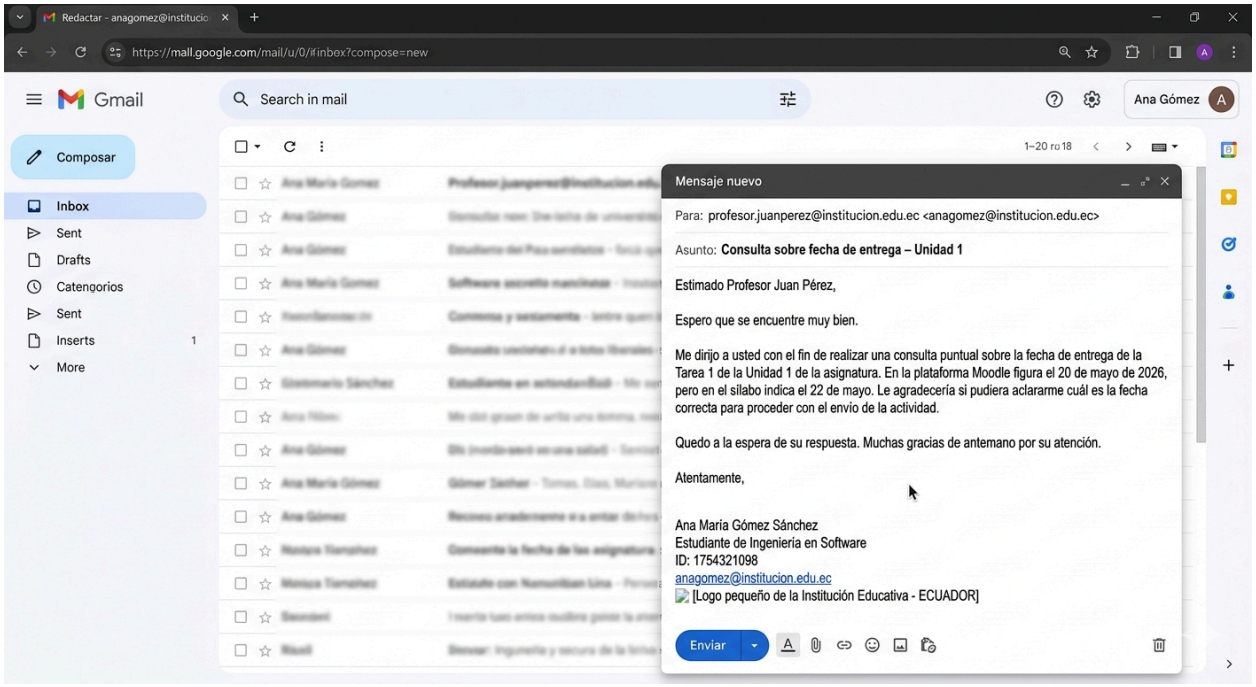
El correo electrónico se parece a una carta formal. No es lo mismo mandarle un mensaje a tu docente que mandarle un mensaje a un amigo. El tono, la estructura y el lenguaje deben adaptarse al contexto. Un mensaje mal redactado puede generar una mala impresión antes de que te conozcan.

Un correo institucional bien redactado generalmente sigue esta estructura:

- Asunto: breve y descriptivo. El docente debe saber de qué trata el correo sin ni siquiera abrirlo. "Consulta sobre tarea – Pensamiento Computacional" es mucho mejor que "Hola" o simplemente "Pregunta".
- Saludo: "Estimado/a Mgs. [apellido]:" o "Buen día, docente:" dependiendo del grado académico.
- Cuerpo del mensaje: primero preséntate brevemente (nombre, carrera, grupo o paralelo), luego expresa tu consulta o solicitud de forma clara, directa y respetuosa.
- Cierre: "Quedo atento/a a su respuesta. Muchas gracias." o "Agradezco de antemano su atención."
- Firma: nombre completo, carrera y semestre o nivel.

> **Ejemplo de correo bien redactado:**
>
> Asunto: Consulta sobre fecha de entrega – Unidad 1
> Estimado Mgs. Ruano:
> Mi nombre es [nombre completo], soy estudiante de Ingeniería en Software, paralelo A. Me dirijo a usted para consultar si la fecha de entrega de la guía de la Unidad 1 sigue siendo el domingo 17 de mayo o si hubo algún cambio.
> Quedo atento a su respuesta. Muchas gracias.
> Atentamente,
> [Nombre completo]
> Ingeniería en Software – Nivel 1

La mensajería instantánea (WhatsApp, Telegram) se usa cuando el docente o el grupo lo habilita para comunicaciones más rápidas e informales. Incluso en esos espacios, mantener un lenguaje respetuoso y claro hace una diferencia. Evitar abreviaturas extremas o emojis en cadena cuando hablamos con docentes es una norma de respeto básica en el entorno universitario.



Preguntas de reflexión

1. ¿Cómo le explicarías la diferencia entre Internet y una red doméstica a alguien que nunca lo ha estudiado?
2. ¿Alguna vez buscaste algo en Internet y no encontraste lo que necesitabas? ¿Qué crees que falló?
3. ¿Qué diferencia hay entre escribirle a un amigo por WhatsApp y escribirle a un docente por correo? ¿Por qué importa esa diferencia?

Actividad práctica

Redacta un correo electrónico dirigido a un docente imaginario solicitando una reunión para aclarar dudas sobre el primer trabajo de la clase. Usa la estructura completa: asunto descriptivo, saludo formal, presentación breve, la solicitud clara y un cierre con firma. Luego intercámbialo con un compañero para que te dé retroalimentación.

Subtema 1.2: Técnicas de búsqueda y validación crítica de información académica

El problema en la actualidad no es encontrar información, sino saber si esa información es correcta, confiable y útil para lo que necesitamos. Según Metzger & Flanagin (2013), evaluar la credibilidad de la información en línea es una de las habilidades más importantes que un estudiante universitario puede desarrollar, especialmente en un entorno donde cualquier persona puede publicar contenido sin ningún tipo de verificación previa.

1.2.1 Búsqueda efectiva de información en Internet

La mayoría de personas buscan información escribiendo una o dos palabras generales en Google, lo que genera millones de resultados en su mayoría irrelevantes. Saber construir búsquedas más precisas ahorra tiempo y mejora mucho la calidad de lo que encontramos.

Google y otros motores de búsqueda aceptan operadores especiales que funcionan como filtros. No es necesario memorizarlos todos, pero conocer los más útiles hace una diferencia real en el trabajo académico.

Operador	¿Qué hace?	Ejemplo de uso
"frase entre comillas"	Busca exactamente esa combinación de palabras	"pensamiento computacional en educación"
site:dominio	Limita los resultados a un dominio específico	site:edu.ec pensamiento computacional
filetype:ext	Busca archivos de un formato específico	algoritmos filetype:pdf
- (guion)	Excluye una palabra de los resultados	algoritmo -python
OR	Busca una cosa u otra	programación OR algoritmos básicos

Una búsqueda bien construida puede verse así: "definición de algoritmo" site:edu filetype:pdf. Esto le dice a Google: busco documentos PDF, de sitios educativos (.edu), que hablen específicamente sobre la definición de algoritmo. El resultado es radicalmente diferente a simplemente escribir "algoritmo".

Otra opción muy útil es Google Académico (scholar.google.com), que filtra automáticamente los resultados hacia publicaciones académicas: artículos científicos, libros, tesis y trabajos de investigación. Es el punto de partida ideal cuando necesitamos fuentes formales para sustentar un trabajo universitario.

1.2.2 Cómo validar si una fuente es confiable

No toda la información en internet es correcta. Encontrar un dato en un blog, en un video de YouTube o en un foro no es lo mismo que encontrarlo en un libro académico o en una publicación científica revisada por expertos.

Cuando encontramos una fuente, hay preguntas sencillas que podemos hacernos antes de usarla:

- ¿Quién la publicó? Un artículo en el repositorio de una universidad o en una revista científica tiene mucho más respaldo que una entrada de blog sin autor identificado. Los dominios .edu, .edu.ec y .gov generalmente indican sitios institucionales o gubernamentales, que suelen tener mayor rigor.
- ¿Tiene autor identificado? Si el texto tiene nombre, apellido, institución y año de publicación, es una buena señal. Si dice "redacción web" o simplemente no tiene autor, hay que desconfiar.
- ¿Cuándo fue publicado? En tecnología y ciencias, una fuente de hace diez años puede estar desactualizada. No siempre invalida el contenido, pero hay que verificar si la información sigue siendo válida hoy.

- ¿Lo confirman otras fuentes? Si una información solo aparece en un lugar de internet y no puedes encontrarla en ningún otro lado, conviene buscarla en otro sitio antes de usarla. Las afirmaciones verdaderas generalmente se pueden encontrar en múltiples fuentes independientes.
- ¿Cómo está escrito? Un texto con errores ortográficos frecuentes, sin citas de respaldo y con lenguaje sensacionalista es una señal de alerta. Un texto bien estructurado, con fuentes citadas y redactado con cuidado, habla bien de su calidad.

Usar Wikipedia como fuente directa en trabajos universitarios no está recomendado, no porque siempre sea incorrecta, sino porque cualquier persona puede editarla sin necesidad de ser experta. Sin embargo, la sección de referencias al final de cada artículo de Wikipedia sí puede ser útil: a veces encontramos ahí libros o artículos formales que podemos ir a buscar directamente.

1.2.3 Fuentes académicas: dónde buscar información confiable

Además de Google Académico, existen repositorios y bases de datos donde se publica investigación verificada por pares (es decir, revisada por otros expertos antes de publicarse). Conocerlos facilita mucho el trabajo cuando necesitamos sustentar un tema con fuentes formales.

Recurso	¿Qué ofrece?	Dirección
Google Académico	Artículos, tesis, libros y publicaciones científicas	scholar.google.com
Repositorio UNEMI	Tesis y trabajos de la propia universidad	repositorio.unemi.edu.ec
LATINDEX	Revistas académicas de Latinoamérica	latindex.org
REDALYC	Revistas científicas de acceso libre en español	redalyc.org
SciELO	Artículos científicos latinoamericanos con acceso libre	scielo.org

No es necesario dominar todos desde el principio. Lo importante es saber que existen y que para trabajos universitarios son mucho más confiables que una búsqueda general en Google.

1.2.4 La inteligencia artificial como herramienta de búsqueda

Las herramientas de inteligencia artificial como ChatGPT, Gemini o Claude son cada vez más usadas por estudiantes para buscar y entender información. Son útiles, pero tienen limitaciones importantes que es necesario conocer.

A diferencia de un buscador tradicional, una IA no "busca" en internet en tiempo real en la mayoría de sus versiones. En cambio, genera respuestas basadas en lo que aprendió durante su entrenamiento, que tiene una fecha de corte. Esto significa que puede cometer errores, confundir datos, o presentar información desactualizada con total confianza y sin advertirte. Russell & Norvig (2020) explican que los modelos de lenguaje generan texto estadísticamente probable, no necesariamente verdadero.

La IA es una herramienta de apoyo, no una fuente. Si una IA te explica un concepto, está bien usarlo como punto de partida para entender el tema. Pero antes de usarlo en un trabajo académico, verifica que la información sea correcta en una fuente formal. Citar a una IA como única fuente en un trabajo universitario no tiene el mismo valor que citar un libro o un artículo científico.

Dicho eso, hay usos donde la IA sí ayuda mucho: entender un concepto difícil con palabras más sencillas, pedir ejemplos cotidianos sobre un tema abstracto, revisar la gramática o estructura de un texto, o generar preguntas de práctica sobre un tema que estás estudiando.

1.2.5 Cómo hacer prompts efectivos

Un prompt es el mensaje o instrucción que le damos a una inteligencia artificial. La calidad de la respuesta que recibimos depende directamente de qué tan claro y completo es el prompt que escribimos. Una IA no puede adivinar lo que realmente necesitamos si no se lo indicamos bien.

Un buen prompt tiene cuatro elementos que trabajan juntos: contexto, tarea, formato y restricciones. No siempre se necesitan los cuatro, pero tenerlos en cuenta ayuda a construir instrucciones mucho más claras.

- Contexto: le dice a la IA quién eres y para qué necesitas la información. Sin contexto, la IA responde de forma genérica.
- Tarea: describe con precisión qué quieres que haga. No es lo mismo "explícame" que "dame un ejemplo" o "compara dos conceptos".
- Formato: indica cómo quieres que te responda. ¿Un párrafo? ¿Una lista? ¿Una tabla? ¿Paso a paso? Si no lo dices, la IA elige por su cuenta.
- Restricciones: establece limitaciones o condiciones especiales. Por ejemplo: sin términos técnicos, en menos de 100 palabras, con ejemplos de la vida diaria, para un estudiante de bachillerato.

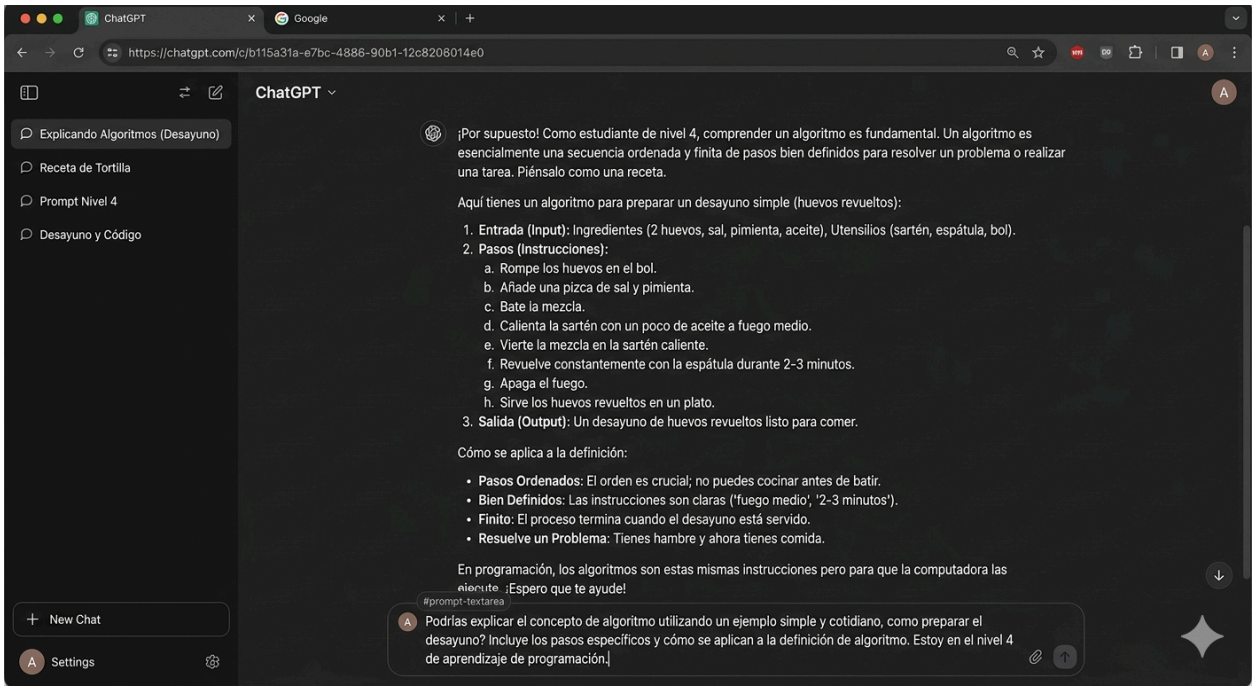
Veamos cómo evoluciona un prompt al agregar estos elementos:

Nivel	Prompt	¿Qué le falta?
1 – Muy pobre	algoritmo	No hay pregunta, no hay contexto, no hay tarea clara
2 – Básico	¿Qué es un algoritmo?	La respuesta puede ser muy técnica o muy breve

3 – Con contexto y tarea	Explica qué es un algoritmo de forma sencilla, como para alguien de bachillerato que nunca ha programado.	Ya es útil, pero no especifica formato ni restricciones adicionales
4 – Completo	Soy estudiante universitario de primer semestre y necesito explicarle a un compañero de bachillerato qué es un algoritmo. Escríbelo en un párrafo corto con un ejemplo de la vida diaria como hacer el desayuno o seguir una receta. Sin términos de programación.	Tiene los cuatro elementos: contexto + tarea + formato + restricción

La diferencia entre el nivel 1 y el nivel 4 es enorme. Con el prompt del nivel 4, la IA sabe para quién es la explicación, qué tipo de respuesta dar, con qué ejemplo ilustrarla y qué evitar. El resultado es mucho más útil y directo.

Elemento	Pregunta que responde	Ejemplo en el prompt del nivel 4
Contexto	¿Quién eres y para qué lo necesitas?	"Soy estudiante universitario de primer semestre..."
Tarea	¿Qué quieres que haga la IA?	"...explicarle a un compañero de bachillerato qué es un algoritmo"
Formato	¿Cómo quieres la respuesta?	"Escríbelo en un párrafo corto con un ejemplo de la vida diaria"
Restricción	¿Qué limitaciones o condiciones hay?	"Sin términos de programación"



Preguntas de reflexión

1. ¿Qué diferencia hace saber usar operadores de búsqueda en Google para un trabajo académico?
2. Si encuentras un dato en internet y en un libro físico, ¿cuál usarías como fuente en un trabajo? ¿Por qué?
3. ¿En qué situaciones sería útil usar una IA para estudiar y en cuáles no sería suficiente?

Actividad práctica

Elige cualquier tema que te interese (puede ser 'inteligencia artificial', 'redes de computadoras' o 'programación'). Hazle a una IA la pregunta más corta posible (una o dos palabras). Luego construye un prompt completo con los cuatro elementos (contexto, tarea, formato, restricción) sobre el mismo tema. Compara las dos respuestas y escribe en tres líneas qué diferencias encontraste.

Cierre del Tema 1

En este primer tema aprendimos cómo funciona internet y por qué los datos viajan en paquetes a través de redes interconectadas. Entendimos qué es una red informática y los tipos más comunes. Aclaramos la diferencia entre navegadores y motores de búsqueda, y aprendimos a redactar correos formales. En la segunda parte del tema profundizamos en cómo buscar información académica usando filtros avanzados, cómo evaluar si una fuente es confiable, cómo usar herramientas de IA de forma responsable y cómo construir prompts efectivos con estructura clara.

Actividad A – Búsqueda con y sin filtros (15 min)

Busca el tema "lógica de programación" en Google de dos formas: primero con esa búsqueda simple, luego con la búsqueda "lógica de programación" site:edu.ec filetype:pdf. Anota cuántos resultados aparecen en cada caso y qué tipo de páginas muestran. ¿Qué diferencias encuentras?

Actividad B – Correo formal (15 min)

> Redacta un correo electrónico dirigido a un docente imaginario solicitando una reunión para aclarar dudas sobre el primer trabajo de la clase. Usa la estructura completa: asunto descriptivo, saludo formal, presentación breve (nombre, carrera, paralelo), la solicitud y cierre con firma.

Tema 2: Herramientas Colaborativas

La universidad implica trabajar en equipo constantemente. Documentos compartidos, presentaciones entre varios, archivos enviados y recibidos, avisos de la plataforma... todo eso requiere manejar herramientas digitales que faciliten el trabajo colaborativo. Según Suárez Guerrero et al. (2013), las herramientas colaborativas en línea permiten que equipos distribuidos geográficamente puedan producir y revisar contenido de forma simultánea, superando las limitaciones del espacio físico.

Subtema 2.1: Almacenamiento y trabajo colaborativo en la nube

2.1.1 ¿Qué es la nube y cómo funciona?

Cuando hablamos de "la nube" en tecnología, no nos referimos a nubes en el cielo. La nube es un conjunto de servidores remotos (computadoras muy potentes) conectados a Internet que almacenan y procesan datos. En lugar de guardar un archivo solo en tu computadora o en una USB, lo guardas en esos servidores, y puedes acceder a él desde cualquier dispositivo con conexión a internet.

Es parecido a tener una bodega de almacenamiento fuera de tu casa, a la que puedes entrar desde cualquier lugar del mundo. No necesitas cargar una llave física encima: con tu usuario y contraseña accedes desde donde sea, en cualquier momento.

Según Mell & Grance (2011) del Instituto Nacional de Estándares y Tecnología de los Estados Unidos (NIST), la computación en la nube permite acceder a recursos informáticos configurables a través de internet de forma conveniente y bajo demanda, sin requerir una gestión técnica profunda por parte del usuario.

Cada vez que usas Google Drive, Gmail, iCloud o incluso Netflix, estás usando la nube. Esos archivos y videos no están guardados en tu celular ni en tu computadora, sino en servidores de esas empresas en algún lugar del mundo, y tú los accedes a través de internet sin notar la diferencia.

2.1.2 Principales servicios de almacenamiento en la nube

Las plataformas de almacenamiento en la nube más usadas en entornos educativos son Google Drive, OneDrive y Dropbox. Las tres tienen propósitos similares pero funcionan de formas distintas y se integran con ecosistemas diferentes. Para el entorno universitario, Google Drive es la más común porque está directamente ligada a Gmail y al resto de aplicaciones de Google.

Servicio	Empresa	Almacenamiento gratuito	Integración principal
Google Drive	Google	15 GB compartidos con Gmail	Docs, Sheets, Slides, Forms, Gmail
OneDrive	Microsoft	5 GB	Word, Excel, PowerPoint, Teams
Dropbox	Dropbox Inc.	2 GB	Compatible con múltiples apps

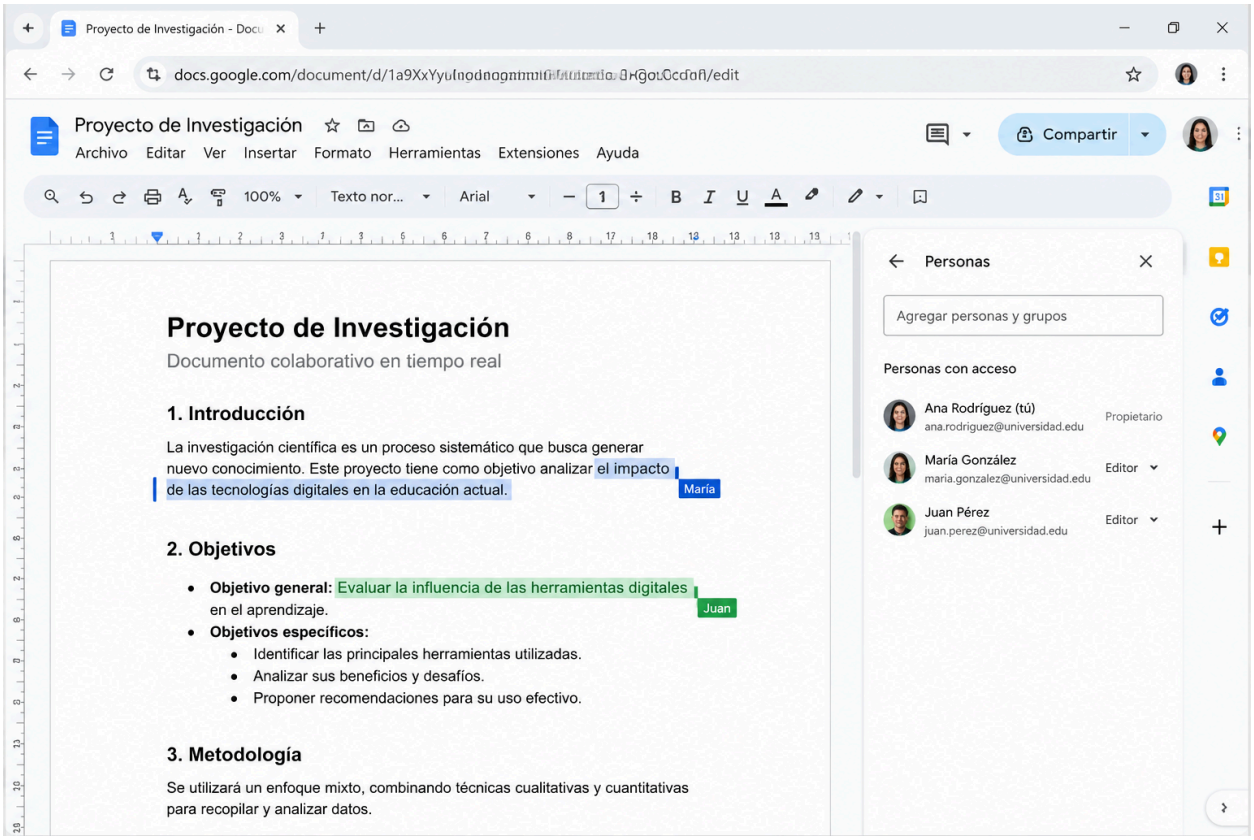
Con estas herramientas podemos guardar archivos de texto, hojas de cálculo, presentaciones, imágenes y más. Lo más importante es que esos archivos son accesibles desde cualquier dispositivo con internet: si guardas algo en Google Drive desde el celular en la mañana, puedes abrirlo esa misma tarde desde la computadora del laboratorio universitario sin llevar ningún pendrive encima.

2.1.3 Trabajo colaborativo en tiempo real

Una de las ventajas más poderosas de herramientas como Google Drive es la posibilidad de trabajar en un mismo documento con varias personas al mismo tiempo. Cuando compartimos un documento de Google Docs con un compañero, ambos podemos abrirlo y editarlo simultáneamente. Los cambios que hace cada persona aparecen en tiempo real para todos los que tienen el documento abierto.

Esto cambia completamente la dinámica de los trabajos en grupo. Ya no es necesario enviarse el archivo por WhatsApp, esperar a que el otro lo edite, que te lo devuelva, que tú lo edites y volvérselo a enviar. Todo ocurre en el mismo lugar y al mismo tiempo, con cada persona trabajando en su parte sin pisar el trabajo de los demás.

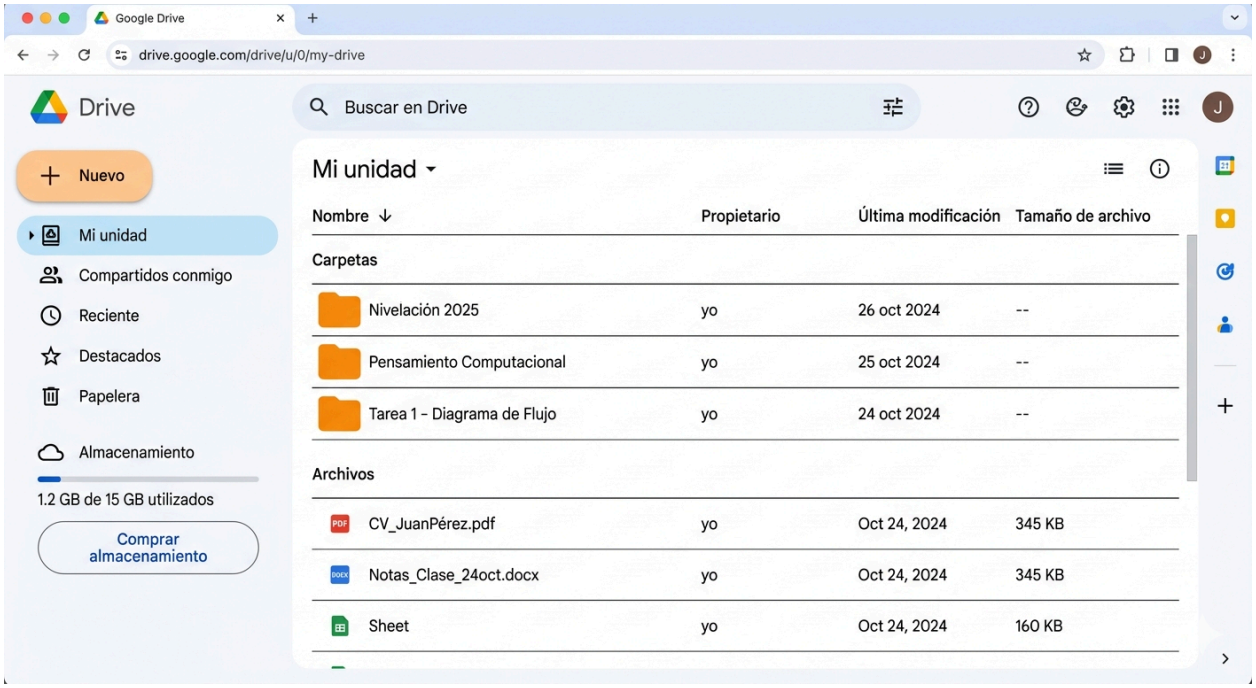
Además, Google Docs guarda automáticamente un historial de versiones. Si alguien borra algo por error, o quieres ver cómo estaba el documento antes de ciertos cambios, puedes revisar ese historial y restaurar versiones anteriores. Es como tener un Ctrl+Z que nunca vence y que recuerda todos los cambios desde que el documento fue creado.



2.1.4 Buenas prácticas al usar la nube

Tener almacenamiento en la nube no es suficiente si los archivos están desorganizados. Una buena organización desde el primer día ahorra mucho tiempo después, cuando la carga de materias aumenta y los archivos se acumulan.

- Usar carpetas con nombres claros: organiza por semestre y materia. Ejemplo: "2025A – Nivelación – Pensamiento Computacional". Eso facilita encontrar cualquier archivo rápido.
- Nombrar los archivos de forma descriptiva: "Tarea1_DiagramaFlujo" dice mucho más que "Documento1" o "Sin título".
- Revisar los permisos antes de compartir: hay diferencia entre "puede editar", "puede comentar" y "puede ver". Usar el permiso correcto evita cambios accidentales o no autorizados en tus archivos.
- Tener copias de respaldo para trabajo crítico: aunque la nube rara vez falla, para entregables importantes tener una copia descargada en tu computadora es una precaución inteligente.



Preguntas de reflexión

1. ¿Qué ventajas tiene guardar archivos en la nube en comparación con guardarlos solo en una USB?
2. Si estás trabajando en grupo en un documento de Google Docs y alguien borra accidentalmente una sección entera, ¿cómo podrías recuperarla?
3. ¿Cómo organizarías tu Google Drive para llevar bien todas tus materias este semestre?

Actividad práctica

En Google Drive, crea una carpeta con tu nombre. Dentro de ella, crea un documento de Google Docs titulado 'Lo que aprendí hoy'. Escribe en él un párrafo sobre cualquier cosa que aprendiste en esta unidad. Comparte el documento con un compañero usando permiso de 'puede comentar', pídele que deje un comentario con su opinión, y responde ese comentario dentro del documento.

Subtema 2.2: Uso de plataformas y herramientas para la gestión educativa

2.2.1 ¿Qué es una plataforma educativa virtual?

Una plataforma educativa virtual es un sistema en línea diseñado específicamente para organizar el proceso de aprendizaje. Permite que los docentes publiquen materiales, asignen actividades, reciban trabajos, registren calificaciones y se comuniquen con los estudiantes, todo en un mismo lugar.

En el mundo universitario, estas plataformas se conocen como LMS (Learning Management System) o Sistema de Gestión de Aprendizaje. No son redes sociales ni simples repositorios de archivos: están diseñadas pensando en las necesidades específicas de un proceso educativo formal, con flujos de trabajo estructurados para docentes y estudiantes.

Para el estudiante, el LMS es el "centro de operaciones" de cada materia. Allí se encuentran el sílabo, las lecturas, los foros de discusión, las actividades evaluadas y las calificaciones. Ignorarlo o revisarlo solo cuando ya hay una urgencia es uno de los errores más comunes durante el primer año universitario, y generalmente tiene consecuencias en las notas.

2.2.2 Google Classroom y Moodle

Las dos plataformas más usadas en universidades del Ecuador son Google Classroom y Moodle. Tienen propósitos similares pero funcionan de formas distintas.

Google Classroom está integrado con el ecosistema de Google. Usa la misma cuenta de Gmail, se conecta directamente con Google Drive, y tiene una interfaz bastante intuitiva, similar a una red social pero orientada a tareas y clases. Cada materia es un "aula" donde el docente publica anuncios, materiales y actividades. Los estudiantes entregan sus trabajos directamente en la plataforma.

Moodle es un sistema más robusto y personalizable, muy usado en universidades a nivel mundial. Permite crear cuestionarios con corrección automática, foros estructurados por temas, lecciones modulares con progreso controlado, y tiene muchas más opciones de configuración que Classroom. La interfaz puede parecer más compleja al principio, pero una vez que se aprende la estructura, es muy potente para el aprendizaje organizado.

Plataforma	Fortaleza principal	Curva de aprendizaje	Dónde se usa
Google Classroom	Sencillo, rápido, bien integrado con Google	Baja: interfaz intuitiva	Colegios, primer nivel universitario
Moodle	Completo, muy configurable, robusto	Media: requiere explorar la plataforma	Universidades, cursos online estructurados

2.2.3 El correo institucional como herramienta académica

Al ingresar a la universidad, se te asigna una cuenta de correo institucional. Esta cuenta no es opcional ni secundaria: es el canal oficial de comunicación entre la institución y el estudiante. Avisos importantes, confirmaciones de matrícula, novedades de docentes y notificaciones administrativas llegan por ahí, no a tu Gmail personal.

Uno de los errores más comunes es ignorar esta cuenta porque "nadie la revisa". Eso cambia con la vida universitaria. Si el docente anuncia un cambio de horario o el envío de material adicional por correo institucional y no lo revisas, es responsabilidad del estudiante, no del docente. La institución asume que tienes acceso a esa cuenta y que la revisas.

Recomendaciones básicas: revisar el correo institucional al menos una vez al día, configurarlo en el celular para recibir notificaciones, y mantener una carpeta por materia para organizar los correos importantes.

Además, el correo institucional da acceso a servicios universitarios que no están disponibles con un Gmail personal: bases de datos académicas con licencia institucional, herramientas con descuento educativo, plataformas de aprendizaje e incluso paquetes de Office de Microsoft de forma gratuita a través de convenios universitarios.

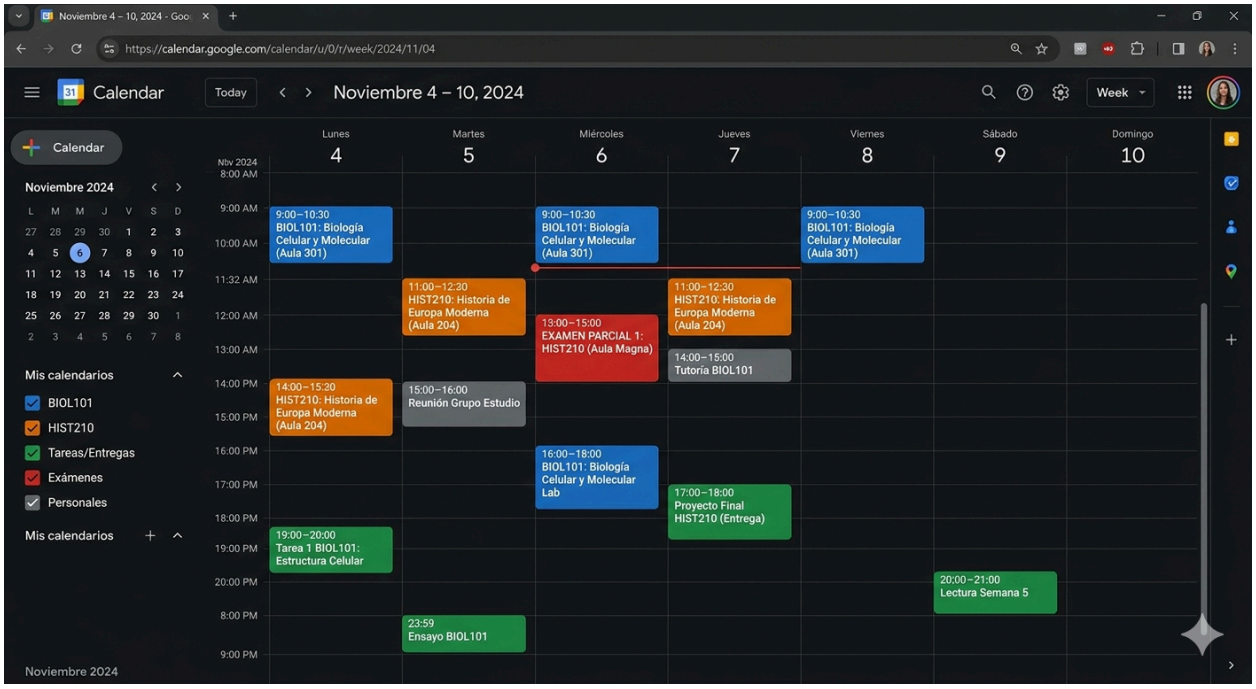
2.2.4 Organización digital del estudiante universitario

Gestionar el tiempo y las tareas universitarias es uno de los mayores retos del primer año. Tener las herramientas digitales bien organizadas ayuda a que no se acumulen pendientes ni se pierdan fechas importantes.

Google Calendar (o cualquier calendario digital) es una herramienta simple pero muy poderosa para esto. Puedes agregar los horarios de clase, las fechas de entrega de trabajos y los exámenes parciales. Muchas plataformas LMS permiten integrar el calendario directamente, de forma que al agregar una tarea en Classroom, aparece automáticamente en tu Calendar.

Otra práctica útil es mantener una lista de pendientes, ya sea en una app como Google Keep, en un documento de Google Docs o incluso en papel. Lo importante es tener en un solo lugar qué falta hacer, cuándo vence y en qué estado está cada cosa. Sin ese sistema, es fácil que algo importante se pierda entre el ruido del día a día.

Usar recordatorios automáticos también ayuda. Google Calendar permite configurar alertas con uno o dos días de anticipación antes de una entrega importante. Ese tipo de sistema pequeño, bien configurado desde el primer día, evita los momentos de pánico de "se me olvidó la tarea".



Preguntas de reflexión

1. ¿Cuál sería la diferencia en tu experiencia universitaria si nunca revisas el correo institucional ni la plataforma del aula virtual?
2. ¿Qué herramientas de las que vimos (Drive, Classroom, Calendar) ya usas y cuáles son nuevas para ti?
3. ¿Cómo organizarías tu Google Drive y Google Calendar para este semestre de forma que todo esté claro y accesible?

Actividad práctica

Explora la plataforma educativa que usa tu universidad (Google Classroom o Moodle). Localiza al menos: el sílabo de una materia, la primera actividad asignada y la sección de calificaciones. Anota en qué parte de la plataforma encontraste cada cosa. Luego, en Google Calendar, crea un evento para la fecha de entrega de tu próxima tarea y activa un recordatorio con un día de anticipación.

Cierre de la Unidad 1

En esta unidad hemos visto los fundamentos del entorno digital universitario. Vimos cómo funciona Internet y por qué los datos viajan en paquetes a través de redes interconectadas. Entendimos qué es una red informática y los tipos más comunes. Aclaramos la confusión entre navegadores y motores de búsqueda. Aprendimos a redactar correos formales con estructura correcta y a diferenciar contextos de comunicación digital.

En la segunda parte del Tema 1 profundizamos en cómo buscar información académica de calidad usando operadores de búsqueda avanzados, cómo evaluar si una fuente es confiable, dónde encontrar fuentes académicas formales, y cómo usar herramientas de inteligencia artificial de forma responsable. Aprendimos también a construir prompts efectivos con los cuatro elementos clave: contexto, tarea, formato y restricciones.

En el Tema 2 conocimos qué es la nube y cómo aprovecharla para almacenar archivos y trabajar de forma colaborativa en tiempo real. Exploramos las plataformas educativas más usadas en el entorno universitario, entendimos la importancia del correo institucional y aprendimos estrategias de organización digital que facilitan el primer año de carrera.

Tareas de la Unidad 1

Tarea 1 – Búsqueda y evaluación de fuentes (15–25 minutos)

Elige un tema relacionado con tecnología (puede ser 'inteligencia artificial', 'redes de computadoras' o 'programación en bloques'). Realiza dos búsquedas: una general en Google y otra usando al menos dos operadores de búsqueda (site:edu.ec, filetype:pdf, comillas). Encuentra una fuente confiable y una poco confiable. En media página compara ambas fuentes respondiendo: ¿quién la publicó?, ¿tiene autor identificado?, ¿cuándo fue publicada? y ¿cómo está escrita?

Tarea 2 – Trabajo colaborativo en Google Drive (20–30 minutos)

En grupos de dos, creen una carpeta compartida en Google Drive. Dentro, creen un documento de Google Docs titulado 'Lo que aprendimos en la Unidad 1'. Cada integrante debe escribir al menos tres cosas que aprendió y una duda que todavía tiene. Usen la función de comentarios para responder a lo que escribió el otro. Al finalizar, el documento debe tener evidencia de que ambos trabajaron (verificable en el historial de versiones del documento).

PENSAMIENTO COMPUTACIONAL

Guía Didáctica – Unidad 2

Proyecto de Vinculación – Facultad de Ciencias e Ingeniería

Universidad Estatal de Milagro (UNEMI)

Unidad 2: Fundamentos del Pensamiento Computacional

Hasta ahora aprendimos a movernos en el entorno digital: buscar información, usar herramientas colaborativas, comunicarnos correctamente. En esta unidad damos un paso más hacia lo que realmente sostiene esta asignatura: el pensamiento computacional.

Según Wing (2006), el pensamiento computacional es un conjunto de habilidades cognitivas que permiten formular problemas y expresar sus soluciones de formas que pueden ser ejecutadas por un computador, pero también por un ser humano. Esta aclaración es importante: el pensamiento computacional no es exclusivo de quienes van a programar. Es una forma de pensar, de ordenar ideas, de atacar problemas, que resulta útil en cualquier carrera y en la vida cotidiana.

En esta unidad vamos a trabajar esa forma de pensar desde dos ángulos. Primero, entendiendo qué es, de dónde viene y cuáles son sus cuatro pilares fundamentales. Segundo, aprendiendo cómo cualquier sistema, computacional o no, procesa información siguiendo el modelo de Entrada, Proceso y Salida.

Tema 1: Fundamentos del Pensamiento Computacional

Subtema 1.1: Concepto de pensamiento computacional y su importancia

Antes de definir el pensamiento computacional formalmente, vale la pena hacernos una pregunta: ¿qué tienen en común un médico que diagnostica una enfermedad paso a paso, un chef que sigue una receta, un ingeniero que diseña un sistema y un estudiante que organiza su semana para no perder ninguna entrega?

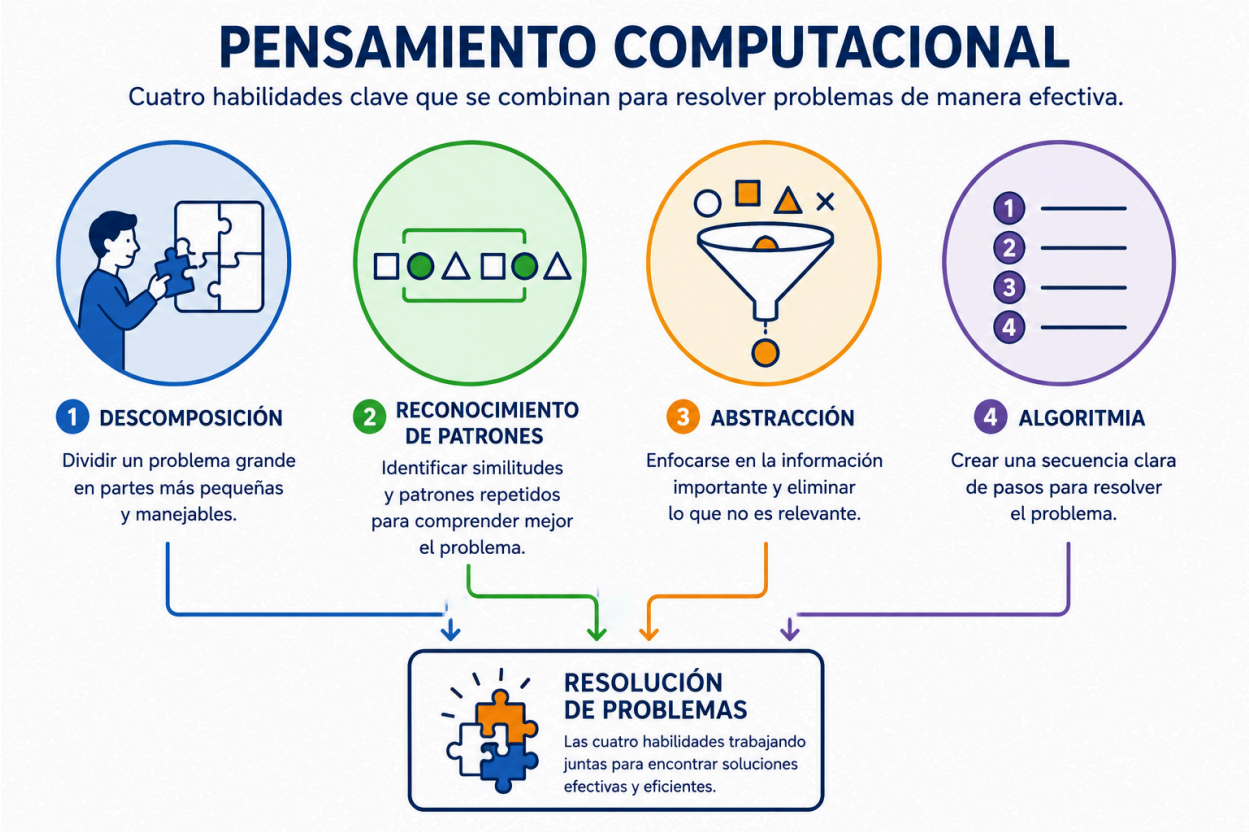
Todos están usando, sin saberlo, pensamiento computacional. Están descomponiendo un problema grande en partes manejables, identificando patrones, ignorando lo que no importa y siguiendo una secuencia lógica de pasos.

1.1.1 ¿Qué es el pensamiento computacional?

Wing (2006) lo define como un proceso cognitivo que implica resolver problemas, diseñar sistemas y comprender el comportamiento humano utilizando los conceptos fundamentales de la informática. Dicho en términos más simples: es una forma estructurada de pensar para resolver cualquier tipo de problema, ya sea con una computadora de por medio o no.

Lo que diferencia al pensamiento computacional de simplemente "pensar" es que tiene una estructura. No es solo intuición ni improvisación. Es organizar las ideas, dividir lo complejo, encontrar patrones y trazar un camino claro desde el problema hasta la solución.

Piensa en cuando necesitas armar un mueble con instrucciones en un idioma que no entiendes. Lo que haces instintivamente es: separar las piezas por tipo (descomposición), identificar cuáles se repiten o se parecen (reconocimiento de patrones), ignorar la información del manual que no aporta como el nombre del mueble en otros idiomas (abstracción) y seguir los pasos numerados del diagrama (algoritmia). Eso, aunque no lo llames así, es pensamiento computacional.



1.1.2 ¿Por qué es importante el pensamiento computacional?

Una de las razones por las que esta asignatura aparece en todas las carreras universitarias, desde medicina hasta administración de empresas, no es para que todos aprendan a programar. Es porque el pensamiento computacional desarrolla una capacidad que tiene valor en cualquier área: saber cómo atacar un problema complejo de forma ordenada.

Según Grover & Pea (2013), el pensamiento computacional desarrolla habilidades que van más allá de la tecnología: la capacidad de analizar información, detectar errores en un razonamiento, automatizar tareas repetitivas y diseñar soluciones escalables. Estas son habilidades que un médico necesita al elaborar un protocolo de atención, que un administrador necesita al diseñar un proceso logístico y que un educador necesita al estructurar un plan de clases.

Además, vivimos en un mundo donde la tecnología está en todas partes. Entender cómo piensan los sistemas que usamos diariamente, aunque sea de forma general, nos convierte en usuarios más críticos, más seguros y más capaces de adaptarnos a los cambios.

1.1.3 Pensamiento computacional en la vida cotidiana

No necesitamos una computadora para usar pensamiento computacional. Lo aplicamos constantemente sin darnos cuenta. Reconocerlo en situaciones cotidianas es el primer paso para desarrollarlo de forma consciente.

Cuando planificas tu semana universitaria estás haciendo pensamiento computacional: identificas todas las tareas (descomposición), ves cuáles tienen el mismo tipo de trabajo o la misma fecha límite (reconocimiento de patrones), decides qué información es relevante para organizar tu tiempo y qué no (abstracción) y creas un orden de prioridades con pasos concretos (algoritmia).

Cuando buscas por qué tu celular dejó de cargar también lo haces: divides el problema en partes posibles (¿el cable? ¿el cargador? ¿el puerto? ¿la batería?), pruebas una a la vez siguiendo una secuencia lógica, y descartas las que no aplican. Eso es diagnóstico con estructura computacional, aunque nunca abras una línea de código.

Situación cotidiana	Cómo aplica el pensamiento computacional
Preparar una receta de cocina	Seguir pasos en orden, con ingredientes y cantidades exactas (algoritmo)
Buscar algo perdido en casa	Dividir la casa en zonas y revisar una por una (descomposición)
Organizar una mochila para la universidad	Ignorar lo innecesario, llevar solo lo relevante (abstracción)
Notar que siempre llegas tarde los lunes	Identificar un patrón repetido para solucionarlo (reconocimiento de patrones)
Armar un horario semanal	Crear una secuencia de pasos para cumplir todas las tareas (algoritmo)

1.1.4 El pensamiento computacional en todas las carreras

La UNEMI incluyó pensamiento computacional como asignatura transversal en todas sus carreras, no solo en las de tecnología. Eso tiene sentido cuando entendemos que esta forma de pensar no es patrimonio de los programadores.

En medicina, el diagnóstico diferencial sigue exactamente la lógica del pensamiento computacional: se descompone el conjunto de síntomas, se buscan patrones comunes entre enfermedades posibles, se abstrae la información relevante del historial del paciente y se construye un algoritmo de descarte hasta llegar al diagnóstico. En administración, diseñar un proceso de atención al cliente o una cadena de suministro

requiere exactamente el mismo tipo de razonamiento estructurado. En educación, elaborar una planificación de clase que funcione para distintos tipos de estudiantes también.

Entender esto desde el principio ayuda a no ver esta asignatura como algo ajeno a la carrera que elegiste, sino como una herramienta de pensamiento que vas a usar, de una forma u otra, a lo largo de toda tu vida profesional.

Preguntas de reflexión

1. ¿En qué momento del día de hoy aplicaste pensamiento computacional sin saberlo?
2. ¿Por qué crees que una carrera como medicina o administración necesita que sus estudiantes desarrollen pensamiento computacional?
3. ¿Qué diferencia hay entre "pensar" para resolver un problema y usar pensamiento computacional estructurado?

Actividad práctica

Piensa en una situación de tu vida cotidiana que involucre resolver un problema (puede ser algo tan simple como decidir qué comer, organizar una salida con amigos o estudiar para un examen). Escribe en un párrafo cómo resolverías ese problema aplicando los cuatro pilares del pensamiento computacional: descomposición, reconocimiento de patrones, abstracción y algoritmia. No necesitas nombrar los pilares explícitamente, solo aplicarlos.

Subtema 1.2: Origen, definición y los cuatro pilares

El pensamiento computacional no es un concepto nuevo aunque el término se popularizó hace relativamente poco. Entender de dónde viene nos ayuda a comprender por qué se organiza en cuatro pilares específicos y cómo estos cuatro pilares trabajan juntos para resolver cualquier tipo de problema.

1.2.1 Origen del pensamiento computacional

Aunque el uso del pensamiento lógico para resolver problemas es tan antiguo como la matemática misma, el término "pensamiento computacional" tal como lo usamos hoy fue introducido y popularizado por Jeannette Wing, investigadora y vicepresidenta de Microsoft Research, en un artículo publicado en 2006 en la revista Communications of the ACM, una de las publicaciones más importantes en ciencias de la computación.

Wing (2006) argumentó que el pensamiento computacional debía ser una habilidad fundamental para todos los seres humanos del siglo XXI, no solo para los científicos de la computación. Su propuesta fue que esta forma de pensar debía integrarse en la

educación desde etapas tempranas, igual que se integra la lectura, la escritura o las matemáticas.

Desde ese artículo seminal, investigadores de todo el mundo comenzaron a trabajar en cómo definir con más precisión qué habilidades componen el pensamiento computacional. Con el tiempo se fue consolidando el modelo de los cuatro pilares que vemos hoy, aunque distintos autores presentan variaciones en cómo los nombran o agrupan.

Año	Hito en la historia del pensamiento computacional
1960s–80s	Seymour Papert usa el computador como herramienta para enseñar a pensar (Logo)
2006	Jeannette Wing publica su artículo seminal en Communications of the ACM
2008–2012	Instituciones educativas de EE.UU. y Europa integran CT en currículos escolares
2010s	Organizaciones como Google y Code.org promueven CT para todos los niveles
2020s	Universidades latinoamericanas incorporan CT como asignatura transversal obligatoria

1.2.2 Descomposición

La descomposición es el primer pilar y, en muchos sentidos, el punto de entrada para resolver cualquier problema complejo. Consiste en dividir un problema grande o una tarea complicada en partes más pequeñas, manejables y comprensibles.

La razón por la que esto funciona es simple: un problema que parece imposible de resolver de un solo golpe se vuelve mucho más atacable cuando lo dividimos en piezas. Cada pieza se puede analizar, resolver y combinar con las demás para llegar a la solución completa.

Pensemos en organizar una fiesta de cumpleaños. Si lo vemos como una sola tarea gigante, puede abrumarnos. Pero si lo descomponemos: invitados, lugar, decoración, comida, música, fecha y hora, cada parte se convierte en un problema mucho más pequeño que podemos resolver por separado. Eso es descomposición.

En el mundo de la programación, esto se traduce en dividir un programa grande en funciones o módulos más pequeños, cada uno con una responsabilidad clara. Pero no necesitamos programar para aplicar este pilar: cualquier proyecto, trabajo de grupo o tarea compleja se beneficia de ser descompuesta antes de atacarla.

> **Ejemplo práctico – Descomposición**
>
> Problema: 'Debo preparar una presentación para la clase del viernes.'En lugar de verlo como una tarea única, lo descomponemos:

1. Definir el tema y el objetivo de la presentación
2. Buscar fuentes confiables sobre el tema
3. Estructurar el contenido (introducción, desarrollo, conclusión)
4. Crear las diapositivas
5. Practicar la exposición
6. Preparar posibles preguntas del docente

Cada paso es un problema pequeño con una solución clara.

1.2.3 Reconocimiento de patrones

El reconocimiento de patrones es la habilidad de identificar similitudes, repeticiones, tendencias o regularidades dentro de un problema o entre distintos problemas. Cuando reconocemos un patrón, podemos reutilizar una solución que ya funcionó antes sin tener que empezar desde cero.

Nuestro cerebro hace esto de forma natural. Cuando aprendemos a sumar con números de un dígito, reconocemos el patrón y lo aplicamos a números de dos dígitos, tres dígitos y así sucesivamente. No aprendemos cada suma individualmente: aprendemos el patrón y lo escalamos.

En el contexto computacional, el reconocimiento de patrones es lo que permite crear loops (bucles) en programación: si un conjunto de instrucciones debe repetirse diez veces, no las escribimos diez veces. Reconocemos que hay un patrón repetitivo y creamos una estructura que lo ejecute automáticamente. Pero nuevamente, este pilar existe mucho antes de la computación.

Un ejemplo cotidiano: si cada vez que llueve llegas tarde al laboratorio porque el piso del pasillo está resbaloso y te toca caminar más despacio, has reconocido un patrón. Lluvia → piso resbaloso → llego tarde. Con ese patrón identificado, puedes tomar una decisión antes de que el problema ocurra, como salir más temprano los días de lluvia.

Situación	¿Qué patrón se identifica?	¿Qué permite hacer?
Tres estudiantes hacen la misma pregunta sobre el mismo subtema	El concepto no quedó claro en clase	Volver a explicarlo con otro enfoque
Una tarea siempre toma el doble del tiempo estimado	Se subestima el esfuerzo en la planificación	Agregar margen de tiempo al planificar
Una fórmula matemática se usa igual en distintos ejercicios	Hay una estructura repetible	Aplicar la misma lógica sin reaprender
Un programa falla siempre que el usuario no ingresa un número	Falta validación de entrada de datos	Agregar una verificación antes de procesar

1.2.4 Abstracción

La abstracción es, quizás, el pilar más difícil de entender al principio, pero también uno de los más poderosos. Consiste en enfocarse únicamente en la información relevante para resolver un problema, ignorando los detalles que no aportan a la solución.

No es que esos detalles no existan o no importen en general, simplemente no importan para el problema específico que estamos resolviendo en este momento. La abstracción nos permite simplificar la complejidad del mundo real para poder trabajar con ella.

Un mapa es uno de los mejores ejemplos de abstracción. El mundo real tiene edificios, árboles, cables eléctricos, personas, texturas en el suelo, cambios de temperatura. Un mapa no muestra nada de eso. Muestra solo lo que importa para el propósito del mapa: calles, nombres, distancias relativas, puntos de referencia. Todo lo demás fue abstraído, es decir, eliminado conscientemente porque no aporta a la función de encontrar una ruta.

Otro ejemplo: cuando le dices a alguien "prepara un café", esa persona no necesita saber la composición química del agua, la variedad exacta del grano de café ni la altitud a la que fue cultivado. Esa información existe, pero fue abstraída porque no es relevante para el proceso de preparar el café. Según Denning & Tedre (2019), la abstracción es el mecanismo que permite a los sistemas computacionales y a los seres humanos manejar la complejidad sin verse abrumados por ella.

Ejemplo práctico – Abstracción

Imagina que debes calcular el promedio de calificaciones de un estudiante.Información disponible del estudiante:- Nombre completo, edad, dirección, carrera, nombre de los padres, foto de perfil, número de cédula, calificaciones de cada materia, actividades extracurriculares, correo electrónico.Información relevante para calcular el promedio:- Solo las calificaciones de cada materia.Todo lo demás (nombre, dirección, foto, etc.) se abstrae. No lo eliminamos del sistema, simplemente lo ignoramos para este problema específico.

1.2.5 Algoritmia

La algoritmia es el cuarto pilar y el que cierra el ciclo: consiste en diseñar una secuencia de pasos precisos, ordenados y finitos que llevan desde un problema hasta su solución. Un algoritmo es exactamente eso: un conjunto de instrucciones claras que, si se siguen correctamente, producen siempre el mismo resultado.

La palabra "algoritmo" viene del nombre del matemático persa Al-Juarismi, quien en el siglo IX escribió tratados matemáticos que describían procedimientos paso a paso para resolver ecuaciones. Cuando sus obras fueron traducidas al latín, su nombre latinizado "Algorismus" pasó a describir ese tipo de procedimientos secuenciales, según Knuth (1997).

Lo importante para entender ahora es que un algoritmo no es exclusivo de la programación. Una receta de cocina es un algoritmo. Las instrucciones para montar un mueble son un algoritmo. El proceso que sigue un médico para recetar un

medicamento es un algoritmo. Lo que tienen en común: son instrucciones precisas, en un orden específico, con un inicio y un fin, que producen un resultado predecible.

Las tres características clave de un buen algoritmo son: que sea preciso (cada paso debe ser claro, sin ambigüedad), que sea finito (debe terminar en algún punto, no repetirse infinitamente) y que sea efectivo (debe producir el resultado esperado si se siguen los pasos correctamente).

Algoritmo	¿Dónde está el algoritmo?	Características que cumple
Receta para preparar arroz	Los pasos del 1 al 8 en orden exacto	Preciso, finito, produce siempre el mismo resultado
Instrucciones de emergencia en un avión	El procedimiento paso a paso ante una evacuación	Claro, ordenado, sin ambigüedad
Proceso de matrícula universitaria	Pasos a seguir: inscripción, pago, confirmación	Finito, con condiciones (si falla el pago, volver al paso anterior)
Cómo desbloquear un celular con patrón	Dibujar el patrón correcto para acceder	Preciso (solo ese patrón funciona), finito

Los cuatro pilares en conjunto

Estos cuatro pilares no trabajan de forma aislada. Cuando resolvemos un problema usando pensamiento computacional, generalmente los usamos todos en distintos momentos del proceso:
1. Descomposición: dividir el problema en partes manejables.
2. Reconocimiento de patrones: identificar qué partes se parecen o se repiten.
3. Abstracción: filtrar la información relevante de cada parte.
4. Algoritmia: diseñar los pasos concretos para resolver cada parte.
El resultado: una solución estructurada, clara y reproducible.

Preguntas de reflexión

1. ¿Cuál de los cuatro pilares crees que usas más naturalmente en tu vida cotidiana? ¿Por qué?
2. Cuando dices que alguien 'es muy organizado para resolver problemas', ¿cuál de los cuatro pilares describe mejor esa habilidad?
3. ¿Por qué crees que el reconocimiento de patrones es tan importante en computación? Piensa en un ejemplo donde reconocer un patrón ahorraría trabajo.

Actividad práctica

Elige un proceso de tu carrera o de la vida universitaria (puede ser 'cómo prepararse para un examen parcial', 'cómo hacer un trabajo en grupo' o 'cómo registrar las materias al inicio del semestre'). Aplica los cuatro pilares explícitamente:– Descomposición: divide el proceso en al menos cuatro pasos o partes.– Reconocimiento de patrones: identifica qué pasos se repiten o son similares entre sí.– Abstracción: señala qué información es relevante y qué puedes ignorar para completar el proceso.– Algoritmia: escribe el procedimiento final en pasos numerados y ordenados.Entrega el resultado como una lista numerada con los cuatro secciones diferenciadas.

Cierre del Tema 1

En este primer tema construimos los fundamentos conceptuales del pensamiento computacional. Vimos de dónde viene el término, por qué es importante más allá de la tecnología y cómo se aplica en la vida cotidiana. Luego profundizamos en los cuatro pilares que lo componen: descomposición para dividir lo complejo, reconocimiento de patrones para identificar lo que se repite, abstracción para quedarnos solo con lo relevante y algoritmia para diseñar una solución paso a paso.

La clave es entender que estos cuatro pilares no son pasos independientes: son lentes distintos con los que mirar el mismo problema, y generalmente los usamos todos en conjunto para llegar a una solución bien estructurada.

Actividad A – Análisis de un problema cotidiano (20 min)

En grupos de dos, elijan un problema cotidiano sencillo (puede ser 'cómo llegar a tiempo a clase', 'cómo preparar el desayuno en 10 minutos' o 'cómo organizar una reunión de grupo virtual'). Apliquen los cuatro pilares por escrito y presenten su análisis al grupo. Luego comparen los resultados: ¿llegaron todos a la misma solución o encontraron distintos caminos?

Actividad B – Identificar pilares en ejemplos dados (15 min)

Lee los siguientes tres escenarios e identifica cuál o cuáles pilares del pensamiento computacional se están aplicando en cada uno. Justifica tu respuesta:
1. Un estudiante nota que siempre obtiene mejores notas en los exámenes que estudia por temas separados, en lugar de revisar todo junto la noche anterior.
2. Un chef tiene una lista con exactamente 12 pasos para preparar su plato especial, desde pesar los ingredientes hasta el emplatado final.
3. Un programador que necesita calcular el área de un rectángulo se da cuenta de que ignora el color del rectángulo porque no afecta el cálculo.

Tema 2: Enfoque del Pensamiento Computacional

En el Tema 1 aprendimos qué es el pensamiento computacional y cómo se estructura en cuatro pilares. Ahora vamos a ver cómo ese pensamiento se aplica a uno de los conceptos más fundamentales de la informática: el procesamiento de la información. Todo sistema, ya sea una computadora, una lavadora automática, un proceso humano o incluso el cuerpo mismo, procesa información siguiendo el mismo modelo básico: entra algo, ocurre una transformación, sale un resultado.

Subtema 2.1: Procesamiento de la información (Entrada – Proceso – Salida)

Cuando hablamos de procesamiento de la información en el contexto del pensamiento computacional, nos referimos a la idea de que cualquier problema puede verse como un sistema que recibe datos, hace algo con ellos y produce un resultado. Este modelo se conoce como el ciclo Entrada–Proceso–Salida, o simplemente EPS.

Según Brookshear & Brylow (2015), este modelo no describe únicamente a las computadoras. Describe cualquier sistema que transforma información: biológico, mecánico, social o computacional. Es un marco mental que nos ayuda a analizar cómo funciona cualquier proceso antes de intentar resolverlo o mejorarlo.

2.1.1 ¿Qué es el procesamiento de información?

Procesar información significa tomar datos en bruto, aplicarles una transformación o un conjunto de operaciones y obtener un resultado útil. La información que entra al sistema no es igual a la información que sale: algo le ocurrió en el medio que la transformó en algo significativo.

Nuestro cerebro es uno de los procesadores de información más sofisticados que existen. Cuando lees este párrafo, tu cerebro recibe señales visuales (entrada), las interpreta como letras, palabras y oraciones (proceso) y construye significado a partir de ellas (salida). Todo ocurre en fracciones de segundo sin que lo notemos.

Una computadora hace exactamente lo mismo, pero de forma explícita y controlable: recibe instrucciones y datos, los procesa siguiendo las instrucciones que le damos y produce resultados que podemos ver, guardar o usar. Lo que nos permite controlar ese proceso es, justamente, el pensamiento computacional.

2.1.2 La Entrada (Input)

La entrada es todo lo que el sistema recibe como material de trabajo antes de comenzar el proceso. Pueden ser datos numéricos, texto, imágenes, comandos del usuario, señales físicas, o cualquier tipo de información que el sistema necesite para hacer su trabajo.

Lo importante de la entrada es que debe estar bien definida. Un sistema no puede procesar lo que no recibe. Si faltan datos en la entrada, el proceso estará incompleto o producirá un resultado incorrecto. Por eso, una de las primeras preguntas que hacemos al diseñar cualquier solución computacional es: ¿qué información necesito que entre al sistema?

En términos cotidianos: cuando vas a una tienda y el cajero escanea los productos, cada artículo escaneado es una entrada. El precio, la cantidad, los descuentos aplicables, todo eso es la entrada del sistema de punto de venta. Sin esa información, el sistema no puede calcular el total a pagar.

Sistema	¿Qué constituye la entrada?
Calculadora	Los números y la operación que el usuario presiona
Motor de búsqueda (Google)	Las palabras clave que el usuario escribe en la barra de búsqueda
Sistema de reconocimiento facial	La imagen o video del rostro capturado por la cámara
Receta de cocina	Los ingredientes y sus cantidades
Diagnóstico médico	Los síntomas, historial y resultados de exámenes del paciente
Formulario de matrícula universitaria	Nombre, cédula, carrera, materias seleccionadas por el estudiante

2.1.3 El Proceso

El proceso es el núcleo del sistema. Es el conjunto de operaciones, cálculos, transformaciones o decisiones que se aplican a la entrada para producir la salida. Sin el proceso, la entrada entraría al sistema y saldría exactamente igual, sin ninguna transformación útil.

En programación, el proceso suele ser la parte más compleja de diseñar, porque es ahí donde se define exactamente qué hace el sistema con la información que recibe. ¿La suma? ¿La ordena? ¿La compara con algo? ¿La filtra? ¿La transforma en otro formato? Cada una de esas operaciones es parte del proceso.

Pero de nuevo, el proceso no es exclusivo de la computación. En una lavadora automática, el proceso es el ciclo de lavado: remojar, agitar, enjuagar, centrifugar. Cada paso es una operación sobre la entrada (la ropa sucia y el detergente) que la transforma progresivamente hacia la salida deseada (ropa limpia). En medicina, el proceso es el diagnóstico y el tratamiento: el médico toma los síntomas del paciente (entrada), aplica su conocimiento y razonamiento clínico (proceso) y produce un diagnóstico y receta (salida).

Uno de los pilares del pensamiento computacional que más se usa en el diseño del proceso es la descomposición: dividir ese proceso en pasos más pequeños y manejables, asegurándose de que cada paso sea claro y tenga un propósito definido.

2.1.4 La Salida (Output)

La salida es el resultado final que el sistema produce después de procesar la entrada. Es lo que hace útil al sistema: si no produce una salida que resuelva el problema o responda la pregunta, el sistema no cumple su función.

La salida debe estar relacionada directamente con la entrada y el proceso. Si el proceso es incorrecto, la salida lo reflejará. Si la entrada tiene errores o está incompleta, la salida también los tendrá. En computación existe una frase que describe esto perfectamente: "garbage in, garbage out" (basura entra, basura sale). Si los datos de entrada son incorrectos, no hay proceso por más sofisticado que sea que produzca un resultado correcto.

La salida también puede convertirse en la entrada de otro proceso. En una cadena de producción, la salida de una etapa es la entrada de la siguiente. En un programa complejo, la salida de una función puede ser el dato que necesita la siguiente función para continuar. Esa encadenación de procesos es uno de los patrones más comunes en la programación y en el diseño de sistemas.

Sistema	Entrada	Proceso	Salida
Preparar un café	Agua, café molido, cafetera	Hervir, filtrar, verter	Taza de café caliente
Cajero automático	Tarjeta, PIN, monto a retirar	Verificar identidad y saldo	Efectivo dispensado y recibo
Traducción automática	Texto en español	Análisis lingüístico y traducción	Texto en inglés
Examen universitario	Preguntas del docente	El estudiante lee, piensa y escribe	Respuestas escritas calificadas
Motor de búsqueda	Palabras clave del usuario	Indexar, clasificar y ordenar por relevancia	Lista de páginas web relevantes

2.1.5 Ejemplos del modelo EPS en contextos cotidianos

Para terminar de consolidar el modelo, veamos cómo aplicarlo al analizar situaciones concretas. El objetivo no es solo identificar las tres partes, sino entender por qué este modelo ayuda a diseñar soluciones y a detectar errores en un proceso.

Ejemplo 1: Una estudiante quiere saber su promedio de notas del semestre. La entrada son sus calificaciones parciales de cada materia. El proceso es sumar todos los valores y dividirlos entre el número de materias. La salida es el promedio numérico final. Si el proceso es correcto pero una de las calificaciones de entrada está mal registrada, la salida también estará mal. Aquí se aplica la abstracción: solo importan las notas numéricas, no el nombre de los docentes ni el horario de las materias.

Ejemplo 2: Un sistema de alarma en una casa. La entrada es la señal del sensor de movimiento. El proceso es verificar si el sistema está activado y si el movimiento fue detectado en un área protegida. La salida es la alarma sonando o no sonando. Si la entrada falla (el sensor no detecta el movimiento real), el proceso nunca se activa aunque esté perfectamente diseñado.

Ejemplo 3: El proceso de inscripción universitaria. La entrada son los datos del estudiante: cédula, nombre, carrera elegida, materias seleccionadas. El proceso es verificar que cumpla con los requisitos (notas, cupos disponibles, prerrequisitos). La salida es la confirmación de matrícula o un mensaje de error con el motivo del rechazo. En este caso, la salida de error también es una salida válida y útil: le dice al estudiante qué debe corregir para reintentar el proceso.

Conexión con los cuatro pilares

El modelo EPS y los cuatro pilares del pensamiento computacional trabajan juntos:
– Descomposición: divide el proceso en pasos más pequeños (entrada, cada etapa del proceso, salida).
– Reconocimiento de patrones: identifica cuando la misma estructura EPS se repite en distintos sistemas.
– Abstracción: selecciona qué datos son relevantes como entrada y cuáles pueden ignorarse.
– Algoritmia: define exactamente cómo debe ocurrir el proceso, paso a paso, para transformar la entrada en la salida deseada.

Preguntas de reflexión

1. ¿Puedes pensar en un ejemplo de la vida cotidiana donde la 'salida' de un proceso sea la 'entrada' del siguiente?2. Si una receta produce un plato que no sabe bien, ¿el error está en la entrada, en el proceso o en la salida? ¿Cómo lo determinarías?3. ¿Por qué crees que es importante definir bien la entrada antes de diseñar el proceso?

Actividad práctica

Elige dos sistemas de la siguiente lista y describe sus tres componentes (entrada, proceso y salida) con el mayor detalle posible. Luego señala qué pasaría si la entrada fuera incorrecta o incompleta:
– Un sistema de semáforos
– Una calculadora de notas finales
– El proceso de preparar un jugo de naranja
– Un sistema de registro de asistencia en clase
– Una aplicación de clima que muestra el pronóstico del día

Cierre de la Unidad 2

En esta unidad construimos los cimientos conceptuales del pensamiento computacional. Comenzamos entendiendo qué es y por qué importa más allá de la tecnología: es una forma estructurada de pensar que aplica a cualquier carrera, a cualquier problema y a cualquier contexto.

Profundizamos en el origen del término gracias al trabajo de Jeannette Wing (2006) y en los cuatro pilares que lo componen. La descomposición nos enseñó a dividir lo grande en partes manejables. El reconocimiento de patrones nos mostró cómo identificar lo que se repite para no resolver el mismo problema dos veces. La

abstracción nos entrenó para quedarnos solo con lo relevante e ignorar el ruido. Y la algoritmia nos dio el lenguaje para diseñar soluciones en pasos claros, ordenados y reproducibles.

En el Tema 2 aprendimos el modelo fundamental de procesamiento de información: Entrada, Proceso y Salida. Este modelo no describe solo a las computadoras: describe cualquier sistema que transforma información. Entender bien sus tres componentes, cómo se relacionan y cómo pueden fallar es el primer paso para diseñar soluciones computacionales en las unidades que siguen.

Tareas de la Unidad 2

Tarea 1 – Análisis con los cuatro pilares (20–30 minutos)

Elige un proceso real de tu área de interés (puede ser de medicina, administración, tecnología, educación o cualquier otra). Puede ser algo como 'el proceso de atención en una clínica' o 'cómo se organiza un evento estudiantil'. Aplica los cuatro pilares del pensamiento computacional al análisis de ese proceso:
– Descomposición: ¿en qué partes se divide?
– Reconocimiento de patrones: ¿qué pasos o elementos se repiten?
– Abstracción: ¿qué información es irrelevante para el proceso central?– Algoritmia: escribe el proceso completo como una secuencia numerada de pasos.Entrega en un documento de una a dos páginas.

Tarea 2 – Diagrama EPS (15–25 minutos)

Diseña tres diagramas de Entrada–Proceso–Salida para tres situaciones diferentes, una de ellas debe ser tecnológica y las otras dos pueden ser cotidianas. Para cada diagrama especifica: todos los datos de entrada relevantes, cada paso del proceso (mínimo tres pasos) y la salida esperada. También señala qué pasaría si uno de los datos de entrada faltara o fuera incorrecto. Puedes presentarlo como tabla, lista estructurada o un diagrama dibujado a mano y fotografiado.

PENSAMIENTO COMPUTACIONAL

Guía Didáctica – Unidad 3

Proyecto de Vinculación – Facultad de Ciencias e Ingeniería

Universidad Estatal de Milagro (UNEMI)

Unidad 3: Lógica Computacional

En las unidades anteriores aprendimos a movernos en el entorno digital y a pensar de forma estructurada usando los cuatro pilares del pensamiento computacional. Ahora llegamos al núcleo técnico de la asignatura: la lógica computacional.

Esta unidad es el puente entre el pensamiento abstracto y la programación real. Antes de escribir código en cualquier lenguaje, necesitamos entender cómo se describe un proceso en términos que una computadora pueda seguir. Para eso existe el lenguaje algorítmico: un conjunto de reglas y elementos que permiten expresar soluciones de forma precisa, ordenada y sin ambigüedad.

Según Cormen et al. (2009), un algoritmo bien expresado no depende del lenguaje de programación: primero se diseña la lógica, y luego esa lógica se traduce al lenguaje que corresponda. Eso es lo que vamos a construir en esta unidad: la habilidad de diseñar y expresar esa lógica, paso a paso, usando herramientas concretas como el pseudocódigo y los flujogramas.

Tema 1: Elementos del Lenguaje Algorítmico

Antes de construir un algoritmo, necesitamos conocer sus piezas básicas. Son los mismos elementos que aparecen en casi todos los lenguajes de programación, aunque con sintaxis distinta. Entenderlos bien aquí hace que aprender cualquier lenguaje de programación después sea mucho más fácil, porque la lógica es siempre la misma.

Subtema 1.1: Tipos de datos, variables, constantes y expresiones

Cuando una computadora procesa información, esa información tiene que estar guardada en algún lugar mientras se trabaja con ella. Y no toda la información es del mismo tipo: un nombre no es lo mismo que un número, y un número entero no es lo mismo que uno con decimales. Por eso, el primer concepto fundamental en el lenguaje algorítmico es entender qué tipos de datos existen, cómo se almacenan y cómo se usan.

1.1.1 Tipos de datos

Un tipo de dato define qué clase de información puede almacenarse en una variable y qué operaciones se pueden hacer con ella. No se puede sumar texto igual que se suman números, y no se puede comparar verdadero/falso igual que se comparan edades. Identificar el tipo de dato correcto para cada situación es uno de los primeros pasos al diseñar un algoritmo.

Pensemos en una ficha de inscripción universitaria. El nombre del estudiante es texto. Su edad es un número entero. Su promedio es un número con decimales. Si está becado o no es una respuesta de sí o no. Cada campo de esa ficha corresponde a un tipo de dato diferente.

Tipo de dato	¿Qué almacena?	Ejemplos cotidianos	Ejemplo en algoritmo
Entero (int)	Números sin decimales, positivos o negativos	Edad, cantidad de materias, piso de un edificio	edad = 19, materias = 6
Real (float)	Números con parte decimal	Promedio, precio, temperatura, peso	promedio = 8.75, precio = 12.50
Texto (string)	Cadenas de caracteres, letras, palabras	Nombre, apellido, correo, dirección	nombre = "Ana", carrera = "Software"
Booleano (bool)	Solo dos valores: verdadero o falso	¿Está aprobado?, ¿está activo?, ¿tiene beca?	aprobado = verdadero, beca = falso
Carácter (char)	Un único símbolo o letra	Letra de calificación, inicial del nombre	calificacion = 'A', inicial = 'M'

Una aclaración importante: cuando en un algoritmo escribimos nombre = "Ana", las comillas indican que es texto. Si escribimos edad = 19 sin comillas, es un número. Esta distinción parece menor ahora, pero es fundamental cuando empezamos a programar, porque confundir tipos de datos es una de las fuentes de error más comunes para los principiantes.

1.1.2 Variables

Una variable es un espacio en la memoria del computador al que le damos un nombre para poder guardar y recuperar información durante la ejecución de un algoritmo. Se llama "variable" porque su valor puede cambiar mientras el algoritmo se ejecuta.

La forma más sencilla de imaginar una variable es como una caja etiquetada. La etiqueta es el nombre de la variable (por ejemplo, edad), y dentro de la caja guardamos el valor actual (por ejemplo, 19). En cualquier momento podemos abrir esa caja para ver qué hay adentro, o reemplazar el contenido por un valor nuevo.

Si el algoritmo calcula el promedio de un estudiante, puede guardar ese resultado en una variable llamada promedio. Más adelante, cuando necesite mostrar el resultado o compararlo con algo, simplemente usa el nombre de la variable en lugar de tener que calcular todo nuevamente.

> **Reglas básicas para nombrar variables**
>
> Aunque cada lenguaje de programación tiene sus propias reglas, hay convenciones generales que aplican al pseudocódigo y a la mayoría de lenguajes:
> – El nombre debe ser descriptivo: 'edad' es mejor que 'x', 'promedioFinal' es mejor que 'pf'.

> – No puede tener espacios: se usa guion bajo (edad_estudiante) o mayúscula en cada palabra (edadEstudiante).
> – No puede comenzar con un número: '2nombre' no es válido, 'nombre2' sí.
> – No debe ser una palabra reservada del lenguaje (como 'si', 'mientras', 'para').

Las variables también tienen un ciclo de vida dentro del algoritmo: se declaran (se crea el espacio), se asignan (se guarda un valor) y se usan (se lee el valor para hacer algo con él). Un buen algoritmo siempre asigna un valor inicial a sus variables antes de usarlas, para evitar trabajar con datos desconocidos o basura de memoria.

1.1.3 Constantes

Una constante es similar a una variable en que también guarda un valor con un nombre, pero con una diferencia clave: su valor no cambia durante la ejecución del algoritmo. Una vez que se define, permanece igual de principio a fin.

Un ejemplo clásico es el número PI (3.14159...). Si un algoritmo calcula el área de varias circunferencias, en lugar de escribir 3.14159 cada vez, define una constante llamada PI con ese valor y la usa en todas las fórmulas. Eso hace el código más legible y más fácil de mantener: si alguien decide usar más decimales de PI, solo cambia un lugar.

	Variable	Constante
¿Puede cambiar su valor?	Sí, en cualquier momento del algoritmo	No, se define una vez y no cambia
¿Cuándo usarla?	Para datos que varían durante la ejecución	Para valores fijos que se usan varias veces
Ejemplo	edad, promedio, contador, nombre	PI = 3.14159, IVA = 0.12, DIAS_SEMANA = 7
Convención de nombre	minúsculas o camelCase	MAYÚSCULAS o MAYUSCULAS_CON_GUION

1.1.4 Expresiones

Una expresión es cualquier combinación de valores, variables, operadores y llamadas a funciones que produce un resultado. Las expresiones son las instrucciones de cálculo dentro de un algoritmo: le dicen a la computadora qué hacer con los datos que tiene.

Una expresión puede ser tan simple como un número solo (5), o tan compleja como una fórmula que combina varias variables y operaciones. Lo que define a una expresión es que siempre produce un valor como resultado.

Tipo de expresión	Ejemplo	Resultado producido
Aritmética	promedio = (nota1 + nota2 + nota3) / 3	Un número (el promedio calculado)
Relacional	edad >= 18	Verdadero o falso
Lógica	aprobado = verdadero Y asistencia >= 70	Verdadero o falso
De texto	saludo = "Hola, " + nombre	Un texto concatenado
De asignación	contador = contador + 1	El valor actual de contador más uno

Entender qué tipo de expresión necesitamos en cada momento es parte de saber diseñar algoritmos. Cuando queremos calcular algo, usamos una expresión aritmética. Cuando queremos tomar una decisión, usamos una expresión relacional o lógica. Cuando queremos combinar texto, usamos concatenación. Todo en un algoritmo se reduce, en última instancia, a expresiones bien construidas.

Preguntas de reflexión

1. ¿Por qué crees que es importante definir bien el tipo de dato antes de usarlo en un algoritmo?
2. ¿Qué diferencia hace usar un nombre descriptivo en una variable versus uno corto como 'x' o 'a'?
3. Piensa en un proceso cotidiano (como calcular cuánto debes pagar en una tienda). ¿Qué variables necesitarías, de qué tipo serían, y habría alguna constante?

Actividad práctica

Para cada una de las siguientes situaciones, identifica: (a) qué variables necesitas, (b) de qué tipo es cada una, (c) si hay alguna constante y (d) qué expresión aritmética o relacional usarías para el cálculo principal.

1. Calcular el total a pagar en una tienda con descuento del 10%.
2. Determinar si un estudiante aprobó una materia (nota mínima: 7).
3. Calcular cuántos días han pasado desde el inicio del semestre hasta hoy.
4. Verificar si alguien tiene edad suficiente para obtener su licencia de conducir (18 años).

Subtema 1.2: Operadores aritméticos, relacionales y lógicos

Los operadores son los símbolos que le dicen al algoritmo qué hacer con los datos. Sin operadores, no podemos calcular, comparar ni tomar decisiones. Son, literalmente, los verbos del lenguaje algorítmico.

Existen tres familias principales de operadores, y cada una tiene un propósito distinto. Vamos a ver cada una con detalle, porque dominarlas es la base para construir cualquier algoritmo correctamente, según Joyanes Aguilar (2008).

1.2.1 Operadores aritméticos

Los operadores aritméticos realizan cálculos matemáticos sobre valores numéricos. Son los que ya conocemos desde la escuela, con la adición de algunos específicos del mundo computacional.

Operador	Operación	Ejemplo	Resultado
+	Suma	8 + 3	11
-	Resta	10 - 4	6
*	Multiplicación (se usa * en lugar de ×)	5 * 6	30
/	División real (resultado con decimales)	7 / 2	3.5
div o //	División entera (descarta los decimales)	7 div 2	3
mod o %	Módulo (residuo de la división entera)	7 mod 2	1
^	Potencia	2 ^ 4	16

El operador módulo (mod o %) es uno de los más útiles y menos obvios. Devuelve el residuo de una división entera. Por ejemplo, 7 mod 2 es 1 porque 7 dividido entre 2 da 3 con residuo 1. ¿Para qué sirve esto en la práctica? Para saber si un número es par o impar (si numero mod 2 = 0, es par), para saber si un año es bisiesto, para repartir elementos en grupos iguales, entre muchas otras aplicaciones.

También es importante tener en cuenta la precedencia de operadores, que es el orden en que se evalúan cuando hay varios en una misma expresión. En algoritmos y en matemáticas aplica la misma regla que aprendemos en el colegio: primero potencias, luego multiplicaciones y divisiones, luego sumas y restas. Para forzar un orden diferente, se usan paréntesis.

> **Ejemplo de precedencia**

> Expresión: resultado = 3 + 4 * 2
> Sin conocer precedencia se podría pensar: (3 + 4) * 2 = 14
> Con precedencia correcta: 3 + (4 * 2) = 3 + 8 = 11
> Si queríamos el primer resultado, debemos escribirlo explícitamente:resultado = (3 + 4) * 2

1.2.2 Operadores relacionales

Los operadores relacionales comparan dos valores y producen un resultado booleano: verdadero (si la comparación es cierta) o falso (si no lo es). Son la base de cualquier decisión dentro de un algoritmo: siempre que el algoritmo necesite "decidir" qué camino tomar, una expresión relacional está detrás de esa decisión.

Operador	Significado	Ejemplo	Resultado
= o ==	Igual a	5 == 5	Verdadero
<> o !=	Distinto de	5 != 3	Verdadero
>	Mayor que	10 > 7	Verdadero
<	Menor que	3 < 1	Falso
>=	Mayor o igual que	5 >= 5	Verdadero
<=	Menor o igual que	4 <= 3	Falso

Un error muy común al principio es confundir el signo = de asignación con el == de comparación. Cuando escribimos edad = 19, estamos guardando 19 en la variable edad. Cuando escribimos edad == 19, estamos preguntando: ¿el valor actual de edad es igual a 19? La respuesta es verdadero o falso. En pseudocódigo muchos autores usan = para ambas cosas y el contexto determina cuál es cuál, pero en lenguajes de programación esta distinción es estricta.

1.2.3 Operadores lógicos

Los operadores lógicos permiten combinar o modificar expresiones booleanas. Se usan cuando una decisión depende de más de una condición al mismo tiempo. Por ejemplo: ¿el estudiante aprobó Y tiene suficiente asistencia? O bien: ¿el usuario es administrador O es el dueño del archivo?

Operador	Significado	Comportamiento
Y (AND)	Ambas condiciones deben ser verdaderas	Verdadero SOLO si las dos condiciones son verdaderas. Si una falla, el resultado es falso.

O (OR)	Al menos una condición debe ser verdadera	Verdadero si al menos una de las condiciones es verdadera. Solo es falso si ambas son falsas.
NO (NOT)	Invierte el valor lógico de una condición	Cambia verdadero a falso y viceversa.

Para entender AND y OR de forma intuitiva, piensa en puertas que necesitan llaves. Una puerta con AND requiere que tengas la llave A y la llave B al mismo tiempo para abrirse. Una puerta con OR se abre si tienes cualquiera de las dos. El NOT es simplemente voltear el resultado: si algo era verdadero, pasa a ser falso y al revés.

Tablas de verdad simplificadas

AND (Y): solo verdadero cuando AMBAS son verdaderas. V Y V = V V Y F = F F Y V = F F Y F = FOR (O): verdadero cuando AL MENOS UNA es verdadera. V O V = V V O F = V F O V = V F O F = FNOT (NO): invierte el resultado. NO verdadero = falso NO falso = verdadero

Estos operadores se combinan en condiciones complejas. Por ejemplo: "el estudiante puede egresar si tiene más de 160 créditos aprobados Y no tiene materias reprobadas Y ha cumplido las horas de vinculación". Esa condición combina tres expresiones relacionales unidas por operadores AND. Si cualquiera de las tres es falsa, el resultado completo es falso.

Preguntas de reflexión

1. ¿Para qué sirve el operador módulo en la vida real? Piensa en al menos dos situaciones donde sería útil.
2. ¿Cuál es la diferencia entre usar AND y OR en una condición? ¿En qué situación usarías cada uno?
3. Si tienes la condición: (nota >= 7) Y (asistencia >= 70), ¿qué debe ocurrir para que el resultado sea verdadero? ¿Qué pasa si solo se cumple una de las dos partes?

Actividad práctica

Evalúa cada una de las siguientes expresiones. Indica si el resultado es verdadero o falso y explica por qué. Considera que: edad = 17, nota = 8.5, aprobado = verdadero, asistencia = 65.

1. edad >= 18
2. nota > 7
3. edad >= 18 Y nota > 7
4. edad >= 18 O nota > 7
5. NO aprobado
6. (nota >= 7) Y (asistencia >= 70)
7. (edad < 18) O (asistencia < 70)
8. NO (nota < 7)

Cierre del Tema 1

En este primer tema construimos el vocabulario básico del lenguaje algorítmico. Aprendimos que los datos tienen tipos y que elegir el tipo correcto importa. Entendimos la diferencia entre variable y constante, y cómo las expresiones combinan ambas para producir resultados. Luego conocimos los tres grupos de operadores: los aritméticos para calcular, los relacionales para comparar y los lógicos para combinar condiciones. Con estos elementos ya tenemos las piezas del lenguaje. En el Tema 2 vamos a aprender a ensamblarlas.

Actividad A – Clasificación de datos (15 min)

Para el siguiente formulario de inscripción a un curso, identifica el tipo de dato de cada campo y propón un nombre de variable adecuado:
Nombre completo / Número de cédula / Edad / Carrera / Promedio del semestre anterior / ¿Tiene beca? / Número de materias aprobadas / Correo electrónico / Fecha de nacimiento

Actividad B – Construcción de expresiones (20 min)

Construye una expresión algoritmica para cada situación:
1. Calcular el precio final de un producto después de aplicar un 15% de descuento.
2. Determinar si un estudiante pasa de año (necesita promedio >= 6 y asistencia >= 75%).
3. Verificar si un número ingresado por el usuario es positivo y par a la vez.
4. Calcular el área de un triángulo (base * altura / 2).
5. Verificar si un año es bisiesto (divisible entre 4 y no entre 100, O divisible entre 400).

Tema 2: Pseudocódigo, Flujogramas y Estructuras de Control

Ya conocemos los elementos básicos del lenguaje algorítmico. Ahora vamos a aprender cómo usarlos para construir algoritmos completos. Para eso tenemos dos herramientas fundamentales: el pseudocódigo, que es una forma de escribir algoritmos en lenguaje casi natural, y los flujogramas, que son la representación gráfica de esos mismos algoritmos. Ambas herramientas tienen el mismo propósito: expresar la lógica de una solución de forma clara, antes de trasladarla a un lenguaje de programación real.

Subtema 2.1: Construcción de pseudocódigo y flujogramas con estructuras secuenciales

Comencemos con el tipo más simple de algoritmo: la estructura secuencial. Un algoritmo secuencial es aquel donde las instrucciones se ejecutan una tras otra, en orden, de arriba hacia abajo, sin ninguna decisión ni repetición. Es la base sobre la que se construyen todos los demás tipos de algoritmos.

2.1.1 ¿Qué es el pseudocódigo?

El pseudocódigo es una forma de describir un algoritmo usando un lenguaje intermedio entre el español natural y un lenguaje de programación real. No es código que una computadora pueda ejecutar directamente, pero tiene la estructura suficiente para que cualquier programador, independientemente del lenguaje que use, pueda entenderlo y traducirlo.

La palabra "pseudo" significa falso o simulado. Pseudocódigo significa, literalmente, código falso. No tiene una sintaxis estricta universal, pero sí tiene convenciones que la mayoría de libros y cursos respetan. Lo más importante es que sea claro, preciso y sin ambigüedades.

Según Joyanes Aguilar (2008), el pseudocódigo es la herramienta más usada para diseñar algoritmos antes de programarlos, porque permite concentrarse en la lógica del

problema sin preocuparse por la sintaxis específica de ningún lenguaje. Es como hacer un borrador antes de escribir el texto final.

Estructura general de un pseudocódigo

ALGORITMO nombre_del_algoritmo VARIABLES tipo nombre_variable INICIO instruccion_1 instruccion_2 ... instruccion_n FIN

Las instrucciones más básicas que usamos en pseudocódigo son LEER (para recibir un dato del usuario), ESCRIBIR o MOSTRAR (para mostrar un resultado), y la asignación con la flecha o el signo igual (para guardar un valor en una variable).

2.1.2 Pseudocódigo secuencial: el primer ejemplo completo

Construyamos juntos el algoritmo más simple posible con estructura secuencial: pedir el nombre de un estudiante y saludarlo. Este ejemplo no hace cálculos ni toma decisiones, solo sigue tres pasos en orden.

```
ALGORITMO SaludarEstudiante
   VARIABLES
      texto nombre
   INICIO
      ESCRIBIR "Por favor, ingresa tu nombre:"
      LEER nombre
      ESCRIBIR "Hola, " + nombre + ". Bienvenido al curso."
   FIN
```

Analicemos qué ocurre paso a paso. Primero se declara la variable nombre de tipo texto. Luego, en el INICIO, se muestra un mensaje pidiendo el nombre, se lee lo que el usuario escriba y se guarda en la variable nombre, y finalmente se construye un mensaje de saludo combinando texto fijo con el valor de la variable.

Ahora un ejemplo un poco más complejo: calcular el promedio de tres notas.

```
ALGORITMO CalcularPromedio
   VARIABLES
      real nota1, nota2, nota3, promedio
   INICIO
      ESCRIBIR "Ingresa la primera nota:"
      LEER nota1
      ESCRIBIR "Ingresa la segunda nota:"
      LEER nota2
```

```
    ESCRIBIR "Ingresa la tercera nota:"
    LEER nota3
    promedio <- (nota1 + nota2 + nota3) / 3
    ESCRIBIR "Tu promedio es: " + promedio
FIN
```

Este algoritmo sigue siendo completamente secuencial: no decide nada, no repite nada. Simplemente pide tres datos, calcula y muestra el resultado. La flecha <- es el operador de asignación: significa "guarda el resultado de la expresión de la derecha en la variable de la izquierda".

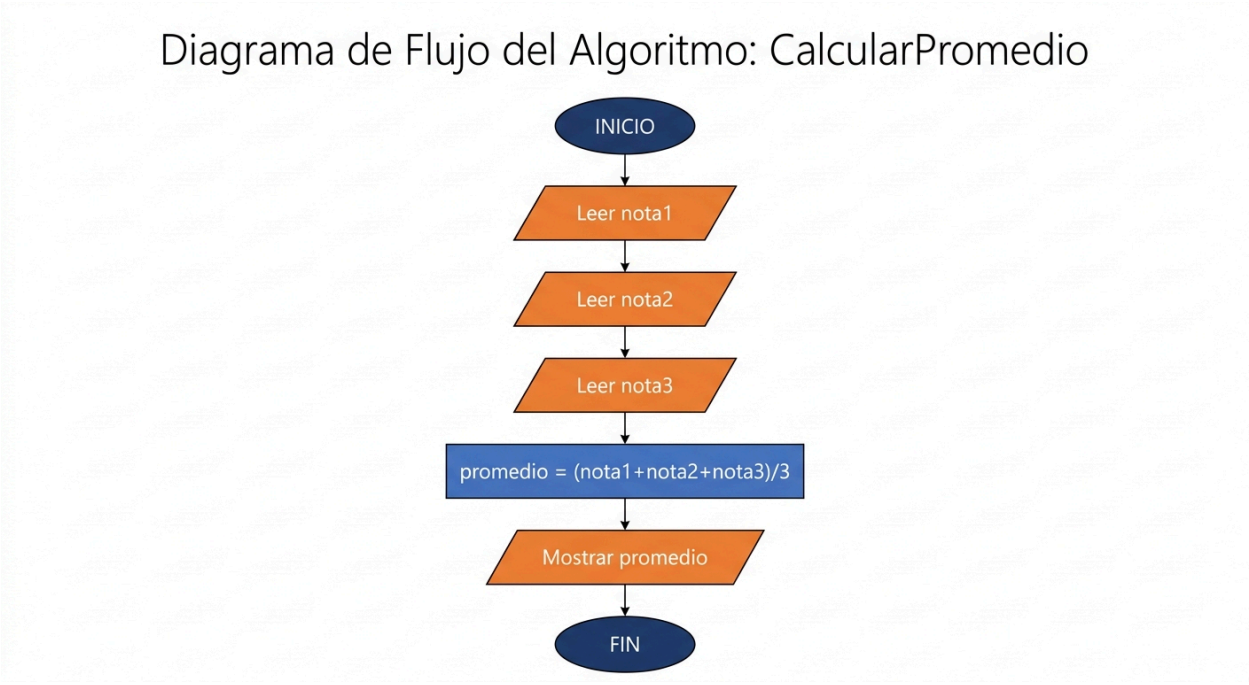
2.1.3 ¿Qué es un flujograma?

Un flujograma (también llamado diagrama de flujo) es la representación visual de un algoritmo. En lugar de texto, usa figuras geométricas conectadas por flechas para mostrar el flujo del proceso. Cada figura tiene un significado específico y estandarizado, lo que hace que un flujograma pueda ser leído por cualquier persona que conozca los símbolos, sin importar el idioma en que estén escritos los textos dentro de las figuras.

La norma ISO 5807 establece los símbolos estándar para los flujogramas. Los más usados en algoritmos básicos son cinco.

Figura	Nombre	¿Para qué se usa?
Óvalo / Elipse	Terminal	Marca el inicio y el fin del algoritmo. Siempre hay uno de cada uno.
Paralelogramo	Entrada / Salida	Representa LEER (entrada de datos) o ESCRIBIR (salida de resultados).
Rectángulo	Proceso	Representa cálculos, asignaciones y cualquier instrucción de procesamiento.
Rombo	Decisión	Representa una condición que produce dos caminos: Sí (verdadero) o No (falso).
Flechas	Flujo	Conectan las figuras e indican la dirección en que avanza el algoritmo.

Para el ejemplo del cálculo de promedio que vimos en pseudocódigo, el flujograma tendría: un óvalo de INICIO, tres paralelogramos de entrada (uno por cada nota), un rectángulo de proceso (el cálculo del promedio), un paralelogramo de salida (mostrar el promedio) y un óvalo de FIN. Todo conectado con flechas de arriba hacia abajo.



2.1.4 Trazas de algoritmos

Una traza es la simulación manual de la ejecución de un algoritmo, paso a paso, con valores concretos. Es una herramienta fundamental para verificar que un algoritmo funciona correctamente antes de programarlo, o para encontrar un error en uno que no produce el resultado esperado.

Para hacer una traza, seguimos el algoritmo instrucción por instrucción y registramos el valor de cada variable en cada paso. Esto nos permite "ver" qué pasa adentro del algoritmo cuando lo ejecutamos con datos reales.

Traza del algoritmo CalcularPromedio con datos: nota1=8, nota2=7, nota3=9

Paso 1: LEER nota1 -> nota1 = 8
Paso 2: LEER nota2 -> nota2 = 7
Paso 3: LEER nota3 -> nota3 = 9
Paso 4: promedio = (8 + 7 + 9) / 3 -> promedio = 24 / 3 -> promedio = 8.0
Paso 5: ESCRIBIR promedio -> Salida en pantalla: "Tu promedio es: 8.0"Resultado: correcto.

Hacer trazas es un hábito que los buenos programadores mantienen durante toda su carrera. Antes de ejecutar código, simular el algoritmo con valores de prueba ayuda a detectar errores lógicos que a veces no son evidentes solo leyendo el código.

Preguntas de reflexión

1. ¿Qué ventaja tiene escribir primero el pseudocódigo antes de programar directamente?
2. ¿Por qué crees que los flujogramas usan figuras diferentes para cada tipo de instrucción y no simplemente texto?
3. Si al hacer la traza de un algoritmo el resultado no es el esperado, ¿en qué parte buscarías primero el error?

Actividad práctica

Diseña el pseudocódigo y el flujograma para el siguiente algoritmo:Un estudiante quiere saber cuánto pagará en total por tres materias de pago en la universidad. El sistema debe pedir el costo individual de cada materia, calcular el subtotal, aplicar un descuento del 5% si el total supera los $200, y mostrar el monto final a pagar.
Pasos a entregar:
1. Pseudocódigo completo con declaración de variables.
2. Flujograma dibujado a mano o en computadora con los símbolos correctos.
3. Traza con datos de prueba inventados por ti.

Subtema 2.2: Estructuras de control

Los algoritmos puramente secuenciales son útiles pero limitados. En la mayoría de situaciones reales, un algoritmo necesita tomar decisiones (hacer una cosa u otra según una condición) o repetir acciones (ejecutar el mismo conjunto de pasos múltiples veces). Para eso existen las estructuras de control.

Las estructuras de control son las instrucciones que alteran el flujo secuencial de un algoritmo. Sin ellas, un algoritmo va siempre de arriba hacia abajo sin desviarse. Con ellas, puede ramificarse, saltar y repetirse, lo que lo hace capaz de resolver problemas mucho más complejos.

2.2.1 Condicionales simples (SI – ENTONCES)

La estructura condicional simple permite ejecutar un bloque de instrucciones solo si una condición es verdadera. Si la condición es falsa, ese bloque se omite completamente y el algoritmo continúa con lo que viene después.

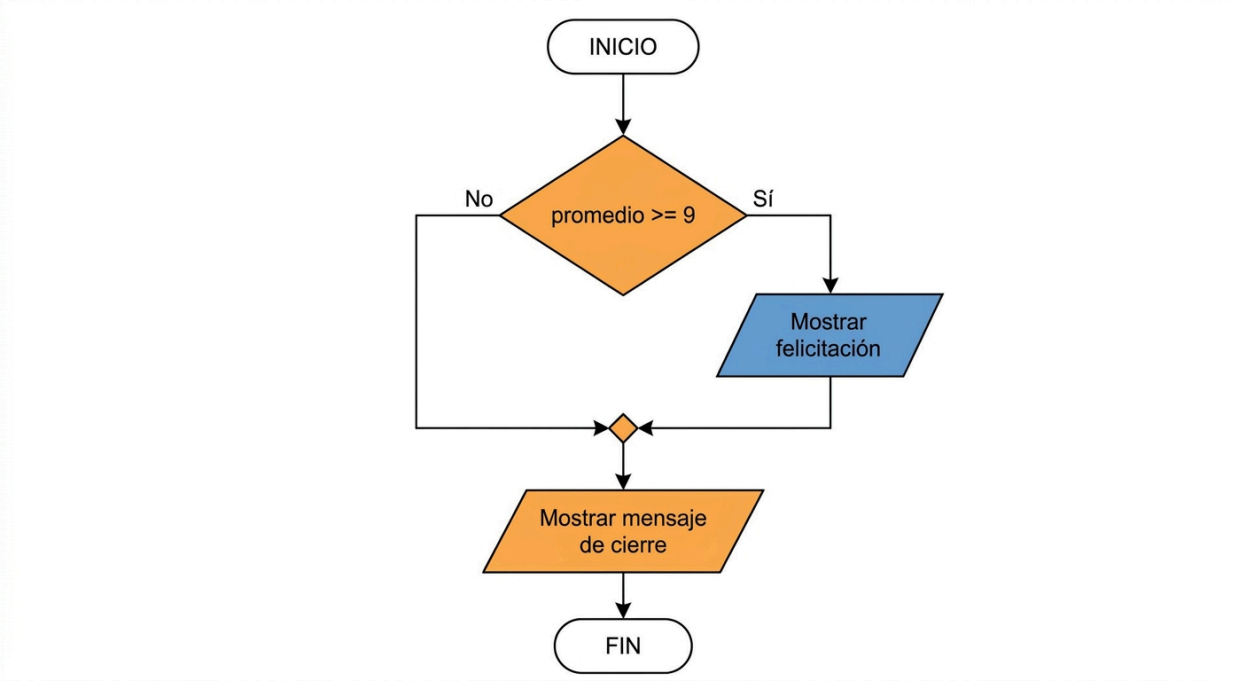
En pseudocódigo se escribe con la palabra SI seguida de la condición, luego ENTONCES con el bloque de instrucciones a ejecutar si la condición es verdadera, y finalmente FIN_SI para cerrar la estructura.

```
SI condicion ENTONCES
   instrucciones_si_es_verdadero
FIN_SI
```

Un ejemplo concreto: mostrar un mensaje de felicitación si el promedio es mayor o igual a 9.

```
SI promedio >= 9 ENTONCES
   ESCRIBIR "Felicitaciones, obtuviste una nota sobresaliente."
FIN_SI
ESCRIBIR "Gracias por completar el cuestionario."
```

Si promedio es 9.5, la condición es verdadera y se muestra el mensaje de felicitación. Si promedio es 7.0, la condición es falsa, ese ESCRIBIR se omite y el algoritmo continúa directamente con "Gracias por completar el cuestionario." que está fuera de la estructura condicional y se ejecuta siempre.



2.2.2 Condicionales compuestas (SI – ENTONCES – SINO)

La condicional compuesta agrega una alternativa: qué hacer si la condición es falsa. Con la estructura simple, si la condición es falsa, simplemente no pasa nada. Con la

compuesta, tenemos dos caminos definidos: uno para cuando la condición es verdadera y otro para cuando es falsa. Siempre se ejecuta exactamente uno de los dos, nunca ambos.

```
SI condicion ENTONCES
   instrucciones_si_verdadero
SINO
   instrucciones_si_falso
FIN_SI
```

Ejemplo: determinar si un estudiante aprobó o reprobó según su promedio.

```
SI promedio >= 7 ENTONCES
   ESCRIBIR "Aprobado. Promedio: " + promedio
SINO
   ESCRIBIR "Reprobado. Promedio: " + promedio
FIN_SI
```

Con un promedio de 8.0 se mostrará "Aprobado". Con un promedio de 5.5 se mostrará "Reprobado". Nunca ambos, nunca ninguno: siempre exactamente uno.

2.2.3 Condicionales anidadas

Las condicionales anidadas ocurren cuando dentro de un SI colocamos otro SI. Esto permite manejar más de dos posibilidades, creando una jerarquía de decisiones donde la segunda decisión solo se evalúa si la primera produjo cierto resultado.

Un ejemplo muy común es la clasificación por rangos, como asignar una letra de calificación según el promedio numérico.

```
SI promedio >= 9 ENTONCES
   ESCRIBIR "Calificación: A (Sobresaliente)"
SINO
   SI promedio >= 8 ENTONCES
      ESCRIBIR "Calificación: B (Muy Bueno)"
   SINO
      SI promedio >= 7 ENTONCES
         ESCRIBIR "Calificación: C (Bueno)"
      SINO
         SI promedio >= 6 ENTONCES
            ESCRIBIR "Calificación: D (Suficiente)"
         SINO
            ESCRIBIR "Calificación: F (Reprobado)"
```

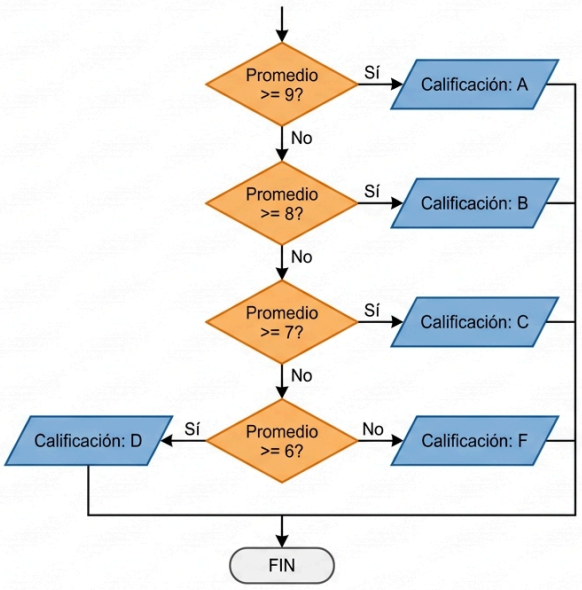
```
        FIN_SI
      FIN_SI
   FIN_SI
FIN_SI
```

El flujo funciona como una cascada: primero se verifica si el promedio es mayor o igual a 9. Si es falso, se verifica si es mayor o igual a 8, y así sucesivamente. En cuanto una condición es verdadera, se muestra la calificación correspondiente y se sale de toda la estructura sin evaluar las siguientes condiciones.

Las condicionales anidadas son poderosas pero deben usarse con cuidado. Cuando hay demasiados niveles de anidamiento, el algoritmo se vuelve difícil de leer y de mantener. Una regla práctica: si hay más de tres o cuatro niveles de anidamiento, probablemente hay una forma más limpia de expresar la misma lógica.

Tipo de condicional	¿Cuántos caminos tiene?	¿Cuándo usarla?
Simple (SI-ENTONCES)	Dos: hace algo o no hace nada	Cuando solo interesa el caso verdadero
Compuesta (SI-ENTONCES-SINO)	Dos: hace una cosa o hace otra	Cuando hay exactamente dos alternativas
Anidada	Tres o más	Cuando hay múltiples rangos o categorías posibles

Diagrama de Flujo: Clasificación de Promedio Anidado (A-F)



Preguntas de reflexión

1. ¿Cuál es la diferencia entre una condicional simple y una compuesta? ¿Cuándo usarías una y cuándo la otra?2. ¿Qué pasa si en una condicional anidada dos condiciones son verdaderas al mismo tiempo? ¿Se ejecutan ambos bloques?3. Piensa en una situación de la vida real que requiera tres o más condiciones anidadas. ¿Cómo la estructurarías?

Actividad práctica

Escribe el pseudocódigo para los siguientes algoritmos:1. (Condicional simple) Verificar si un número ingresado es positivo. Si lo es, mostrar 'Número positivo'. Si no es positivo, no hacer nada más.2. (Condicional compuesta) Determinar si una persona puede votar (18 años o más). Mostrar un mensaje diferente según el resultado.3. (Condicional anidada) Clasificar la temperatura del día: menos de 10°C es 'Frío', entre 10 y 20°C es 'Fresco', entre 21 y 30°C es 'Agradable', más de 30°C es 'Caluroso'.

2.2.4 Estructuras repetitivas

Las estructuras repetitivas, también llamadas bucles o ciclos, permiten que un bloque de instrucciones se ejecute múltiples veces sin tener que escribirlo múltiples veces. Son una de las características más poderosas de los algoritmos, porque permiten automatizar tareas repetitivas.

Sin bucles, si quisiéramos calcular el promedio de 30 estudiantes, tendríamos que copiar el mismo bloque de instrucciones 30 veces. Con un bucle, lo escribimos una vez y le decimos cuántas veces repetirlo. Según Cormen et al. (2009), las estructuras repetitivas son fundamentales para que los algoritmos sean escalables: funcionan igual de bien con 3 datos que con 3 millones.

Existen tres tipos principales de estructuras repetitivas: el bucle PARA (cuando sabemos exactamente cuántas veces repetir), el bucle MIENTRAS (cuando repetimos mientras una condición sea verdadera) y el bucle REPETIR-HASTA (cuando repetimos al menos una vez y paramos cuando se cumple una condición). Vamos a ver cada uno.

2.2.5 Bucle PARA (FOR)

El bucle PARA se usa cuando sabemos de antemano cuántas veces debe repetirse el bloque. Usamos un contador que avanza automáticamente desde un valor inicial hasta un valor final.

```
PARA contador DESDE inicio HASTA fin CON PASO incremento HACER
   instrucciones
FIN_PARA
```

Ejemplo: mostrar los números del 1 al 5.

```
PARA i DESDE 1 HASTA 5 CON PASO 1 HACER
   ESCRIBIR i
FIN_PARA
```

En la primera vuelta, i vale 1 y se muestra 1. En la segunda, i vale 2 y se muestra 2. Y así hasta que i llega a 5. Después de mostrar 5, el contador intenta avanzar a 6, pero como eso supera el límite (5), el bucle termina.

El paso puede ser cualquier número, no necesariamente 1. Si el paso es 2, el contador avanza de dos en dos. Si el paso es -1, el contador disminuye (cuenta regresiva). Por ejemplo, PARA i DESDE 10 HASTA 1 CON PASO -1 produciría los números del 10 al 1 en orden descendente.

> **Ejemplo – Calcular la suma de los primeros 10 números**
>
> ALGORITMO SumarPrimerosDiez VARIABLES entero i, suma INICIO suma <- 0 PARA i DESDE 1 HASTA 10 CON PASO 1 HACER suma <- suma + i FIN_PARA ESCRIBIR "La suma es: " + suma FINTraza resumida:i=1: suma=0+1=1 | i=2: suma=1+2=3 | i=3: suma=3+3=6 | ...i=10: suma=45+10=55Resultado: La suma es: 55

2.2.6 Bucle MIENTRAS (WHILE)

El bucle MIENTRAS repite un bloque de instrucciones mientras una condición sea verdadera. La diferencia fundamental con el PARA es que aquí no necesitamos saber de antemano cuántas veces va a repetirse: el bucle sigue mientras la condición se cumpla y se detiene en el momento en que deja de cumplirse.

La condición se evalúa al inicio de cada vuelta. Si la condición ya es falsa desde el principio, el bloque nunca se ejecuta. Esto es importante: es posible que un bucle MIENTRAS no se ejecute ni una sola vez.

```
MIENTRAS condicion HACER
   instrucciones
```

```
FIN_MIENTRAS
```

Ejemplo: pedirle al usuario que ingrese un número positivo. Si ingresa un número negativo o cero, volver a pedirlo.

```
ALGORITMO PedirNumeroPositivo
   VARIABLES
      real numero
   INICIO
      ESCRIBIR "Ingresa un número positivo:"
      LEER numero
      MIENTRAS numero <= 0 HACER
         ESCRIBIR "Error: el número debe ser positivo. Intenta de
nuevo:"
         LEER numero
      FIN_MIENTRAS
      ESCRIBIR "Número aceptado: " + numero
   FIN
```

Si el usuario ingresa directamente un número positivo, la condición numero <= 0 es falsa desde el inicio y el bucle nunca se ejecuta. Si ingresa -3, la condición es verdadera, se muestra el error y se vuelve a pedir. Esto se repite hasta que el usuario ingrese un valor positivo.

Hay un riesgo con el bucle MIENTRAS: si la condición nunca llega a ser falsa, el bucle se repite infinitamente y el programa se cuelga. Siempre hay que asegurarse de que, en algún momento dentro del bucle, algo cambie para que la condición pueda volverse falsa.

2.2.7 Bucle REPETIR – HASTA (DO-WHILE)

El bucle REPETIR-HASTA es similar al MIENTRAS, pero con una diferencia clave: la condición se evalúa al final, no al principio. Esto garantiza que el bloque de instrucciones se ejecuta al menos una vez, sin importar si la condición es verdadera o falsa desde el inicio.

```
REPETIR
   instrucciones
HASTA condicion
```

Nótese que aquí la condición funciona al revés: el bucle repite mientras la condición sea falsa y se detiene cuando la condición se vuelve verdadera. En el MIENTRAS, el

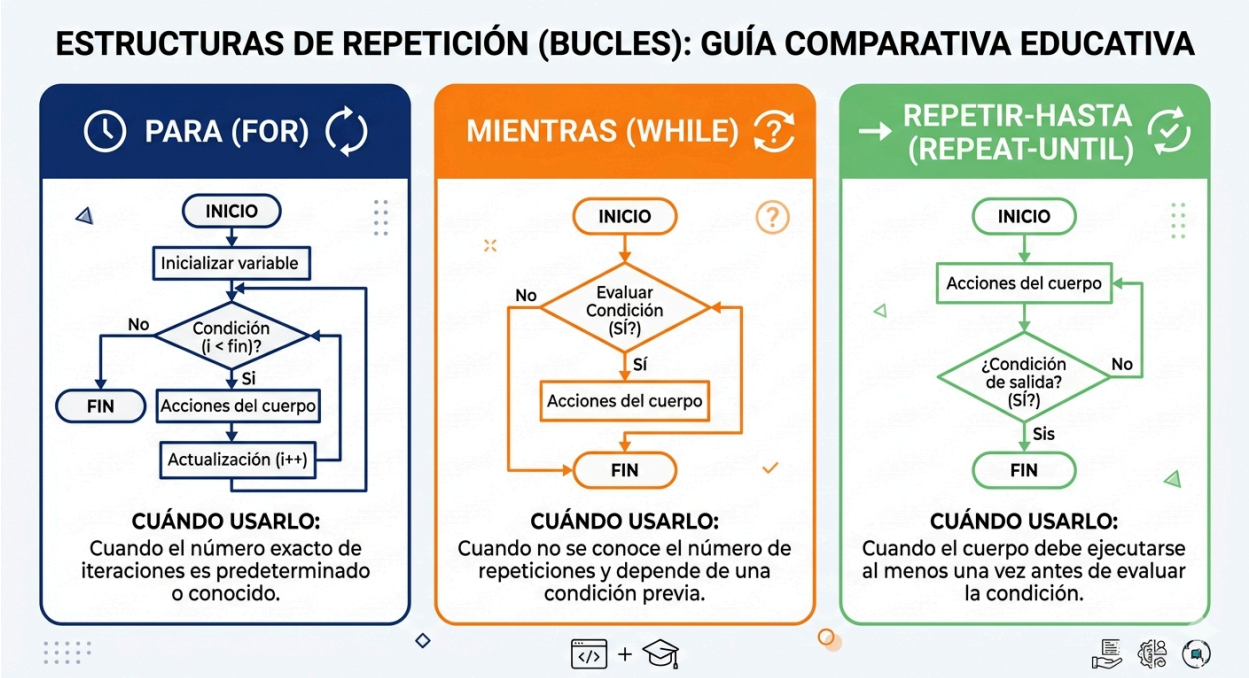
bucle corre mientras la condición es verdadera. En el REPETIR-HASTA, el bucle corre hasta que la condición sea verdadera.

El mismo ejemplo de pedir un número positivo, pero ahora con REPETIR-HASTA:

```
ALGORITMO PedirNumeroPositivoV2
   VARIABLES
      real numero
   INICIO
      REPETIR
         ESCRIBIR "Ingresa un número positivo:"
         LEER numero
         SI numero <= 0 ENTONCES
            ESCRIBIR "Error: el número debe ser positivo."
         FIN_SI
      HASTA numero > 0
      ESCRIBIR "Número aceptado: " + numero
   FIN
```

Este algoritmo siempre pide el número al menos una vez, lo que tiene más sentido: no podemos saber si el número es válido antes de pedirlo. Con el MIENTRAS necesitábamos hacer una lectura inicial antes de entrar al bucle. Con el REPETIR-HASTA eso no es necesario porque la primera ejecución siempre ocurre.

Bucle	¿Cuándo se evalúa la condición?	¿Mínimo de ejecuciones?	¿Cuándo usarlo?
PARA	Al inicio; el contador avanza automáticamente	0 (si inicio > fin)	Cuando sabemos exactamente cuántas veces repetir
MIENTRAS	Al inicio de cada vuelta	0 (puede no ejecutarse)	Cuando no sabemos cuántas veces, condición al inicio
REPETIR-HASTA	Al final de cada vuelta	1 (siempre se ejecuta al menos una vez)	Cuando necesitamos ejecutar al menos una vez antes de verificar



Preguntas de reflexión

1. ¿Qué tipo de bucle usarías para recorrer una lista de 20 estudiantes y calcular su promedio? ¿Por qué?2. ¿En qué situación es importante usar REPETIR-HASTA en lugar de MIENTRAS?3. ¿Qué pasa si en un bucle MIENTRAS la condición nunca se vuelve falsa? ¿Cómo evitarías eso?

Actividad práctica

Escribe el pseudocódigo para cada uno de los siguientes algoritmos e indica qué tipo de bucle usarías y por qué:1. (Bucle PARA) Calcular la tabla de multiplicar del número que ingrese el usuario (del 1 al 10).2. (Bucle MIENTRAS) Pedir contraseñas repetidamente hasta que el usuario ingrese la contraseña correcta 'unemi2025'. Mostrar cuántos intentos necesitó.3. (Bucle REPETIR-HASTA) Un cajero automático muestra un menú con opciones (1-Consultar saldo, 2-Retirar, 3-Salir). El menú debe mostrarse al menos una vez y repetirse hasta que el usuario elija la opción 3.4. (Combinación) Pedir 5 notas de un estudiante usando un bucle PARA, sumarlas y mostrar el promedio. Dentro del bucle, si alguna nota está fuera del rango 0-10, mostrar un mensaje de error y no contarla.

Cierre de la Unidad 3

En esta unidad hicimos el salto del pensamiento abstracto a la expresión técnica concreta. Comenzamos construyendo el vocabulario del lenguaje algorítmico: tipos de datos, variables, constantes y expresiones. Luego conocimos los tres grupos de operadores que le dan vida a las expresiones: los aritméticos para calcular, los relacionales para comparar y los lógicos para combinar condiciones.

En el Tema 2 aprendimos a ensamblar todo eso en algoritmos completos usando pseudocódigo y flujogramas. Empezamos con la estructura más simple, la secuencial, donde las instrucciones se ejecutan una tras otra. Luego avanzamos a las estructuras de control: las condicionales (simple, compuesta y anidada) que permiten tomar decisiones, y las repetitivas (PARA, MIENTRAS y REPETIR-HASTA) que permiten automatizar tareas que se repiten.

Con estos elementos ya tenemos todo lo necesario para diseñar algoritmos que resuelven problemas reales. En la Unidad 4 trasladaremos esa lógica a un entorno visual de programación usando Scratch, donde cada uno de estos conceptos tendrá su equivalente en bloques gráficos.

Tareas de la Unidad 3

Tarea 1 – Diseño algorítmico completo (25–35 minutos)

Diseña el pseudocódigo y el flujograma para el siguiente sistema:
Un sistema de calificación universitaria recibe las notas de 5 actividades de un estudiante. Cada actividad tiene un peso diferente: actividad 1 y 2 valen el 15% cada una, actividad 3 y 4 valen el 20% cada una, y la actividad 5 (proyecto final) vale el 30%. El sistema debe calcular la nota final ponderada, determinar si el estudiante aprobó (nota >= 7) y mostrar un mensaje indicando el resultado con la nota obtenida.
Entrega: pseudocódigo con declaración de variables, flujograma con símbolos correctos y traza con datos de prueba inventados por ti.

Tarea 2 – Análisis de algoritmos con bucles (20–30 minutos)

Para cada uno de los siguientes algoritmos en pseudocódigo, realiza una traza completa con los datos indicados y responde: ¿qué hace el algoritmo?, ¿cuántas veces se ejecuta el bucle?, ¿cuál es la salida?
Algoritmo A (con datos: n = 4): suma = 0 PARA i DESDE 1 HASTA n HACER suma = suma + (i * i) FIN_PARA ESCRIBIR suma
Algoritmo B (con datos: inicio = 1): x = inicio MIENTRAS x < 100 HACER x = x * 2 FIN_MIENTRAS ESCRIBIR x

PENSAMIENTO COMPUTACIONAL

Guía Didáctica – Unidad 4

Proyecto de Vinculación – Facultad de Ciencias e Ingeniería

Universidad Estatal de Milagro (UNEMI)

Unidad 4: Programación Visual con Scratch

Hasta aquí hemos aprendido a movernos en el entorno digital, entendimos cómo funciona el pensamiento computacional, conocimos sus cuatro pilares y construimos algoritmos con pseudocódigo y flujogramas. Ahora llegamos a una unidad más práctica: la programación visual con Scratch.

Esta unidad es donde toda la teoría anterior se hará uso. Las variables, los condicionales, los bucles, las estructuras secuenciales que diseñamos en papel ahora se convierten en bloques de colores que podemos arrastrar, encajar y hacer funcionar en tiempo real. Y lo mejor: vamos a poder ver el resultado inmediatamente en la pantalla.

Scratch fue diseñado precisamente con esa idea: que cualquier persona, sin importar su edad o experiencia previa, pueda aprender los conceptos fundamentales de la programación de forma intuitiva y visual. Según Resnick et al. (2009), Scratch hace que programar sea tan accesible como armar piezas de Lego: si dos bloques no encajan, simplemente no se conectan, lo que elimina los errores de sintaxis que tanto frustran a los principiantes en lenguajes tradicionales.

En esta unidad vamos a recorrer Scratch desde lo más básico, paso a paso. Primero nos familiarizaremos con la herramienta y su interfaz. Luego aprenderemos cómo darle vida a los personajes con secuencias, eventos y movimientos. Después aplicaremos todo lo aprendido sobre variables y estructuras de control directamente en Scratch. Y para cerrar, construiremos proyectos interactivos completos que demuestren todo lo que hemos aprendido.

Tema 1: Introducción a Scratch

Subtema 1.1: Instalación, interfaz y lógica de bloques

Antes de empezar a programar en Scratch, necesitamos conocer qué es exactamente esta herramienta, de dónde viene y cómo acceder a ella. También es fundamental familiarizarse con su interfaz, porque saber dónde está cada cosa nos va a ahorrar mucho tiempo más adelante cuando empecemos a construir proyectos.

1.1.1 ¿Qué es Scratch y de dónde viene?

Scratch es un lenguaje de programación visual y un entorno de desarrollo en línea creado por el grupo Lifelong Kindergarten en el MIT Media Lab, bajo la dirección del investigador Mitchel Resnick. Fue lanzado públicamente en 2007 y desde entonces se ha convertido en la herramienta más usada en el mundo para enseñar programación a principiantes.

La idea detrás de Scratch parte de una pregunta simple: ¿por qué programar tiene que ser difícil de aprender? Los lenguajes tradicionales como Python, Java o C++ obligan al

programador a escribir cada instrucción con una sintaxis exacta. Un punto y coma faltante, una mayúscula equivocada o un paréntesis mal cerrado puede impedir que el programa funcione. Esa barrera técnica desanima a muchos antes de que siquiera lleguen a entender la lógica.

Scratch elimina ese problema. En lugar de escribir texto, arrastras bloques de colores que se conectan entre sí como piezas de un rompecabezas. Si dos bloques no pueden conectarse, físicamente no encajan, así que es imposible cometer errores de sintaxis. Esto permite que el aprendiz se concentre en lo importante: la lógica del programa.

Según Maloney et al. (2010), Scratch fue diseñado siguiendo tres principios fundamentales: hacer la programación más conectada (a intereses personales y diversas), más significativa (resultados visibles y compartibles) y más social (poder compartir y remezclar el trabajo de otros). Esa filosofía explica por qué Scratch incluye personajes, sonidos, gráficos y una comunidad en línea desde el primer día.

1.1.2 ¿Por qué aprender programación visual?

Algunos estudiantes se preguntan si aprender Scratch "sirve para algo real" cuando existen lenguajes profesionales más potentes. La respuesta corta es: sí, y mucho. La programación visual no es un sustituto de los lenguajes profesionales, es un puente para llegar a ellos.

Cuando trabajamos con Scratch, internalizamos los conceptos fundamentales de la programación: secuencias, variables, condicionales, bucles, eventos, parámetros. Esos conceptos son universales y existen en absolutamente todos los lenguajes de programación. La diferencia es que en Scratch los aprendemos viendo cómo funcionan, sin que la sintaxis nos distraiga del concepto.

Brennan & Resnick (2012) demostraron en sus investigaciones que estudiantes que aprenden los fundamentos con Scratch hacen una transición más fluida a lenguajes textuales como Python, porque ya entienden la lógica subyacente. Lo único que tienen que aprender después es la sintaxis específica del nuevo lenguaje, no los conceptos.

Concepto algorítmico	En la Unidad 3 (pseudocódigo)	En Scratch (visual)
Asignación de variable	edad <- 19	Bloque naranja: dar a (edad) el valor (19)
Condicional simple	SI x > 10 ENTONCES ... FIN_SI	Bloque naranja con forma de boca: si <x>10> entonces
Bucle repetitivo	PARA i DESDE 1 HASTA 10 HACER ...	Bloque naranja: repetir (10) veces
Salida de datos	ESCRIBIR "Hola"	Bloque morado: decir (Hola) durante (2) segundos
Operador relacional	promedio >= 7	Bloque verde con forma de rombo: (promedio) > (7)

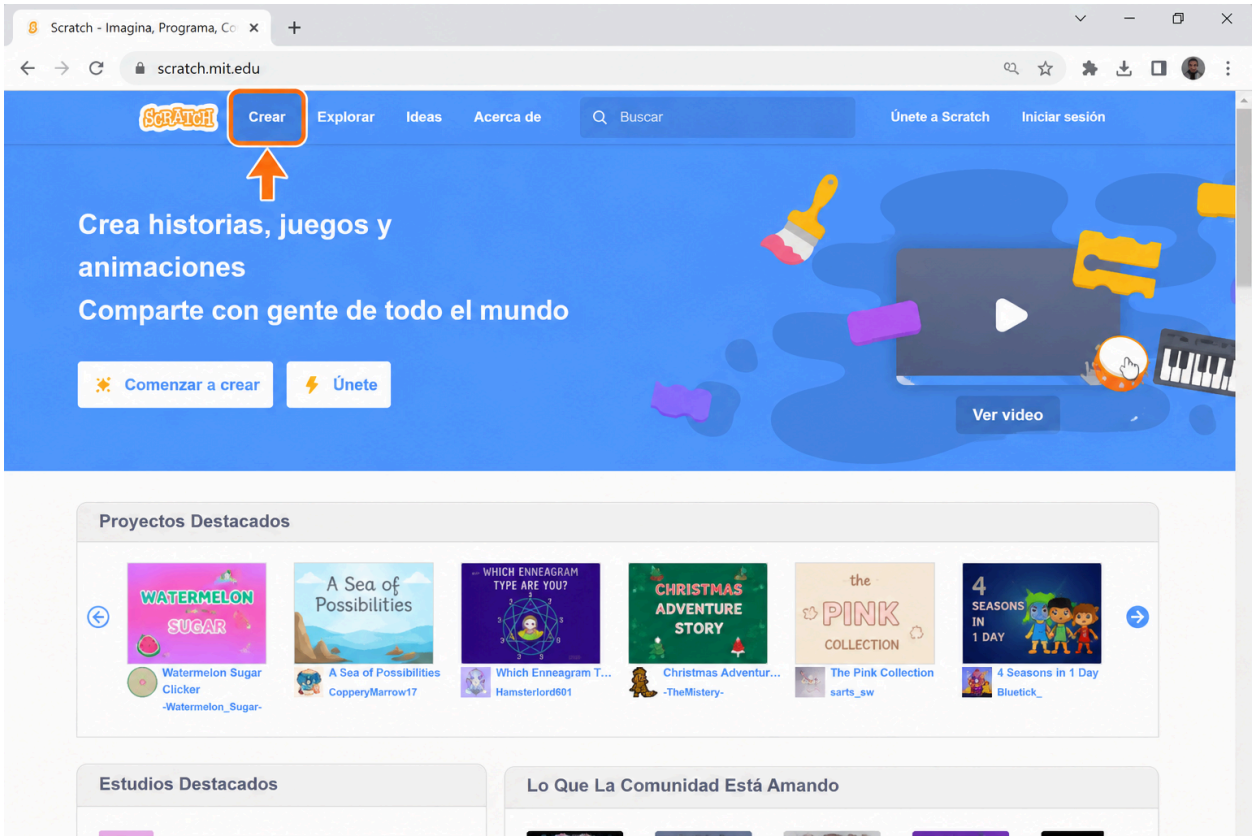
1.1.3 Cómo acceder a Scratch

Scratch tiene dos formas de uso, ambas gratuitas. La más recomendada para empezar es la versión en línea, que funciona directamente desde el navegador sin necesidad de instalar nada.

Para acceder a la versión en línea, basta con ir a scratch.mit.edu desde cualquier navegador moderno (Chrome, Firefox, Edge o Safari). Desde ahí se puede empezar a crear un proyecto haciendo clic en "Crear", que abre el editor directamente. No es necesario tener una cuenta para usar Scratch, pero crear una cuenta gratuita permite guardar los proyectos, recuperarlos desde cualquier dispositivo y compartirlos con otras personas.

La versión sin conexión, llamada Scratch Desktop, es una aplicación que se descarga e instala en la computadora. Es útil cuando no se tiene conexión a internet constante o cuando se trabaja en entornos donde la conexión es lenta. Funciona igual que la versión en línea pero sin necesidad de red. Se descarga desde scratch.mit.edu/download.

Versión	¿Cómo se accede?	Ventajas	Cuándo usarla
Scratch en línea	scratch.mit.edu desde el navegador	No requiere instalación. Permite guardar en la nube y compartir.	Trabajo desde casa o universidad con buena conexión
Scratch Desktop	Descarga e instalación local	Funciona sin internet. Más rápido en computadoras lentas.	Trabajo sin conexión o en computadoras compartidas

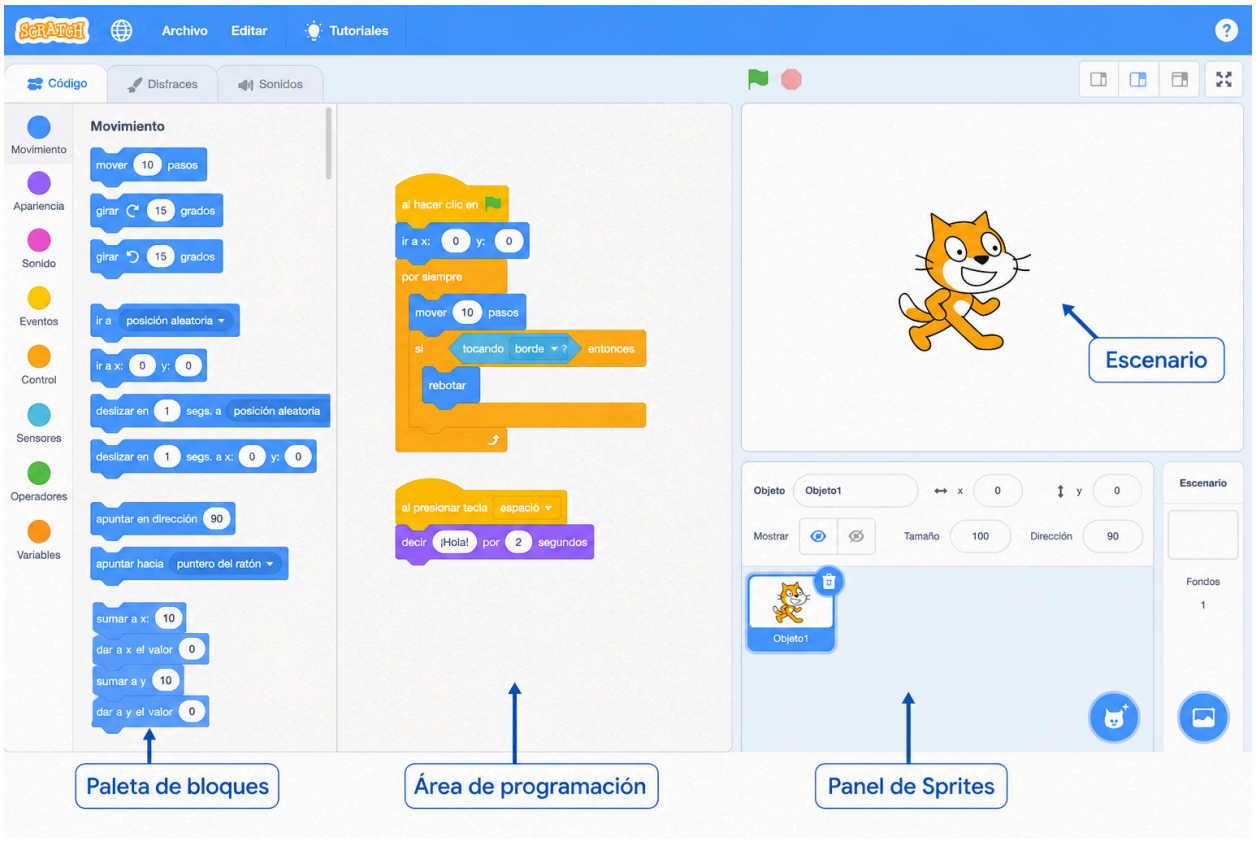


1.1.4 La interfaz de Scratch

Cuando abrimos el editor de Scratch nos encontramos con una pantalla dividida en varias zonas. Entender qué hace cada una desde el principio nos ahorra confusión más adelante. Veamos las cinco zonas principales.

- Escenario (Stage). Es la zona superior derecha, donde vemos lo que ocurre cuando ejecutamos nuestro proyecto. Es el equivalente a la pantalla del juego o de la historia que estamos creando. Mide 480 pixels de ancho por 360 de alto.
- Panel de Sprites. Es la zona debajo del escenario. Aquí aparecen todos los personajes (sprites) que hemos agregado al proyecto. El gato naranja que aparece por defecto es el sprite inicial, pero podemos agregar muchos más.
- Paleta de bloques. Es la columna a la izquierda. Aquí están todos los bloques disponibles, organizados por categorías de colores. Cada categoría agrupa bloques que hacen cosas similares.
- Área de programación (script area). Es la zona grande del centro, donde arrastramos los bloques y los conectamos para crear los scripts. Cada sprite tiene su propia área de programación.

- Pestañas superiores. Permiten cambiar entre Código (los bloques de programación), Disfraces (las apariencias del sprite) y Sonidos (los efectos de audio del sprite).



1.1.5 Las categorías de bloques

Los bloques de Scratch están organizados en nueve categorías principales, cada una con su propio color. Esta organización por colores no es solo decorativa: ayuda a localizar rápidamente el tipo de bloque que necesitamos cuando ya conocemos el sistema.

Categoría	Color	¿Para qué sirve?
Movimiento	Azul	Cambiar posición, dirección y orientación del sprite
Apariencia	Morado	Cambiar disfraz, tamaño, mensajes de diálogo, efectos visuales
Sonido	Rosa	Reproducir sonidos y efectos auditivos
Eventos	Amarillo	Detectar cuándo algo ocurre y disparar acciones

Control	Naranja claro	Estructuras como bucles, condicionales y esperas
Sensores	Celeste	Detectar el ratón, teclas presionadas, colores, distancias
Operadores	Verde	Operaciones matemáticas, lógicas y de texto
Variables	Naranja oscuro	Crear y manipular variables y listas
Mis bloques	Rosa oscuro	Crear bloques personalizados (funciones propias)

Cuando uno está aprendiendo, lo más útil es familiarizarse poco a poco con cada categoría. No hace falta dominar todos los bloques: con unos pocos de cada categoría se pueden hacer proyectos sorprendentes. Las que más vamos a usar en esta unidad son Movimiento, Eventos, Control y Variables.

1.1.6 Lógica de los bloques: encajar como un rompecabezas

Lo que hace especial a Scratch es que los bloques no son intercambiables al azar. Cada bloque tiene una forma específica que indica cómo se conecta con los demás. Aprender a leer esas formas es como aprender el alfabeto de Scratch.

- Bloques de sombrero. Tienen la parte superior curva. Son los que inician un script. Por ejemplo, "al hacer clic en la bandera verde". Solo pueden ir al inicio de una secuencia, nunca al medio o al final.
- Bloques de pila. Tienen una muesca arriba y un saliente abajo, como las piezas de un puzzle. Se apilan uno encima del otro. Ejemplo: "mover 10 pasos". La mayoría de bloques son de este tipo.
- Bloques de envoltura (C-blocks). Tienen forma de C abierta y dentro pueden contener otros bloques. Son los bloques de control: bucles y condicionales. Ejemplo: "repetir 10 veces".
- Bloques reporteros. Son redondeados u ovalados. No se apilan: se encajan dentro de huecos redondos en otros bloques. Representan valores. Ejemplo: el valor de una variable o el resultado de una operación.
- Bloques booleanos. Tienen forma de rombo. Solo se encajan en huecos con forma de rombo. Representan condiciones verdaderas o falsas. Ejemplo: "tocando el borde".

Esta lógica de formas es lo que hace que Scratch sea "a prueba de errores" desde el punto de vista sintáctico. Si intentas conectar un bloque ovalado donde va un bloque de pila, simplemente no se conecta. Esa restricción visual evita los errores que típicamente frustran a quienes empiezan a programar con lenguajes textuales.

Preguntas de reflexión

1. ¿Qué ventaja crees que tiene aprender programación con bloques visuales antes de pasar a un lenguaje textual como Python o JavaScript?

2. ¿Por qué piensas que los creadores de Scratch decidieron usar colores diferentes para cada categoría de bloques?
3. Si una persona ya aprendió Scratch, ¿qué le resultaría más fácil al aprender un lenguaje de programación tradicional: la lógica del programa o la sintaxis específica del lenguaje?

Actividad práctica

Entra a scratch.mit.edu y crea un nuevo proyecto. Tu tarea: explorar el editor sin escribir código todavía. Haz lo siguiente paso a paso:
1. Identifica las cinco zonas de la interfaz (escenario, sprites, paleta de bloques, área de programación, pestañas).
2. Haz clic en cada una de las nueve categorías de bloques y observa qué bloques aparecen en cada una.
3. Arrastra al menos un bloque de cada categoría al área de programación, sin conectarlos todavía.
4. Intenta conectar dos bloques que tengan formas compatibles, y luego intenta conectar dos que no lo sean. Observa qué pasa.Al final, anota en un párrafo qué te resultó más confuso o más sorprendente de la interfaz.

Subtema 1.2: Secuencias, eventos y movimientos

Ya conocemos la interfaz. Ahora vamos a hacer que algo ocurra en la pantalla. En esta sección aprenderemos a construir secuencias de instrucciones, a hacer que esas secuencias se ejecuten cuando ocurre algo específico (un evento) y a controlar los movimientos del sprite por el escenario.

1.2.1 La secuencia: el bloque más básico

Una secuencia en Scratch no es muy diferente a una secuencia en pseudocódigo: es una lista de instrucciones que se ejecutan una tras otra, de arriba hacia abajo, en el orden en que están apiladas. Cada bloque representa una acción, y al apilarlos en orden creamos un script.

Veamos un ejemplo muy simple. Un script que hace que el gato avance, se detenga un momento y diga "¡Hola!":

```
Al hacer clic en la bandera verde
   mover 10 pasos
   esperar 1 segundo
   decir "¡Hola!" durante 2 segundos
```

Cuando hagamos clic en la bandera verde del escenario, este script se ejecutará exactamente en ese orden: primero el sprite se moverá 10 pasos hacia adelante, luego esperará un segundo, y finalmente mostrará un bocadillo con la palabra "¡Hola!" durante dos segundos. Si cambiamos el orden de los bloques, cambia lo que vemos.

Lo importante a notar es que las secuencias en Scratch funcionan exactamente como las que aprendimos en pseudocódigo en la Unidad 3. El concepto es idéntico: una serie de pasos ordenados. Lo que cambia es la forma de expresarlo: en lugar de escribir cada instrucción, las arrastramos como bloques.

1.2.2 Los eventos: ¿cuándo empieza algo?

Un evento es una acción que dispara la ejecución de un script. Sin un evento que inicie el script, ese script no se ejecutará nunca. Por eso todos los scripts en Scratch empiezan con un bloque de sombrero (amarillo) que define el evento desencadenante.

Existen varios eventos disponibles. Los más comunes y útiles son los siguientes:

Evento	¿Cuándo se dispara?	Uso típico
Al hacer clic en bandera verde	Cuando se inicia el proyecto	Inicio principal de cualquier proyecto
Al presionar tecla _____	Cuando se presiona la tecla indicada	Mover sprites, disparar acciones
Al hacer clic en este objeto	Cuando se hace clic sobre el sprite	Botones interactivos, elementos clicables
Al cambiar a fondo _____	Cuando el escenario cambia al fondo indicado	Historias con escenas distintas
Al recibir mensaje _____	Cuando otro sprite envía ese mensaje	Coordinar acciones entre sprites

Los eventos permiten hacer que un mismo proyecto reaccione a diferentes acciones del usuario. Por ejemplo, podemos tener un sprite que se mueva a la derecha cuando presionemos la tecla flecha derecha, a la izquierda con la flecha izquierda, y diga algo cuando hagamos clic sobre él. Cada uno de esos comportamientos sería un script separado, cada uno iniciando con su propio bloque de evento.

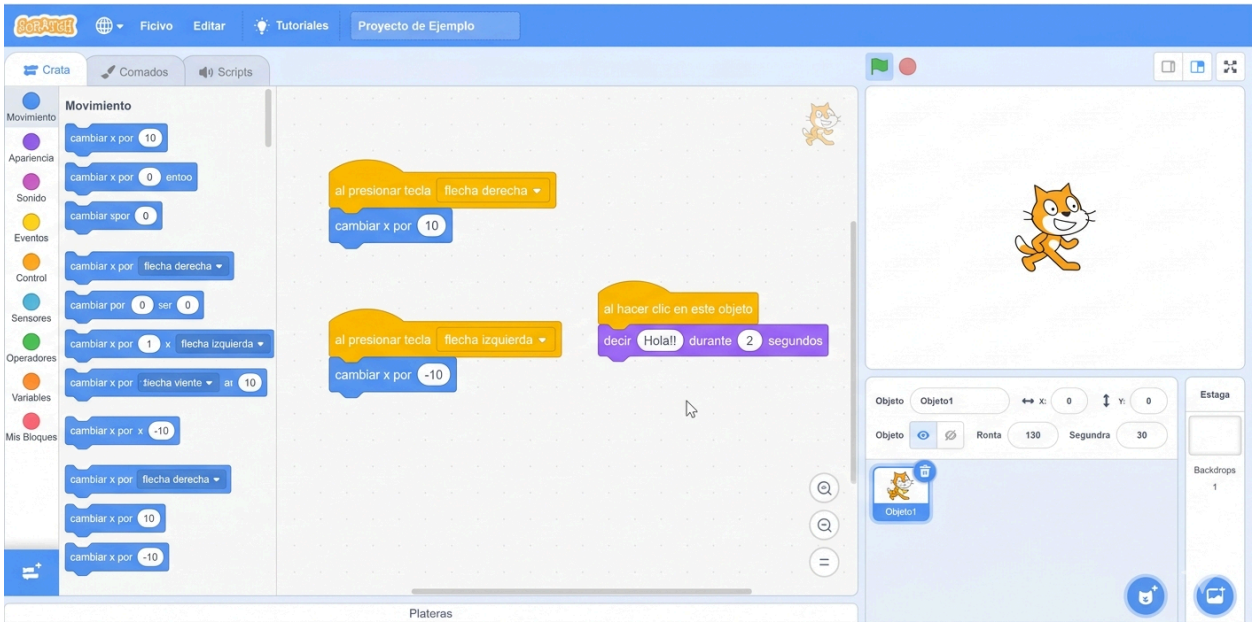
```
Al presionar tecla 'flecha derecha'
   cambiar x por 10

Al presionar tecla 'flecha izquierda'
   cambiar x por -10

Al hacer clic en este objeto
```

```
    decir "¡Me hiciste cosquillas!" durante 2 segundos
```

Estos tres scripts coexisten en el mismo sprite. Cada uno se ejecuta independientemente cuando ocurre su evento. Esto es lo que permite que los proyectos de Scratch sean realmente interactivos: el sprite no hace solo una cosa, reacciona a distintas situaciones.



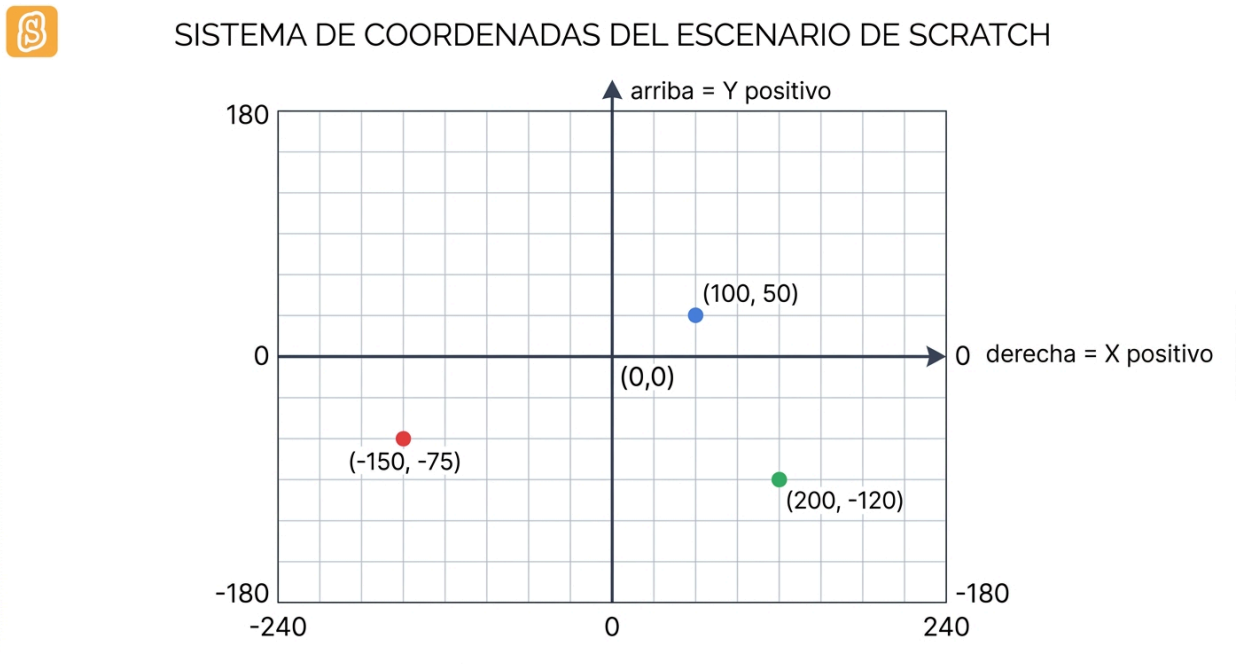
1.2.3 El sistema de coordenadas en Scratch

Para mover sprites con precisión en el escenario, Scratch usa un sistema de coordenadas X-Y. Es el mismo sistema cartesiano que se ve en matemáticas, solo que aplicado a la pantalla.

El centro del escenario tiene coordenadas (0, 0). El eje X va de -240 (extremo izquierdo) a 240 (extremo derecho). El eje Y va de -180 (extremo inferior) a 180 (extremo superior). Por lo tanto, la posición (100, 50) significa 100 unidades a la derecha del centro y 50 unidades arriba del centro.

> **Coordenadas clave en el escenario de Scratch**
>
> Centro: (0, 0)Esquina superior derecha: (240, 180)
> Esquina superior izquierda: (-240, 180)
> Esquina inferior derecha: (240, -180)
> Esquina inferior izquierda: (-240, -180)

Mover hacia la derecha: aumentar X
Mover hacia la izquierda: disminuir X
Mover hacia arriba: aumentar Y
Mover hacia abajo: disminuir Y



1.2.4 Movimientos básicos

Los bloques de la categoría Movimiento (azul) son los que controlan dónde está el sprite y cómo se mueve. Hay muchos, pero los más usados para empezar son cinco.

Bloque	¿Qué hace?	Ejemplo de uso
mover _ pasos	Avanza la cantidad de pasos en la dirección que mira	mover 10 pasos
girar a la derecha _ grados	Rota el sprite a la derecha	girar 90 grados
girar a la izquierda _ grados	Rota el sprite a la izquierda	girar 45 grados
ir a x: _ y: _	Mueve el sprite directamente a esa posición	ir a x: 0 y: 0 (al centro)
deslizar en _ segs a x: _ y: _	Movimiento suave durante el tiempo indicado	deslizar 2 segs a x:100 y:50

cambiar x por _	Aumenta (o disminuye) la posición X	cambiar x por 10
cambiar y por _	Aumenta (o disminuye) la posición Y	cambiar y por -5

La diferencia entre "ir a" y "deslizar" es importante. El bloque "ir a" mueve al sprite instantáneamente: aparece de golpe en la nueva posición. El bloque "deslizar" hace un movimiento gradual durante los segundos que indiquemos: el sprite se desliza suavemente, lo cual se ve mucho más natural en animaciones.

También vale la pena entender la diferencia entre "mover" y "cambiar x/y". El bloque "mover" depende de la dirección que tenga el sprite: si mira hacia la derecha, avanza a la derecha; si mira hacia arriba, avanza hacia arriba. El bloque "cambiar x" siempre afecta la coordenada horizontal sin importar la dirección del sprite. Para juegos donde el sprite gira en distintas direcciones, "mover" es más útil. Para movimientos puramente horizontales o verticales como en plataformas, "cambiar x" y "cambiar y" son más precisos.

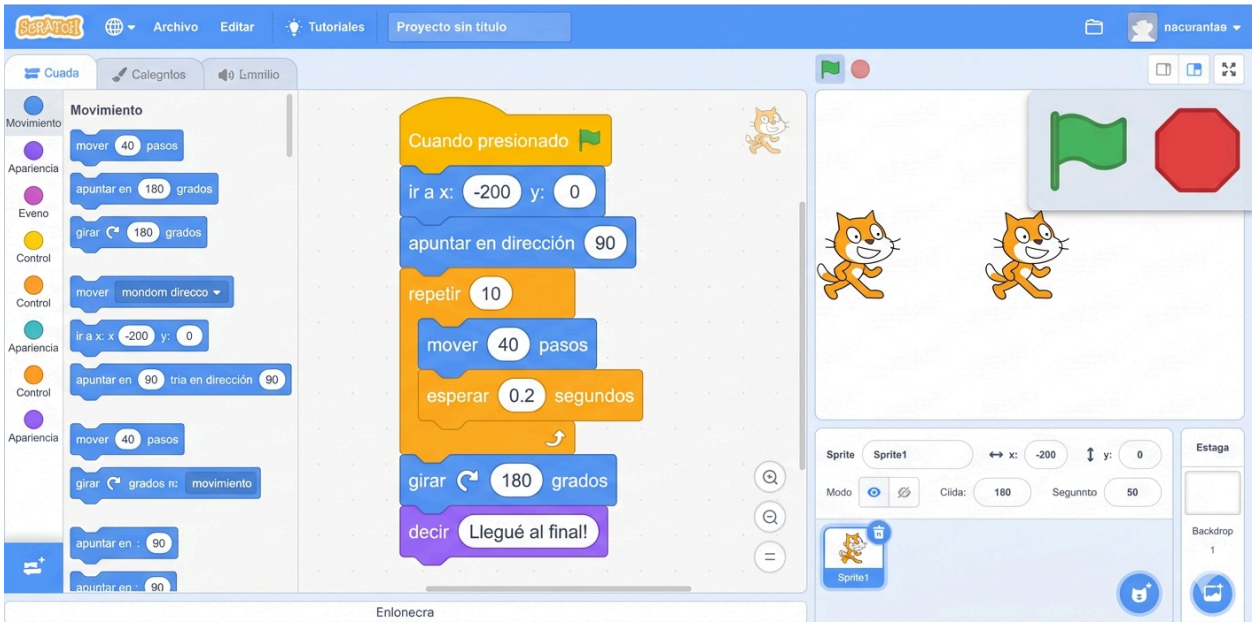
1.2.5 Ejemplo completo: el sprite que camina

Pongamos todo junto en un primer proyecto completo. Vamos a hacer que el gato camine de izquierda a derecha del escenario, luego se gire y diga algo.

```
Al hacer clic en la bandera verde
   ir a x: -200 y: 0
   apuntar en dirección 90  (a la derecha)
   repetir 10 veces
      mover 40 pasos
      esperar 0.2 segundos
   girar a la derecha 180 grados
   decir "¡Llegué al final!" durante 2 segundos
```

Analicemos qué hace este script. Primero coloca al sprite en la parte izquierda del escenario, en posición (-200, 0). Luego lo apunta hacia la derecha. Después entra en un bucle que se repite 10 veces, donde cada vuelta avanza 40 pasos y espera 0.2 segundos. Al terminar el bucle, el sprite habrá avanzado en total 400 pasos (10 × 40). Finalmente gira 180 grados y muestra un mensaje.

Lo interesante es que aquí vemos cómo se conectan los conceptos de la Unidad 3 con Scratch. El bucle "repetir 10 veces" es exactamente el equivalente al bucle PARA que vimos antes, solo que expresado como un bloque que contiene otros bloques dentro de su forma de C.



Tu primer reto: modifica el ejemplo

Toma el ejemplo del sprite que camina y modifícalo así:
1. Haz que en lugar de moverse en línea recta, suba y baje en zigzag. Pista: alterna 'cambiar y por 30' y 'cambiar y por -30' dentro del bucle.
2. Haz que el sprite cambie de disfraz mientras camina, para que parezca que está animado. Pista: agrega el bloque 'siguiente disfraz' dentro del bucle.
3. Cuando termine de caminar, en lugar de decir 'Llegué al final', haz que diga la posición exacta donde quedó. Pista: usa el bloque 'decir' con el bloque reportero 'posición x'.

Preguntas de reflexión

1. ¿Qué diferencia hace usar 'deslizar' en lugar de 'ir a' al mover un sprite? ¿Cuándo elegirías uno u otro?
2. Si hago clic en la bandera verde pero no he conectado mi script a ese evento, ¿qué pasa al ejecutar el proyecto?
3. ¿Por qué un mismo sprite puede tener varios scripts ejecutándose 'al mismo tiempo'? ¿Qué los activa?

Actividad práctica

Crea un nuevo proyecto en Scratch llamado 'Mi primer sprite'. Cumple los siguientes objetivos:
1. Cuando se haga clic en la bandera verde, el sprite debe comenzar en la posición (0, 0).
2. Al presionar la tecla 'flecha derecha', el sprite avanza 20 pasos a la derecha.
3. Al presionar la tecla 'flecha izquierda', el sprite avanza 20 pasos a la izquierda.
4. Al hacer clic sobre el sprite, este debe girar 90 grados y decir 'Soy interactivo' durante 2 segundos.
5. Guarda el proyecto con tu nombre en el título.
Entrega capturas de pantalla del editor mostrando los bloques de cada script y una captura del escenario con el sprite en movimiento.

Cierre del Tema 1

En este primer tema dimos los primeros pasos en Scratch. Conocimos su origen como herramienta del MIT Media Lab, entendimos por qué la programación visual es valiosa como puente hacia los lenguajes textuales, aprendimos a acceder al editor en línea y nos familiarizamos con la interfaz y sus cinco zonas principales. Vimos cómo se organizan los bloques por colores en nueve categorías y cómo sus formas determinan qué se puede conectar con qué. Luego construimos nuestras primeras secuencias, aprendimos a disparar acciones con eventos, entendimos el sistema de coordenadas del escenario y dominamos los bloques de movimiento básicos. Con esto ya podemos hacer sprites que se mueven, reaccionan a eventos y responden al usuario. Eso es la base de cualquier proyecto interactivo.

Actividad A – Familiarización con bloques (20 min)

Crea un proyecto vacío en Scratch. Sin ver tutoriales ni guías adicionales, intenta hacer que tu sprite haga estas seis acciones, cada una en un script separado:
– Al iniciar, ir al centro del escenario.
– Al presionar la tecla espacio, decir 'Hola' durante 2 segundos.
– Al presionar la tecla flecha arriba, subir 10 pasos.
– Al presionar la tecla flecha abajo, bajar 10 pasos.
– Al hacer clic en el sprite, cambiar de tamaño (más grande o más pequeño).
– Al hacer clic en el sprite por segunda vez, volver al tamaño original.

Actividad B – Carrera de sprites (25 min)

Trabajando en parejas, cada uno tendrá su propio sprite en el mismo proyecto. Diseñen una pequeña carrera:
– Ambos sprites empiezan en la esquina izquierda del escenario.
– Al hacer clic en la bandera verde, ambos comienzan a avanzar.

> – Cada uno debe avanzar de forma 'aleatoria': dentro de un bucle, alternar entre pasos de 10 y pasos de 20 con esperas distintas.
> – El primero que llegue al borde derecho del escenario gana y debe decir '¡Gané!'.
> Usen sensores básicos (tocando el borde) si ya conocen esa categoría, o calculen visualmente cuándo cada sprite llegará al final.

Tema 2: Desarrollo de Soluciones Interactivas

En el Tema 1 aprendimos los fundamentos del entorno. Ahora vamos a dar el salto a proyectos reales. Para eso necesitamos dos cosas más: aprender a usar variables en Scratch para guardar datos durante la ejecución, y combinar todo lo aprendido para construir proyectos interactivos completos.

Subtema 2.1: Variables y datos

Una variable, como vimos en la Unidad 3, es un espacio donde guardamos un valor que puede cambiar durante la ejecución del programa. En Scratch, las variables funcionan exactamente igual. La única diferencia es que aquí también podemos verlas en pantalla durante la ejecución, lo que las hace mucho más fáciles de entender para quienes recién empiezan.

2.1.1 Variables en Scratch

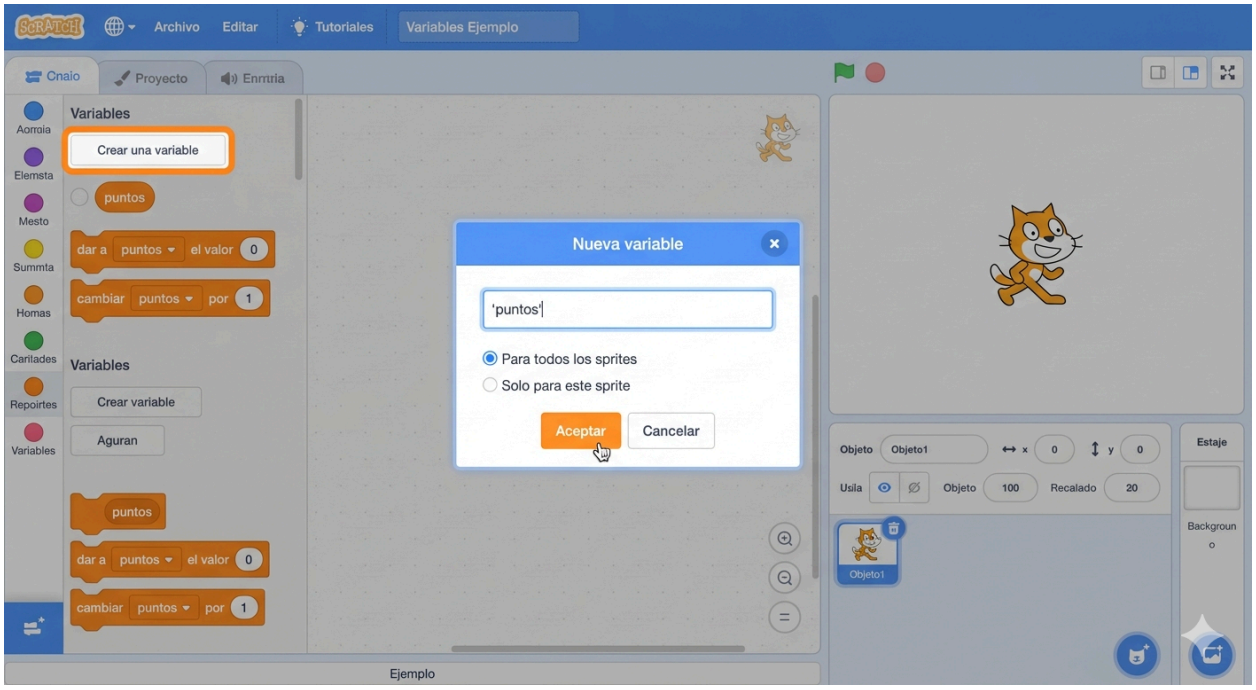
Las variables en Scratch viven en la categoría Variables (color naranja oscuro). A diferencia de los lenguajes tradicionales donde declaramos variables escribiendo código, en Scratch las creamos haciendo clic en un botón. Una vez creadas, aparecen como bloques nuevos que podemos arrastrar a nuestros scripts.

Para crear una variable se hace clic en "Crear una variable", se le da un nombre descriptivo y se decide si será visible para todos los sprites del proyecto o solo para uno. Para empezar, dejamos la opción "para todos los sprites" (variable global), que es lo más cómodo en proyectos pequeños.

Una vez creada la variable, aparecen varios bloques nuevos en la paleta:

Bloque	¿Qué hace?	Ejemplo de uso
dar a (var) el valor _	Asigna un valor inicial o lo reemplaza	dar a puntos el valor 0
cambiar (var) por _	Suma (o resta) un valor a la variable actual	cambiar puntos por 1
mostrar variable (var)	Muestra el valor en el escenario	Útil al depurar

esconder variable (var)	Oculta el valor del escenario	Cuando no queremos que se vea
(var) (reportero)	Representa el valor actual de la variable	Se usa dentro de otros bloques



2.1.2 Mostrando una variable en pantalla

Una ventaja muy didáctica de Scratch es que las variables se pueden mostrar directamente en el escenario mientras el proyecto se ejecuta. Esto permite ver en tiempo real cómo cambia su valor, lo que ayuda enormemente a entender qué está pasando dentro del programa.

Cuando creamos una variable, automáticamente aparece en el escenario un pequeño rectángulo con el nombre de la variable y su valor actual. Podemos arrastrarlo donde queramos, ocultarlo si no nos interesa verlo, o cambiar su estilo visual. Esto es muy útil para crear marcadores de puntos en juegos, contadores de vidas, cronómetros, o cualquier indicador numérico.

2.1.3 Operadores en Scratch

Los operadores que vimos en la Unidad 3 (aritméticos, relacionales y lógicos) también están disponibles en Scratch, en la categoría Operadores (verde). La diferencia es que aquí se usan como bloques ovalados o de rombo que encajan dentro de otros bloques.

Tipo	Bloque en Scratch	Equivalente en pseudocódigo
Aritmético	_ + _ , _ - _ , _ * _ , _ / _	+, -, *, /
Módulo	_ mod _	mod o %
Aleatorio	número aleatorio entre _ y _	Sin equivalente directo, útil para juegos
Relacional	_ < _ , _ = _ , _ > _	<, =, >
Lógico	_ Y _ , _ O _ , no _	AND, OR, NOT
Texto	unir _ _ , largo de _ , letra _ de _	Concatenación y manipulación de texto

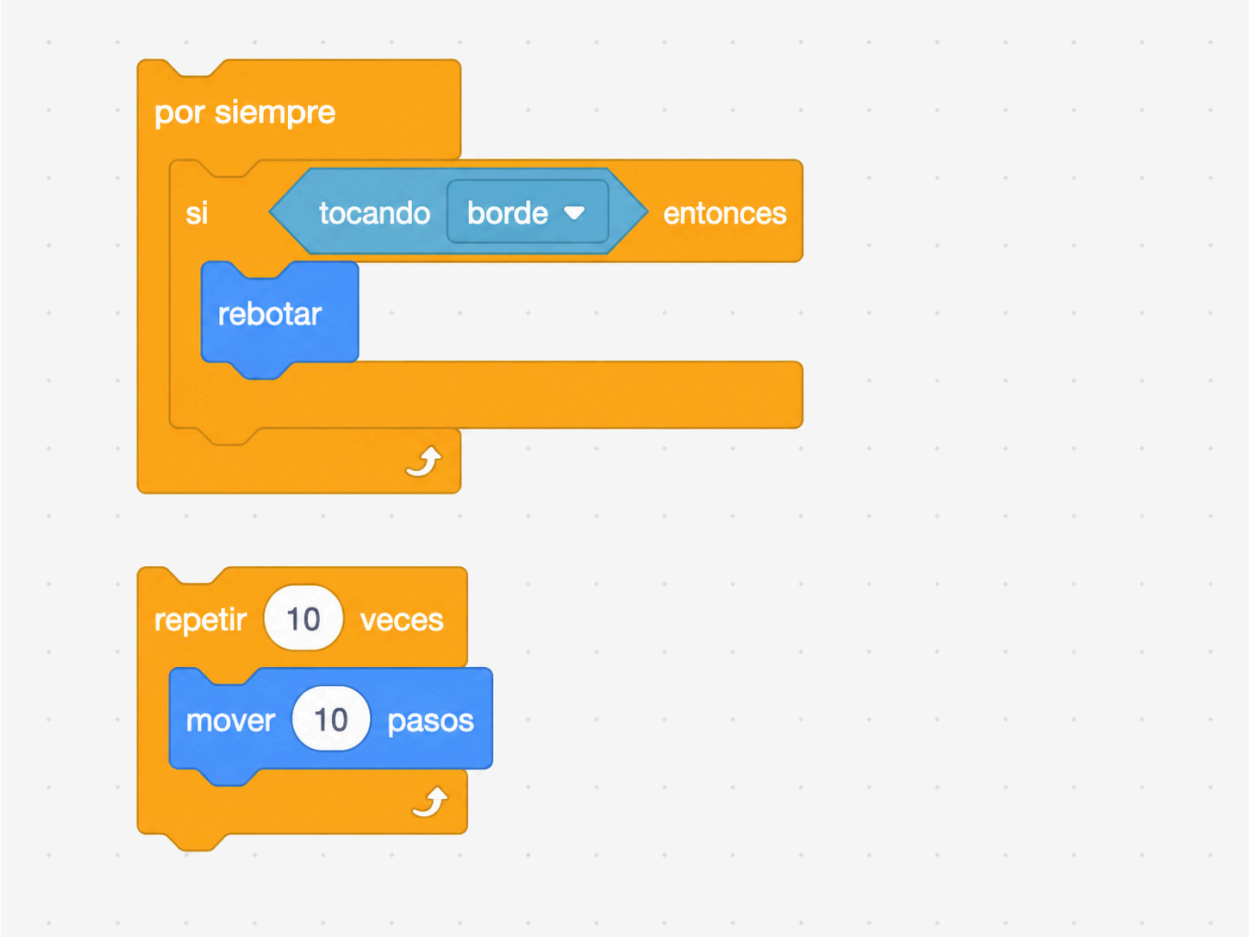
El bloque "número aleatorio entre _ y _" merece una mención especial. Es muy útil en juegos: nos permite hacer que un sprite aparezca en una posición aleatoria, que un enemigo se mueva de forma impredecible, o que se haga una pregunta diferente cada vez. Por ejemplo, "ir a x: (aleatorio entre -200 y 200) y: (aleatorio entre -150 y 150)" coloca al sprite en cualquier punto del escenario al azar.

2.1.4 Control de flujo en Scratch

Los condicionales y bucles que aprendimos en la Unidad 3 también existen en Scratch, en la categoría Control (color naranja claro). Tienen forma de C, lo que significa que pueden envolver otros bloques dentro.

Bloque	¿Qué hace?	Equivalente en pseudocódigo
repetir _ veces	Ejecuta los bloques internos N veces	PARA i DESDE 1 HASTA N
por siempre	Repite los bloques internos infinitamente	MIENTRAS verdadero (bucle infinito)
si _ entonces	Condicional simple	SI condición ENTONCES
si _ entonces / si no	Condicional compuesta	SI condición ENTONCES ... SINO
repetir hasta que _	Repite hasta que la condición sea verdadera	REPETIR ... HASTA condición
esperar _ segundos	Pausa la ejecución el tiempo indicado	No hay equivalente directo
esperar hasta que _	Pausa hasta que la condición sea verdadera	Espera basada en evento

Una novedad útil de Scratch es el bucle "por siempre". En pseudocódigo evitábamos los bucles infinitos porque colgaban el programa, pero en Scratch este bucle es perfectamente válido y se usa todo el tiempo. Por ejemplo, para que un sprite esté siempre detectando si el usuario presionó una tecla, encerramos esa lógica en un bucle "por siempre". El bucle solo se detiene cuando se detiene el proyecto entero o cuando dentro se ejecuta un bloque "detener este script".



2.1.5 Ejemplo: el contador interactivo

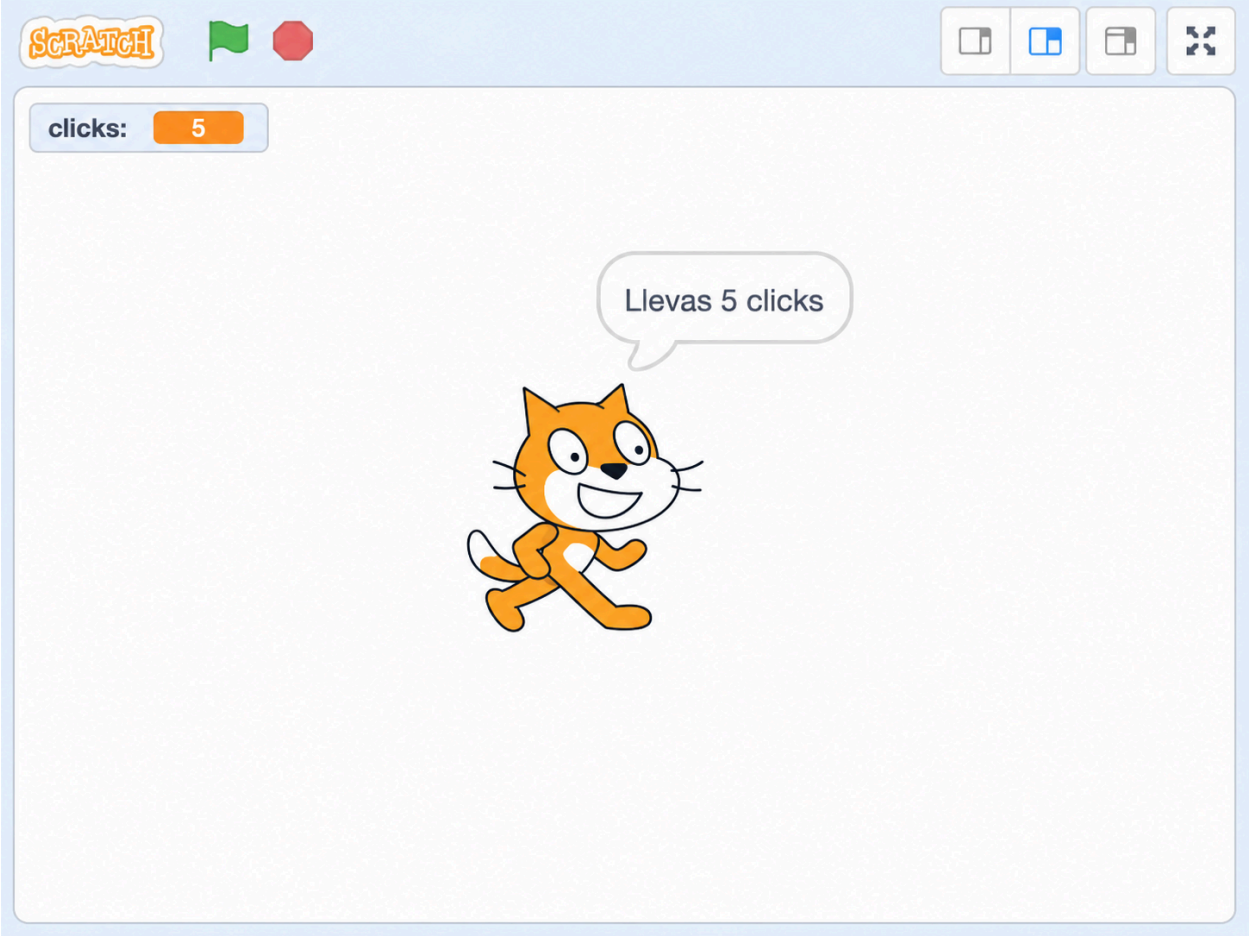
Pongamos en práctica lo aprendido. Vamos a construir un contador que aumenta cada vez que hacemos clic sobre el sprite. Este pequeño proyecto combina variables, eventos y operaciones básicas.

Paso a paso: primero creamos una variable llamada "clicks" y la inicializamos en cero. Luego hacemos que al hacer clic en el sprite, el contador aumente en uno y el sprite diga el valor actual. Cuando llegue a 10, mostraremos un mensaje especial.

```
Al hacer clic en la bandera verde
   dar a clicks el valor 0

Al hacer clic en este objeto
   cambiar clicks por 1
   decir (unir "Llevas " (unir clicks " clicks")) durante 1
segundo
   si <clicks = 10> entonces
      decir "¡Felicitaciones! Llegaste a 10 clicks" durante 3
segundos
      detener todos
   fin si
```

Al iniciar el proyecto, el contador se pone en cero. Cada vez que hacemos clic sobre el sprite, el contador aumenta y el sprite nos dice cuántos clicks llevamos. Cuando llega a 10, muestra un mensaje de felicitación y detiene el proyecto. En este script estamos usando: una variable, dos eventos distintos, una operación de asignación, un operador de unión de texto (concatenación), un condicional simple y un comparador de igualdad.



Preguntas de reflexión

1. ¿En qué se parece el bloque 'cambiar puntos por 1' en Scratch al pseudocódigo 'puntos = puntos + 1' de la Unidad 3?
2. ¿Por qué crees que es útil que las variables se vean en el escenario mientras el proyecto se ejecuta?
3. ¿En qué situaciones usarías el bucle 'por siempre' en lugar de 'repetir N veces'?

Actividad práctica

Crea un proyecto en Scratch llamado 'Mini calculadora visual'. Debe hacer lo siguiente:
1. Crea dos variables: 'numero1' y 'numero2'.
2. Al iniciar el proyecto, el sprite pregunta '¿Cuál es el primer número?' usando el bloque 'preguntar y esperar' de la categoría Sensores. La respuesta del usuario debe guardarse en 'numero1'.
3. Luego pregunta el segundo número y lo guarda en 'numero2'.
4. Calcula la suma y muestra el resultado diciendo: 'La suma es: [resultado]'.
5. Crea una versión adicional que muestre, además de la suma, la resta, la multiplicación y la división del usuario.Entrega capturas de pantalla del editor con todos los bloques visibles y una captura del escenario durante la ejecución mostrando los valores de las variables.

Subtema 2.2: Creación de proyectos interactivos

En este último subtema. Ya tenemos todas las piezas: la interfaz, los bloques, las secuencias, los eventos, los movimientos, las variables, los operadores y las estructuras de control. En esta sección vamos a combinarlas para construir tres proyectos interactivos completos. Cada uno es más complejo que el anterior, pero todos están al alcance de quien siguió la guía hasta aquí.

2.2.1 Pasos para diseñar un proyecto

Antes de empezar a arrastrar bloques, los buenos proyectos en Scratch siguen un proceso de diseño. No es un proceso rígido, pero seguirlo ahorra mucho tiempo y evita confusiones más adelante. Lo dividimos en cinco pasos.

1. Definir qué hará el proyecto. En una frase corta, escribe qué quieres construir. Ejemplo: 'Un juego donde un gato debe atrapar manzanas que caen del cielo'.

2. Identificar los sprites necesarios. Lista los personajes u objetos que aparecerán. Para el ejemplo del juego: un gato (jugador), manzanas (objetos a atrapar), y posiblemente un fondo.
3. Listar las acciones de cada sprite. Para cada sprite, escribe en lenguaje natural qué debe hacer. Esto es como hacer un pseudocódigo simplificado antes de programar.
4. Diseñar las variables necesarias. ¿Vas a llevar un puntaje? ¿Vas a contar vidas? ¿Vas a medir tiempo? Cada uno requiere una variable.
5. Implementar paso a paso. Empieza con un solo sprite y un solo comportamiento. Pruébalo. Si funciona, agrega el siguiente. Construir todo de un golpe y probarlo al final es la receta segura para no poder encontrar errores.

> **Tip de oro: prueba siempre con la bandera verde**
>
> Cada vez que agregues unos pocos bloques al proyecto, haz clic en la bandera verde para ver si funciona como esperas. Es mucho mejor encontrar un error después de agregar 5 bloques que después de agregar 50. A esto se le llama programación incremental y es una de las mejores prácticas en cualquier lenguaje, no solo en Scratch.

2.2.2 Proyecto 1: Atrapa la manzana

Este es un juego clásico para empezar. El jugador controla un gato con las flechas izquierda y derecha. Del cielo caen manzanas en posiciones aleatorias. El objetivo es atraparlas tocándolas. Cada manzana atrapada suma un punto.

- Sprites necesarios: el gato (jugador) y la manzana.
- Variables: 'puntos' (entero, comienza en 0).
- Lógica del gato: se mueve a la izquierda y derecha con las flechas; permanece a una altura fija en la parte baja del escenario.
- Lógica de la manzana: aparece en la parte superior con posición X aleatoria, cae lentamente hacia abajo; si toca al gato, suma un punto y vuelve a aparecer arriba en posición aleatoria; si llega al suelo sin ser atrapada, también vuelve arriba pero sin sumar puntos.

Script del gato:

```
Al hacer clic en la bandera verde
   ir a x: 0 y: -150
   dar a puntos el valor 0
```

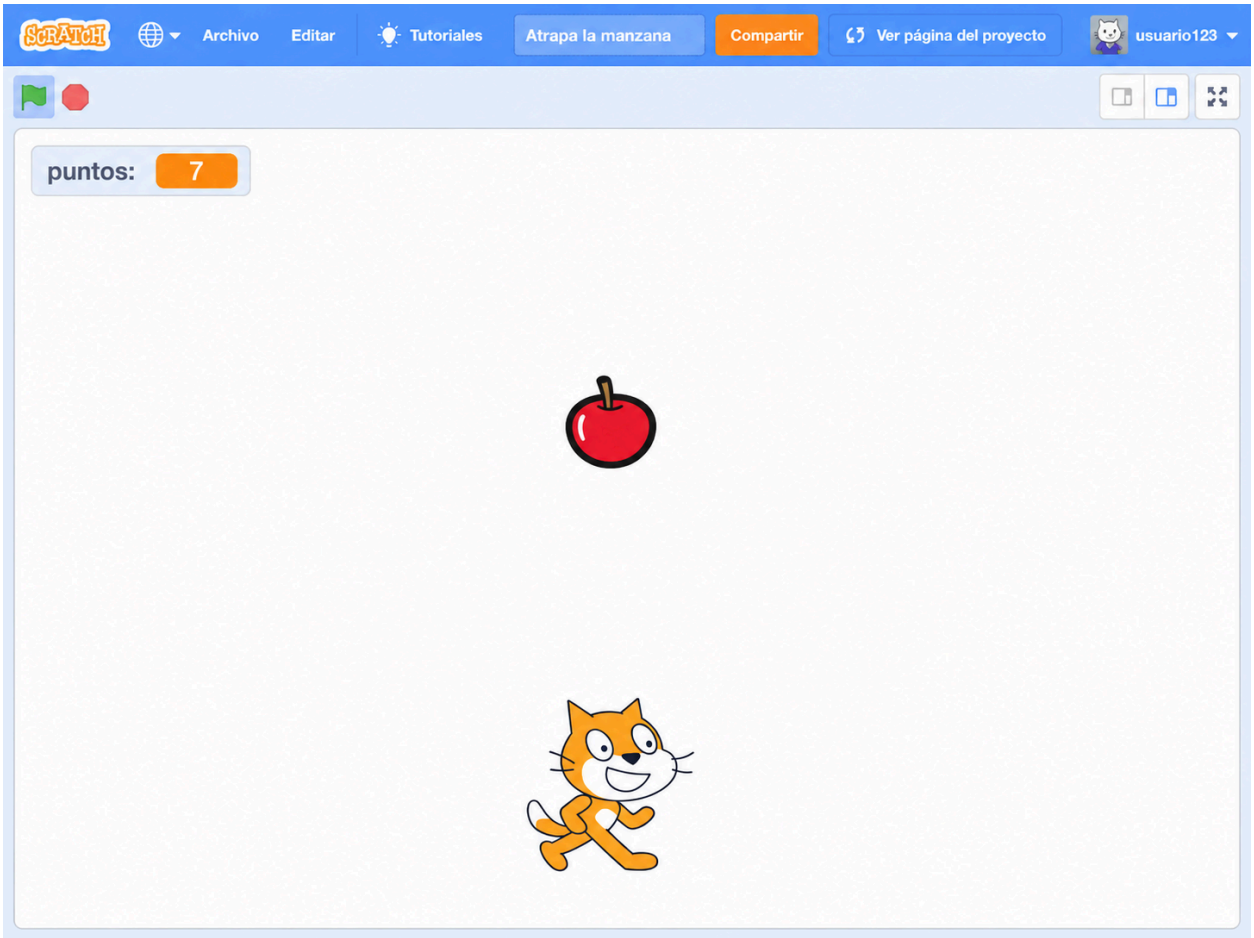
```
Al presionar tecla 'flecha derecha'
   cambiar x por 15

Al presionar tecla 'flecha izquierda'
   cambiar x por -15
```

Script de la manzana:

```
Al hacer clic en la bandera verde
   por siempre
      ir a x: (número aleatorio entre -200 y 200) y: 180
      repetir hasta que <tocando 'Gato' o (posición y) < -170>
         cambiar y por -5
         esperar 0.05 segundos
      si <tocando 'Gato'> entonces
         cambiar puntos por 1
      fin si
```

Analicemos lo que ocurre. El gato comienza en la parte baja del centro del escenario y los puntos se ponen en cero. Al presionar las flechas, el gato se mueve horizontalmente. La manzana entra en un bucle infinito ("por siempre"): aparece arriba en posición X aleatoria, luego cae bajando 5 pixels cada 0.05 segundos hasta que toca al gato o llega abajo. Si tocó al gato, suma un punto. Sea como sea, vuelve a aparecer arriba para repetir el ciclo.



Mejoras opcionales para el juego

Una vez que tengas el juego básico funcionando, puedes agregar:
– Un cronómetro: crea una variable 'tiempo', inicialízala en 30 al iniciar y haz que disminuya en 1 cada segundo (con un bucle 'por siempre' que contenga 'esperar 1 segundo' y 'cambiar tiempo por -1'). Cuando tiempo = 0, detén el juego y muestra los puntos finales.
– Manzanas que caen más rápido conforme avanza el juego: en lugar de 'cambiar y por -5', usa 'cambiar y por (- (5 + puntos))'. Cada punto sumado hace que las manzanas caigan un poco más rápido.
– Sonidos: agrega un sonido cuando se atrapa una manzana usando el bloque 'tocar sonido' de la categoría Sonido.

2.2.3 Proyecto 2: Historia interactiva con diálogos

Este proyecto cambia de género: en lugar de un juego, vamos a construir una pequeña historia interactiva. Es un proyecto excelente para practicar el envío de mensajes entre sprites, que es la forma estándar de coordinar acciones complejas.

- Sprites necesarios: dos personajes que conversen (por ejemplo, un gato y un perro).
- Variables: no son necesarias para este proyecto básico.
- Lógica: el gato dice algo, luego envía un mensaje al perro para que responda; el perro responde y envía otro mensaje al gato; así sucesivamente.

Script del gato:

```
Al hacer clic en la bandera verde
   ir a x: -150 y: 0
   decir "Hola, ¿cómo estás?" durante 2 segundos
   enviar mensaje 'turno_perro'

Al recibir mensaje 'turno_gato'
   decir "¡Genial! Yo también." durante 2 segundos
   decir "¿Te gustaría dar un paseo?" durante 2 segundos
   enviar mensaje 'fin_dialogo'
```

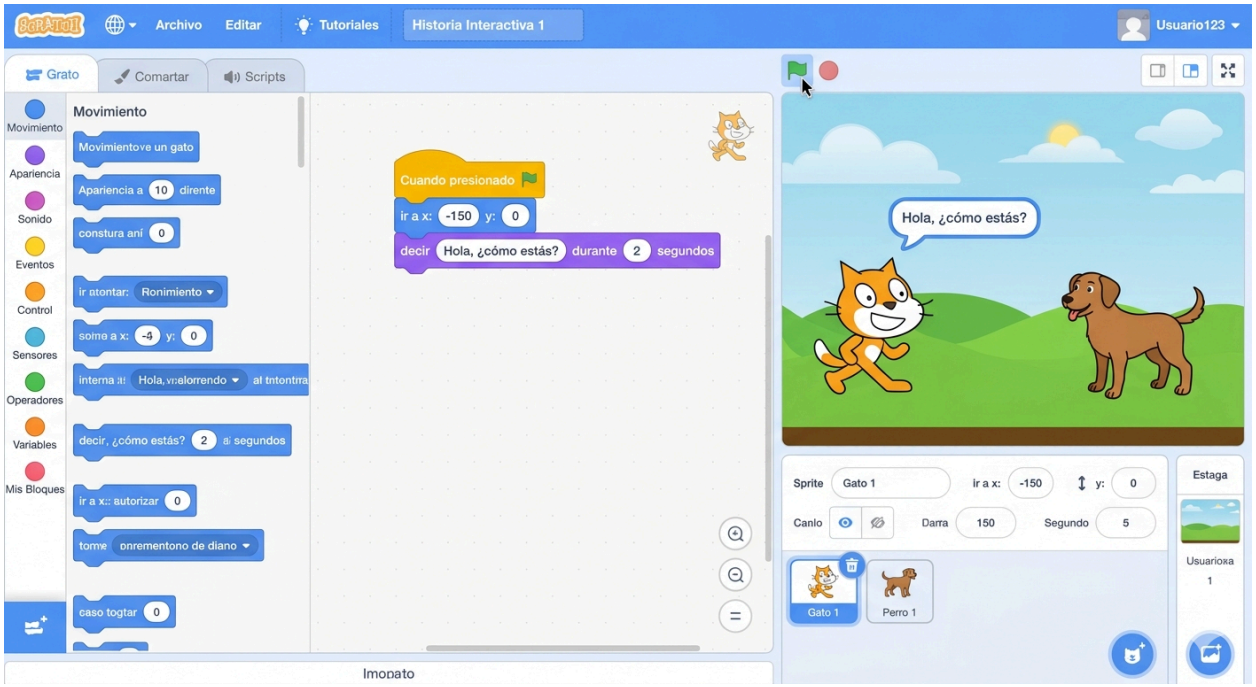
Script del perro:

```
Al hacer clic en la bandera verde
   ir a x: 150 y: 0

Al recibir mensaje 'turno_perro'
   decir "¡Muy bien, gracias!" durante 2 segundos
   decir "¿Y tú?" durante 2 segundos
   enviar mensaje 'turno_gato'

Al recibir mensaje 'fin_dialogo'
   decir "¡Vamos!" durante 2 segundos
```

Los bloques "enviar mensaje" y "al recibir mensaje" son el sistema de comunicación entre sprites en Scratch. Cuando un sprite envía un mensaje, todos los demás sprites pueden detectarlo y reaccionar. Esto permite coordinar acciones que ocurren en secuencia entre varios personajes, sin que cada uno tenga que saber lo que están haciendo los demás.



2.2.4 Proyecto 3: Calculadora simple

El tercer proyecto demuestra cómo Scratch puede usarse incluso para hacer una pequeña aplicación práctica: una calculadora que suma, resta, multiplica o divide dos números según lo que elija el usuario.

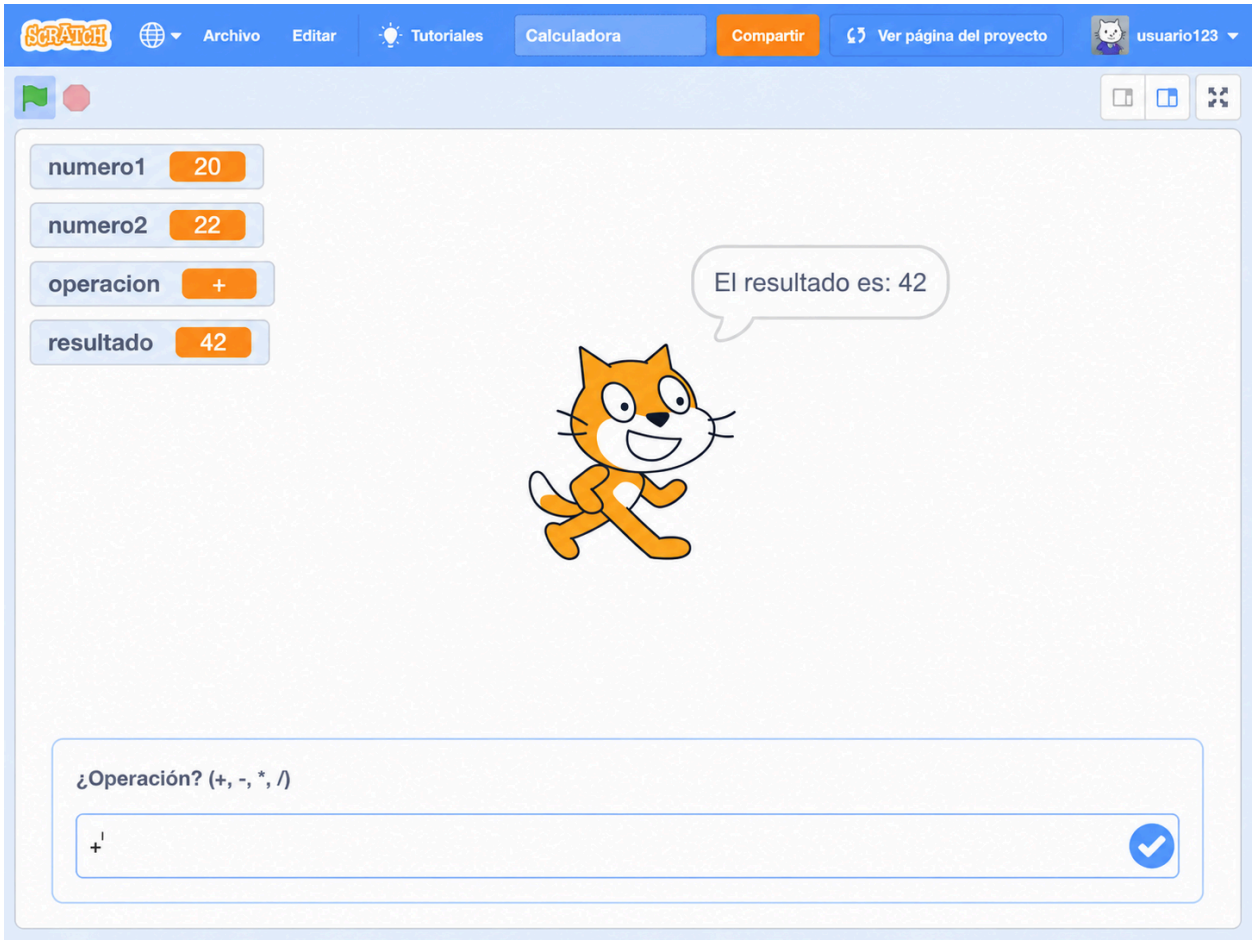
- Sprite necesario: cualquier sprite que actúe como 'asistente' (puede ser el gato).
- Variables: 'numero1', 'numero2', 'operacion' y 'resultado'.
- Lógica: el sprite pregunta los dos números y la operación deseada, hace el cálculo y muestra el resultado. Si la operación es inválida o si se intenta dividir entre cero, muestra un mensaje de error.

```
Al hacer clic en la bandera verde
   preguntar "¿Primer número?" y esperar
   dar a numero1 el valor (respuesta)
   preguntar "¿Segundo número?" y esperar
   dar a numero2 el valor (respuesta)
   preguntar "¿Operación? (+, -, *, /)" y esperar
   dar a operacion el valor (respuesta)
```

```
   si <operacion = "+"> entonces
      dar a resultado el valor (numero1 + numero2)
   si no
      si <operacion = "-"> entonces
         dar a resultado el valor (numero1 - numero2)
      si no
         si <operacion = "*"> entonces
            dar a resultado el valor (numero1 * numero2)
         si no
            si <operacion = "/"> entonces
               si <numero2 = 0> entonces
                  decir "Error: división entre cero" durante 3
segundos
                  detener todos
               fin si
               dar a resultado el valor (numero1 / numero2)
            si no
               decir "Operación no válida" durante 3 segundos
               detener todos
            fin si
         fin si
      fin si
   fin si

   decir (unir "El resultado es: " resultado) durante 4 segundos
```

Este proyecto pone en juego prácticamente todos los conceptos que hemos visto: variables, entrada de datos del usuario, condicionales anidadas (mira cuántos SI dentro de otros SI), operadores aritméticos, operadores de comparación, manejo de errores y concatenación de texto. Si seguiste la guía hasta aquí y este código tiene sentido para ti, has dominado los fundamentos.



2.2.5 Cómo guardar, compartir y depurar tu proyecto

Construir el proyecto es una parte. Saber guardarlo, compartirlo y arreglar cuando algo falla es la otra parte que vale la pena conocer.

Si trabajas con Scratch en línea y tienes una cuenta, los proyectos se guardan automáticamente en la nube. Aparecen en "Mis cosas" en tu cuenta, y puedes acceder a ellos desde cualquier dispositivo. Si trabajas sin cuenta o con Scratch Desktop, puedes guardar el proyecto en tu computadora como un archivo .sb3 desde el menú Archivo.

Para compartir un proyecto con otra persona, hay dos opciones. La primera es hacerlo público dentro de la comunidad de Scratch desde el botón "Compartir" en tu cuenta. Esto genera un enlace que cualquiera puede ver y remixar. La segunda es descargar el archivo .sb3 y enviarlo manualmente.

Cuando algo no funciona: la depuración

Es completamente normal que un proyecto no funcione como esperabas la primera vez. Eso es parte del proceso. Estas son las técnicas más útiles para encontrar y corregir errores en Scratch:
– Ejecuta paso a paso: haz clic directamente sobre un bloque o conjunto de bloques en el área de programación para ejecutarlos sin disparar el evento. Esto permite probar partes aisladas del script.
– Muestra variables temporalmente: aunque al final no quieras ver una variable en el escenario, durante la depuración es muy útil mostrarla para ver cómo cambia su valor.– Usa el bloque 'decir' para mostrar mensajes intermedios: si un script no hace lo que esperas, agrega un 'decir' en partes clave para ver hasta dónde llega la ejecución y con qué valores.
– Comenta dudas con un compañero: explicar el problema en voz alta muchas veces ayuda a encontrar la solución por uno mismo. A esto se le conoce como 'depuración del patito de goma'.

Preguntas de reflexión

1. ¿Por qué es importante dividir un proyecto grande en pasos pequeños y probarlos uno por uno?
2. ¿En qué situaciones es útil usar mensajes entre sprites en lugar de manejar todo desde un solo sprite?
3. ¿Cómo se relaciona el proyecto de la Calculadora con el pensamiento computacional que estudiamos en la Unidad 2 (descomposición, abstracción, algoritmia)?

Actividad práctica – Tu propio proyecto

Diseña y construye un proyecto interactivo original en Scratch. Tienes total libertad sobre el tema. Algunas ideas: un test de cultura general con varias preguntas, un mini juego de adivinar el número, una historia interactiva donde el usuario decide qué pasa, un cronómetro o temporizador visual.
Requisitos mínimos:
– Mínimo 2 sprites distintos.
– Mínimo 2 variables que cambien durante la ejecución.
– Al menos un evento de bandera verde, uno de tecla y uno de clic.
– Al menos un condicional y un bucle.
– Mínimo un operador aritmético y uno relacional.
Documenta tu proyecto explicando en un párrafo: qué hace, cómo se usa y qué fue lo más difícil de construir.

Cierre de la Unidad 4

En esta última unidad llevamos a la práctica todo lo aprendido. Comenzamos conociendo Scratch como herramienta, su origen en el MIT, su filosofía de aprendizaje y la lógica detrás de su sistema de bloques. Exploramos la interfaz, las nueve categorías de bloques y las cinco formas geométricas que determinan cómo se conectan entre sí.

Luego nos lanzamos a construir nuestras primeras secuencias, aprendimos a usar eventos para hacer los proyectos interactivos, dominamos el sistema de coordenadas del escenario y los bloques de movimiento. Vimos cómo cada concepto que aprendimos en pseudocódigo tiene su equivalente visual en Scratch, lo que confirma que la lógica de programar es universal: solo cambia la forma de expresarla.

En el Tema 2 trabajamos con variables, operadores y estructuras de control en Scratch, y construimos tres proyectos completos: un juego de atrapar manzanas, una historia interactiva con diálogos coordinados por mensajes, y una calculadora con manejo de errores. Cada proyecto integró todos los conceptos previos.

Llegado hasta este punto, ya tenemos los fundamentos sólidos del pensamiento computacional y la programación. Y además ya aprendimos a pensar como programador.

Tareas de la Unidad 4

Tarea 1 – Proyecto guiado completo (30–45 minutos)

Construye desde cero el juego 'Atrapa la manzana' de la sección 2.2.2. Después agrégale al menos dos mejoras de la lista de mejoras opcionales (cronómetro, velocidad creciente, sonidos, o cualquier otra que se te ocurra).
Entregar: enlace al proyecto compartido en scratch.mit.edu (o archivo .sb3) y un documento corto que explique qué mejoras agregaste y cómo las implementaste con bloques. Incluye capturas del editor mostrando los scripts y del escenario durante el juego.

Tarea 2 – Proyecto libre con criterios (45–60 minutos)

Diseña tu propio proyecto interactivo siguiendo los pasos de diseño de la sección 2.2.1. Puede ser un juego, una historia, una simulación, una herramienta utilitaria, lo que prefieras. El proyecto debe cumplir los siguientes criterios técnicos:
– Al menos 3 sprites con scripts propios.– Al menos 3 variables que cambien dinámicamente.
– Uso de mensajes entre sprites (enviar y recibir).
– Al menos un condicional simple y uno compuesto.– Al menos un bucle por siempre y un bucle repetir N veces.
– Uso de números aleatorios en alguna parte del proyecto.

> – Manejo de al menos una entrada del usuario (con preguntar y esperar).Entrega: enlace o archivo del proyecto más un documento de una página describiendo qué hace, cómo se usa, qué dificultades enfrentaste al construirlo y cómo las resolviste.